Data Science e Tecnologie per le Basi di Dati

Antonio Villella

Anno accademico 2026–2027

Indice

1 Introduzione alla Data Science e ai Big Data **7**
- 1.1 L'Evoluzione dei Sistemi di Gestione dei Dati . . . 7
  - 1.1.1 Sistemi Transazionali vs Sistemi Decisionali . . . 7
  - 1.1.2 Il Linguaggio SQL e i Sistemi Interni . . . 7
- 1.2 Il Contesto dei Big Data . . . 8
  - 1.2.1 Chi genera i Big Data? . . . 8
  - 1.2.2 Le 5 V dei Big Data . . . 8
- 1.3 Il Processo di Knowledge Discovery from Data (KDD) e Data Science . . . 9
  - 1.3.1 Generazione e Acquisizione . . . 10
  - 1.3.2 Memorizzazione (Storage) . . . 10
  - 1.3.3 Pre-processing e Preparazione del Dato . . . 10
  - 1.3.4 L'Analisi e le Tipologie di Algoritmi . . . 10
- 1.4 Tecniche di Machine Learning e Data Mining . . . 11
  - 1.4.1 Regole di Associazione (Association Rules) . . . 11
  - 1.4.2 Classificazione (Algoritmi Supervised) . . . 11
  - 1.4.3 Clustering (Algoritmi Unsupervised) . . . 11
  - 1.4.4 Altre Tecniche . . . 12
- 1.5 Sfide Aperte, Rappresentatività e Impatto Sociale . . . 12
  - 1.5.1 Bias nei Dati e Modelli Parziali . . . 12
  - 1.5.2 Monitoraggio e Concept Drift . . . 12
  - 1.5.3 Explainable AI (XAI) e Interpretabilità . . . 12

2 Introduzione ai Sistemi Decisionali e Data Warehouse **14**
- 2.1 Il Supporto alle Decisioni Aziendali . . . 14
  - 2.1.1 La Business Intelligence . . . 14
- 2.2 Il Data Warehouse: Definizione e Motivazioni . . . 15
  - 2.2.1 Perché mantenere i dati separati? . . . 15
- 2.3 Struttura e Analisi dei Dati: Il Modello Multidimensionale . . . 16
  - 2.3.1 Esempio applicativo: Catena di Supermercati . . . 16
  - 2.3.2 Rappresentazione Relazionale: Il Modello a Stella . . . 17
  - 2.3.3 Dimensionamento del Data Warehouse: Calcolo della Tabella dei Fatti 17
- 2.4 Strumenti di Analisi e Presentazione . . . 18
- 2.5 Architetture per Data Warehouse . . . 18
  - 2.5.1 Data Warehouse e Data Mart . . . 19
  - 2.5.2 Server per Data Warehouse . . . 19
  - 2.5.3 Il Processo ETL (Extraction, Transformation, Loading) e i Metadati . 20
  - 2.5.4 Tipologie di Architetture: 2 Livelli e 3 Livelli . . . 20

3 Progettazione di un Data Warehouse **22**
3.1 Fattori di Rischio e Approcci alla Progettazione . . . 22
3.1.1 Fattori di rischio . . . 22
3.1.2 Approcci: Top-Down vs Bottom-Up . . . 23
3.2 Analisi dei Requisiti . . . 23
3.2.1 Requisiti Applicativi . . . 24
3.2.2 Requisiti Strutturali . . . 24
3.3 Progettazione Concettuale: Dimensional Fact Model (DFM) . . . 24
3.3.1 Elementi Base del DFM . . . 25
3.3.2 Confronto: DFM vs Modello ER . . . 26
3.4 Misure e Operatori di Aggregazione . . . 26
3.4.1 Classificazione delle Misure . . . 26
3.4.2 Operatori di Aggregazione . . . 27
3.5 Costrutti Avanzati nel DFM . . . 27
3.5.1 Attributi Descrittivi e Gerarchie Condivise . . . 27
3.5.2 L'Arco Multiplo . . . 28
3.5.3 L'Attributo Configurazione . . . 28
3.5.4 Schemi di Fatto Vuoti . . . 28
3.6 La Rappresentazione del Tempo (Slowly Changing Dimensions) . . . 29
3.6.1 Rappresentazione di Tipo 1 (Sovrascrittura) . . . 29
3.6.2 Rappresentazione di Tipo 2 (Nuova Istanza) . . . 29
3.6.3 Rappresentazione di Tipo 3 (Campionamento Temporale) . . . 29
3.7 Dimensionamento del Data Warehouse: Carico di Lavoro e Volume dei Dati . 30
3.7.1 La Sparsità dei Dati . . . 30
3.8 Progettazione Logica: Il Modello Relazionale per il Data Warehouse . . . 30
3.8.1 Lo Schema a Stella (Star Schema) . . . 30
3.8.2 Lo Schema a Fiocco di Neve (Snowflake Schema) . . . 31
3.8.3 Traduzione Logica degli Archi Multipli . . . 32
3.8.4 Dimensioni Degeneri e Junk Dimension . . . 33

4 Analisi dei Dati in Ambienti OLAP ed Estensioni SQL **35**
4.1 Operazioni di analisi dei dati nel Data Warehouse . . . 35
4.2 Strumenti di interfaccia per l'interrogazione . . . 35
4.2.1 Ambiente controllato di query . . . 35
4.2.2 Ambiente di query ad hoc . . . 36
4.3 Analisi OLAP e Navigazione Multidimensionale . . . 36
4.3.1 Roll up . . . 36
4.3.2 Drill down . . . 38
4.3.3 Slice and dice . . . 39
4.3.4 Pivot . . . 41
4.4 Estensioni del linguaggio SQL . . . 42
4.5 Le Finestre di Calcolo (Window Functions) . . . 42
4.5.1 Elementi della clausola OVER . . . 42
4.5.2 Definizione dell'intervallo: Fisico vs Logico . . . 43
4.5.3 Applicazioni Pratiche ed Esempi . . . 43
4.6 Integrazione delle Finestre di Calcolo con la clausola GROUP BY . . . 45
4.7 Funzioni di Ranking (Classifiche) . . . 46
4.8 Estensioni della clausola GROUP BY . . . 47
4.8.1 L'operatore ROLLUP . . . 47
4.8.2 L'operatore CUBE . . . 48

4.8.3 L'operatore GROUPING SETS 48

5 Progettazione Fisica e Alimentazione del Data Warehouse 49
5.1 Progettazione Fisica 49
5.1.1 Caratteristiche del carico di lavoro nel Data Warehouse 49
5.1.2 Viste Materializzate 49
5.1.3 Altre Strutture Fisiche: Indici e Parallelismo 52
5.2 Alimentazione del Data Warehouse (Processo ETL) 53
5.2.1 Estrazione (Extraction) 53
5.2.2 Pulitura (Data Cleansing) 55
5.2.3 Trasformazione 55
5.2.4 Caricamento (Loading) 56

6 Estensioni al Linguaggio SQL in Oracle per l'Analisi dei Dati 57
6.1 Le Finestre di Calcolo 57
6.1.1 Raggruppamento a livello fisico (`ROWS`) 57
6.1.2 Raggruppamento a livello logico (`RANGE`) 58
6.2 Funzioni di Ranking 59
6.2.1 `RANK` e `DENSE_RANK` 59
6.2.2 Doppio Ranking 60
6.2.3 Selezione Top-N nel Ranking (Utilizzo di Table Function) 60
6.3 Ulteriori Funzioni per Finestre di Calcolo 61
6.3.1 `ROW_NUMBER` 61
6.3.2 `CUME_DIST` 61
6.3.3 `NTILE(n)` 62

7 Data Lakes 63

8 Data Mining Process 64

9 Data Preparation 65

10 Data Mining: Association Rules 66

11 Data Mining: Classificazione 67

12 Data Mining: Clustering 68

13 ESERCIZI 69

14 Interrogazioni Analitiche in SQL Esteso: Esercitazione Pratica 70
14.1 Osservazione preliminare: Ordinamento e selezione degli ultimi elementi . . . 70
14.2 Analisi dello Schema del Data Warehouse 70
14.2.1 Gerarchie e particolarità delle Dimensioni 71
14.3 Esercizio 1: Calcolo di aggregati e Ranking Globale 71
14.3.1 Testo della richiesta 71
14.3.2 Ragionamento e Passaggi Logici 71
14.3.3 Codice SQL 72
14.4 Esercizio 2: Ranking con Partizionamento 72
14.4.1 Testo della richiesta 72
14.4.2 Ragionamento e Passaggi Logici 72
14.4.3 Codice SQL 73

14.5 Esercizio 3: Ranking Multi-Dimensionale 73
14.5.1 Testo della richiesta 73
14.5.2 Ragionamento e Passaggi Logici 74
14.5.3 Codice SQL 74
14.6 Esercizio 4: Calcolo dell'Incasso Cumulativo (Running Total) 74
14.6.1 Testo della richiesta 74
14.6.2 Ragionamento e Passaggi Logici 75
14.6.3 Codice SQL 75

15 Progettazione di un Data Warehouse: Caso Studio Magazzini 77
15.1 Metodologia di Progettazione 77
15.2 Analisi delle Specifiche e Identificazione dei Fatti 78
15.3 Progettazione Concettuale: Schema a Stella 78
15.3.1 Fatto 1: Uso dei Magazzini 78
15.3.2 Fatto 2: Stima dei Costi delle Polizze 80
15.4 Verifica del Carico di Lavoro (Query Tipiche) 81
15.5 Progettazione Logica: Dallo Schema Concettuale alle Tabelle 81
15.5.1 Dimensioni Condivise (Conformed Dimensions) 81
15.5.2 Definizione delle Tabelle Relazionali 82

Introduzione al Corso e Modalità d'Esame

Descrizione dell'Insegnamento

L'insegnamento di Data Science e Tecnologie per le Basi di Dati è un corso obbligatorio per la Laurea Magistrale in Ingegneria Informatica. L'obiettivo principale del corso è descrivere gli aspetti fondamentali della tecnologia dei sistemi per la gestione delle basi di dati (DBMS) e le metodologie di gestione delle basi di dati destinate all'analisi (comunemente denominate data warehouse).

Questi sistemi si caratterizzano per la necessità di gestire enormi volumi di dati. Nel corso vengono esplorate sia le tecniche di analisi tradizionali, come l'analisi OLAP (On Line Analytical Processing), sia tecniche di analisi più complesse basate sul data mining.

Programma e Argomenti Trattati

Il programma del corso si articola in diverse macro-aree fondamentali per la gestione e l'analisi dei dati:

- **Caratteristiche tecnologiche dei DBMS:** vengono studiati la gestione dell'accesso concorrente ai dati, l'affidabilità del sistema, le strutture del livello fisico e l'ottimizzazione dell'accesso ai dati.
- **Basi di dati NoSQL:** introduzione al modello dei dati e ai linguaggi di interrogazione di sistemi non relazionali, con particolare riferimento a MongoDB ed Elastic.
- **Basi di dati distribuite:** studio dell'architettura dei sistemi distribuiti e delle relative tecniche di gestione della replicazione.
- **Data warehouse (DW):** analisi dell'architettura e delle metodologie di progettazione concettuale, logica e fisica. Vengono inoltre trattate la preparazione dei dati e la formulazione di interrogazioni OLAP tramite il linguaggio SQL.
- **Data science e Machine learning:** panoramica sul processo di data science e approfondimento dei principali algoritmi di apprendimento automatico per la classificazione, il clustering e l'estrazione di regole di associazione.

Struttura e Regole dell'Esame

L'esame si compone di una prova scritta obbligatoria e della valutazione di eventuali homework assegnati durante il semestre (elaborato progettuale individuale). Gli homework sono facoltativi, ma fortemente consigliati. A discrezione del docente, potrebbe essere richiesta una prova integrativa a conferma del voto ottenuto.

La Prova Scritta

La prova scritta si svolge in aula tramite PC, utilizzando la piattaforma di ateneo, e ha una durata complessiva di 100 minuti. Durante lo svolgimento non è permesso l'utilizzo di libri, appunti o dispositivi elettronici di alcun tipo.

Il test è composto da domande a risposta aperta e chiusa, oltre a esercizi pratici di progettazione. Le regole di valutazione delle domande prevedono che le risposte errate ai quesiti a risposta chiusa comportino una penalizzazione, mentre le risposte omesse valgano zero.

L*a struttura tipica della prova scritta è la seguente:

- **Domande teoriche (6-8 quesiti):** vertono sui concetti principali del corso, quali aspetti tecnologici dei DBMS, basi di dati distribuite, preparazione dei dati e algoritmi di analisi (massimo 8 punti).
- **Progettazione fisica (1-3 esercizi):** esercizi sulla creazione di strutture fisiche per basi di dati (massimo 3 punti).
- **Progettazione Data Warehouse (1-2 esercizi):** modellazione di un DW (massimo 5 punti).
- **Preparazione dei dati (1 esercizio):** operazioni di data preparation per data warehouse in linguaggio SQL (massimo 5 punti).
- **Interrogazioni OLAP (3 esercizi):** scrittura di istruzioni SQL per l'accesso e l'analisi dei dati di un data warehouse (massimo 11 punti).

Homework e Valutazione Finale

Durante il corso verranno proposti quattro homework individuali. Questi lavori sono mirati a sviluppare il pensiero critico e le capacità di problem solving sugli argomenti trattati a lezione.

Per quanto riguarda il calcolo del voto finale:

- Gli homework possono conferire un massimo di 2 punti aggiuntivi.
- Le relazioni degli homework vengono prese in considerazione per il voto finale esclusivamente se la valutazione della prova scritta è pari o superiore a 18/30.
- La lode viene assegnata nel caso in cui il punteggio complessivo superi il 31/30.

Capitolo 1

Introduzione alla Data Science e ai Big Data

1.1 L'Evoluzione dei Sistemi di Gestione dei Dati

La scelta del sistema di gestione di basi di dati (DBMS) dipende non solo dal volume di dati da memorizzare, ma soprattutto dall'obiettivo dell'analisi e da come questi dati devono essere processati.

1.1.1 Sistemi Transazionali vs Sistemi Decisionali

I database tradizionali (relazionali) sono progettati per supportare le attività ordinarie e operative. Ad esempio, le operazioni bancarie come il prelievo a uno sportello bancomat sono ripetitive e molto specifiche: il sistema deve identificare il cliente, verificare il saldo e autorizzare o meno la transazione. In questo contesto operativo (sistemi transazionali), è fondamentale garantire che un'operazione, se interrotta (es. mancata erogazione di contante), venga annullata per non generare errori nel saldo.

Se ci spostiamo su un sistema per il supporto decisionale, l'obiettivo cambia radicalmente. Un responsabile dei prodotti assicurativi di una grande banca non esegue query ripetitive su singoli clienti, ma interrogazioni complesse (spesso in SQL) su volumi di dati significativi, aggregando informazioni su base mensile e su diverse filiali per valutare l'andamento delle vendite rispetto al budget. L'originalità e la specificità delle query in ambito *analytics* richiedono un'architettura di database profondamente diversa rispetto a quella transazionale.

1.1.2 Il Linguaggio SQL e i Sistemi Interni

Il linguaggio SQL è un linguaggio *dichiarativo*: l'utente specifica *cosa* vuole ottenere (es. l'elenco dei clienti con saldo superiore a 50.000 euro) senza indicare *come* il sistema debba eseguire la ricerca. È il motore interno del DBMS che traduce questa richiesta dichiarativa in una sequenza di operazioni procedurali (es. scansione sequenziale o accesso tramite indice). Oggi l'evoluzione tecnologica ci impone di non utilizzare la stessa tecnologia per ogni problema. Accanto ai tradizionali sistemi relazionali, si utilizzano diffusamente sistemi NoSQL e Data Lake, ottimizzati per gestire dati strutturati, semi-strutturati e non strutturati, a seconda dei requisiti specifici dell'applicazione.

1.2 Il Contesto dei Big Data

Oggi la nostra capacità di generare dati è notevolmente superiore rispetto al passato. I dati raccolti, tuttavia, se meramente storicizzati rappresentano solo un costo di infrastruttura. Il valore si genera quando si è in grado di processarli ed estrarne conoscenza.

1.2.1 Chi genera i Big Data?

La generazione dei dati può essere classificata in base al grado di partecipazione dell'utente e alla tecnologia impiegata:

- **Generazione passiva (Log files e sistemi transazionali):** Riguarda tracce di operazioni che l'utente compie e che vengono storicizzate di default, come i pagamenti con carta di credito o i file di log di un server web o di uno smartphone. Questi log, ad esempio, descrivono la maggior parte delle abitudini quotidiane di un utente, rappresentando una risorsa fondamentale (e ponendo questioni di privacy).
- **Generazione attiva (User Generated Content):** L'utente diventa un produttore attivo di informazioni (post, foto, video su social network, dati di geolocalizzazione).
- **Produzione automatica (Internet of Things e Sensori):** Sensori, smart meter e dispositivi IoT generano campionamenti in modo continuo e automatico. Questo permette di creare modelli *data-driven* di fenomeni che prima venivano studiati solo attraverso modelli fisici (es. il degrado di un motore industriale monitorato dai suoi sensori).



Figura 1.1: L'evoluzione nella generazione dei dati: da un pubblico passivo nel 2005 a utenti generatori attivi di contenuti (video, foto, streaming) nel 2013.

1.2.2 Le 5 V dei Big Data

Il termine "Big Data" definisce dati la cui scala, diversità e complessità richiedono nuove architetture e algoritmi per estrarne valore. Essi sono caratterizzati dalle "5 V":

1. **Volume:** La quantità di dati cresce esponenzialmente. Molti algoritmi predittivi sono addestrati per funzionare bene proprio perché ricevono moli immense di dati in input. Un'interruzione di questo volume porta a un drastico calo di accuratezza. *Esempio:* Durante la pandemia di COVID-19, la drastica riduzione dei voli aerei ha privato i

modelli meteorologici dei dati raccolti dai sensori a bordo degli aerei, peggiorando temporaneamente le previsioni meteo.

2. **Velocity (Velocità):** Riguarda il tasso di generazione dei dati (streaming data). Una generazione veloce impone una conseguente velocità di elaborazione (*near real-time processing*) per restituire l'output in tempo utile. *Esempio:* Nelle app di navigazione GPS, i dati sul traffico vengono aggiornati costantemente dalle posizioni condivise dagli utenti. L'applicativo deve aggregare e processare rapidamente questi flussi per suggerire percorsi alternativi istantaneamente.

3. **Variety (Varietà):** Nelle applicazioni reali i dati provengono da flussi profondamente eterogenei (dati numerici, serie storiche, testi, audio, video) e presentano disallineamenti temporali e spaziali (granularità diverse). *Esempio (Emergency Management):* In un sistema di gestione delle emergenze, i dati di input combinano flussi video differiti da droni, rilevazioni meteo continue da sensori IoT, immagini satellitari e messaggi testuali/geolocalizzati dai social media (i cui picchi di pubblicazione dipendono dalle abitudini degli utenti e non sono costanti nel tempo).

4. **Veracity (Veracità):** È la sfida legata alla qualità, affidabilità e precisione del dato. Poiché la velocità di generazione è elevatissima, vi è un'alta probabilità di raccogliere dati "sporchi". Appellandosi al principio "Garbage In, Garbage Out", se si forniscono dati di scarsa qualità a un modello, i risultati saranno inaffidabili. Identificare anomalie e pulire i dati è uno dei compiti più onerosi (spesso richiede il 70-80% del tempo del progetto).

5. **Value (Valore):** L'obiettivo ultimo è tradurre i dati immagazzinati in un vantaggio. Questo vantaggio non è solo economico o di business, ma anche sociale (es. migliorare un servizio al cittadino o salvare vite in situazioni di rischio). Trasformare i bit in valore richiede l'immaginazione e la progettazione umana per combinare le tecnologie corrette.

1.3 Il Processo di Knowledge Discovery from Data (KDD) e Data Science

La Data Science si occupa dell'intero ciclo di vita del dato, dalla sua generazione fino alla trasformazione in conoscenza. Questo processo iterativo è noto come **KDD (Knowledge Discovery in Databases)** ed è composto dalle seguenti fasi logiche.

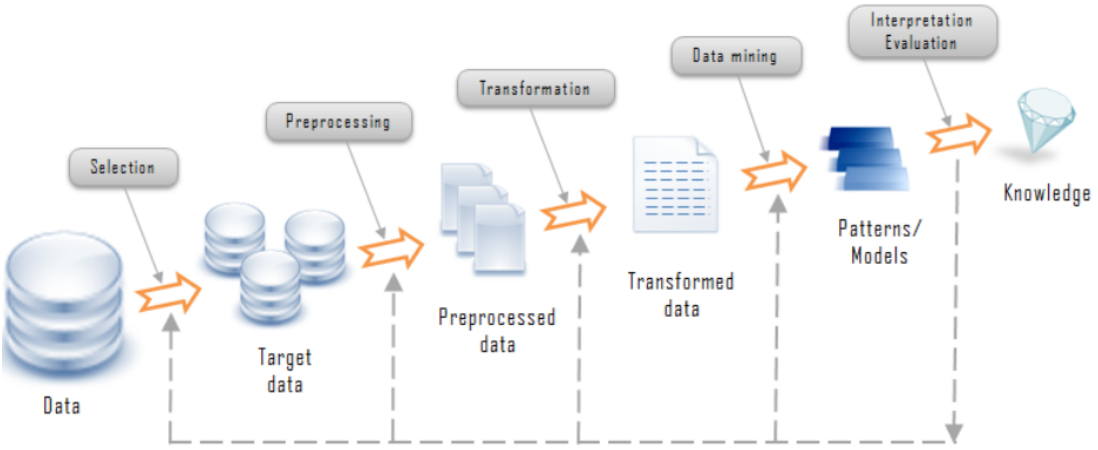


Figura 1.2: Fasi del processo iterativo di Knowledge Discovery from Data (KDD).

1.3.1 Generazione e Acquisizione

I dati generati devono essere acquisiti dal sistema tramite due approcci principali:

- **Pull-based:** Il sistema si attiva per recuperare i dati. Ad esempio, lo sviluppo di un *web crawler* che preleva da social network solo i post pubblici contenenti uno specifico hashtag di interesse.
- **Push-based:** I dati vengono inviati al sistema dal generatore. Ad esempio, applicazioni di videosorveglianza o raccolte *clickstream*. Le applicazioni *clickstream* tracciano i clic dell'utente durante la navigazione web (es. su un e-commerce), permettendo di profilarne gli interessi, la fascia oraria di attività e i comportamenti d'acquisto per alimentare sistemi di raccomandazione.

Durante la trasmissione su canali di rete, si attua un primissimo livello di pre-processing (es. filtro on-the-fly di misurazioni fisicamente impossibili, come 200°C rilevati nella stanza da un sensore guasto) per evitare di intasare lo storage con dati palesemente errati.

1.3.2 Memorizzazione (Storage)

A causa del volume e della varietà dei dati, una singola organizzazione utilizzerà contemporaneamente DBMS relazionali (per le transazioni aziendali strutturate) e sistemi NoSQL (documentali, key-value, colonnari o a grafo) ospitati su file system distribuiti o Data Lake, per memorizzare dati non strutturati.

1.3.3 Pre-processing e Preparazione del Dato

Si stima che un Data Scientist spenda l'80-90% del proprio tempo in questa fase, ben prima di scrivere il codice per l'algoritmo. Le sfide principali includono:

- **Data Cleaning:** Ridurre il rumore, rimuovere gli outlier e risolvere inconsistenze.
- **Data Integration e Trasformazione:** Reconciliare dati da fonti diverse e allineare la granularità. Ad esempio, se un braccio robotico emette dati al millisecondo, ma l'usura meccanica è apprezzabile solo su scala mensile, il dato al millisecondo va aggregato prima di alimentare l'algoritmo di manutenzione predittiva, altrimenti la mole immensa di dati non porterà a un modello accurato.

1.3.4 L'Analisi e le Tipologie di Algoritmi

Una volta che i dati sono puliti e preparati, si applicano modelli matematici per estrarre informazioni potenzialmente utili e precedentemente ignote. A seconda dell'obiettivo, le analisi si dividono in:

- **Descrittive (Characterization):** Raggruppano o descrivono i dati esistenti senza fare previsioni sul futuro. Rispondono alla domanda "Come sono composti i miei dati?".
- **Predittive:** Inferiscono regole dai dati passati per etichettare o calcolare un valore su dati futuri.
- **Prescrittive:** Combinano un algoritmo predittivo con una tecnica di ottimizzazione matematica. Non si limitano a prevedere un problema, ma prescrivono un'azione. *Esempio (Bike Sharing / EV Charging):* In un sistema di bike sharing, l'algoritmo predittivo calcola che, alle 18:00, la stazione X sarà vuota e la stazione Y sarà piena. Il livello prescrittivo prende questi input e calcola il percorso ottimo per i furgoni della manutenzione al fine di ribilanciare proattivamente le biciclette sul territorio.

1.4 Tecniche di Machine Learning e Data Mining

Le tecniche di estrazione automatica utilizzano algoritmi che generano modelli astratti (pattern). La scelta dell'algoritmo dipende fortemente dal tipo di dato, dalla complessità del problema e dalla necessità di interpretare il risultato.

1.4.1 Regole di Associazione (Association Rules)

Estraggono correlazioni frequenti all'interno di un database transazionale. Questa tecnica ammette dati di input di lunghezza variabile (es. scontrini con un numero di articoli diverso).

Il caso Walmart: Negli anni '90, per ottimizzare il layout dei grandi magazzini riducendo i tempi di permanenza alla ricerca dei prodotti, fu introdotta la tecnica delle regole di associazione (grazie ai ricercatori IBM). Il primo step fu il *pre-processing*: dallo scontrino fiscale reale (che contiene prezzo, quantità, sconti e nome esatto del prodotto) si estraggono solo le *categorie* degli oggetti acquistati. Dal database sintetizzato emerse la famosa regola: `Pannolini` $\Rightarrow$ `Birra`. Questa indicava un forte tasso di acquisto congiunto di questi due articoli nei carrelli, portando alla decisione commerciale di avvicinarli sugli scaffali. Oggi le regole di associazione sono alla base dei sistemi di raccomandazione ("Spesso comprati insieme") e dell'analisi cross-selling.

TID (ID Transazione)	**Items (Oggetti acquistati)**
1	Pane, Coca-Cola, Latte
2	Birra, Pane
3	Birra, Coca-Cola, Pannolini, Latte
4	Birra, Pane, Pannolini, Latte
5	Coca-Cola, Pannolini, Latte

Tabella 1.1: Esempio semplificato di Database Transazionale per regole di associazione.

1.4.2 Classificazione (Algoritmi Supervised)

La classificazione si utilizza per prevedere un'etichetta categorica. Richiede un dataset di addestramento (*training set*) annotato. L'algoritmo apprende dai dati storici annotati le relazioni tra gli attributi (ad es. il testo di un documento) e la relativa etichetta. Una volta addestrato, il modello può assegnare autonomamente l'etichetta a nuovi oggetti. *Esempio:* Uno strumento automatico che smista le pratiche e le richieste ricevute dai cittadini verso l'ufficio comunale di competenza, leggendo il contenuto del documento. A seconda del caso d'uso, un classificatore può essere nativamente interpretabile (come un Albero di Decisione o un insieme di regole) oppure opaco (come una Rete Neurale profonda).

1.4.3 Clustering (Algoritmi Unsupervised)

A differenza della classificazione, il clustering è una tecnica *non supervisionata*: non richiede un dataset preventivamente annotato da esperti. Il suo obiettivo è raggruppare (partizionare) gli oggetti in base a una metrica di distanza o similarità matematica; oggetti simili finiscono nello stesso cluster, oggetti dissimili in cluster separati. *Esempio:* In ambito marketing, gli algoritmi di clustering analizzano i pattern di consumo, il tempo di navigazione e le caratteristiche demografiche per segmentare la clientela in gruppi (es. studenti, professionisti, famiglie). Ogni cluster riceverà campagne promozionali mirate, superando l'approccio *broadcast* generalista.

1.4.4 Altre Tecniche

- **Regressione:** Specializzazione predittiva dove l'output atteso è un valore numerico continuo, non una classe (es. previsione del mercato azionario).
- **Sequence Mining:** Utilizza criteri di ordinamento spaziale e/o temporale (es. pattern ricorrenti nelle proteine).
- **Outlier Detection:** Tecniche finalizzate a individuare anomalie o deviazioni pesanti rispetto alla distribuzione normale, utilizzate pesantemente per identificare intrusioni informatiche (Network Intrusion Detection) o frodi bancarie.

1.5 Sfide Aperte, Rappresentatività e Impatto Sociale

Un progetto di Data Science fallisce se i dati non rappresentano in modo accurato e imparziale il dominio applicativo reale.

1.5.1 Bias nei Dati e Modelli Parziali

I modelli data-driven imparano esattamente dai dati che vengono forniti in input. Se il dataset è parziale, il modello apprenderà un pattern distorto e potrà generare discriminazioni. *Il caso Amazon (2013):* Amazon tentò di automatizzare un filtro per i CV candidati usando algoritmi addestrati sugli assunti dei 10 anni precedenti. Siccome il dataset storico rifletteva un settore a forte dominanza maschile, l'algoritmo dedusse arbitrariamente (e in modo discriminatorio) che le donne non fossero adatte per fare carriera tecnica nell'azienda e iniziò a scartare i CV femminili. Non è l'algoritmo a discriminare intenzionalmente, ma l'algoritmo ha semplicemente appreso la parzialità latente nel dato (bias). Per mitigare questi problemi si adottano tecniche di fine-tuning o si integrano i dataset reali con **dati sintetici** accuratamente calibrati, per bilanciare le classi e aumentare la rappresentatività del modello prima del training.

1.5.2 Monitoraggio e Concept Drift

Le distribuzioni dei dati possono evolvere nel tempo, richiedendo manutenzione e aggiornamento del modello. *Esempio (Il Golden Retriever):* Supponiamo di addestrare una rete per riconoscere le razze canine. Nel set di addestramento, quasi tutte le foto dei Golden Retriever hanno come sfondo un prato verde o mostrano il cane con un guinzaglio. Il modello funzionerà bene in laboratorio. Messo in produzione, il sistema riceverà foto di Golden Retriever sul divano, fallendo la classificazione. In realtà il modello non aveva imparato l'anatomia del cane, bensì aveva correlato "sfondo verde + guinzaglio = Golden Retriever". Quando si registra una deriva delle distribuzioni dei dati operativi (Concept Drift), è necessario eseguire un riaddestramento (o fine-tuning) utilizzando campioni di scenari aggiornati.

1.5.3 Explainable AI (XAI) e Interpretabilità

Con il proliferare di modelli *Black Box* (come le reti neurali), spesso accuratissimi ma internamente incomprensibili, è nata l'esigenza dell'Interpretabilità (Explainable AI), cruciale per applicazioni ad alto rischio (European AI Act) o ambito clinico. C'è un noto *trade-off* in letteratura tra l'accuratezza di un modello e la sua nativa interpretabilità. Tuttavia, fornire spiegazioni (almeno a livello locale su singole previsioni) è fondamentale.

Il caso Polmonite e Asma: Un gruppo di ricercatori americani studiò un dataset clinico per prevedere la probabilità di decesso in pazienti con polmonite. L'algoritmo interpretabile

(rule-based) generato dal sistema produsse una regola apparentemente folle: *"I pazienti asmatici hanno minori probabilità di morire di polmonite rispetto ai non asmatici"*. Grazie all'interpretabilità, fu facile scoprire il bias del dataset originario: negli ospedali americani studiati, un paziente asmatico ai primi sintomi di polmonite veniva immediatamente ricoverato in terapia intensiva, abbattendo il tasso di mortalità grazie a un trattamento ultra-intensivo tempestivo. Se i ricercatori avessero usato una rete neurale opaca, il sistema avrebbe appreso la stessa correlazione latente e avrebbe raccomandato, in fase produttiva, di de-prioritizzare le cure verso i pazienti asmatici con polmonite, provocando esiti letali. Questo sottolinea come l'interpretabilità sia l'unico scudo umano contro modelli tecnicamente performanti ma addestrati su distribuzioni di contesto parziali.

Capitolo 2

Introduzione ai Sistemi Decisionali e Data Warehouse

2.1 Il Supporto alle Decisioni Aziendali

In tutte le aziende, a prescindere dalle loro dimensioni, è presente un sistema informativo (o base di dati operazionale) progettato per garantire l'operatività quotidiana. Questi sistemi gestiscono le transazioni ordinarie di un'azienda: in un contesto assicurativo, ad esempio, il sistema gestisce la sottoscrizione di una polizza, i termini del contratto, il pagamento dei premi e la gestione di eventuali sinistri. Tali basi di dati, che potremmo definire "tradizionali", costituiscono una potenziale miniera di informazioni utili, ma sono progettate principalmente per supportare le operazioni di routine, non per l'analisi strategica.

Per poter garantire servizi efficienti e definire strategie vincenti, le aziende necessitano di prendere decisioni di tipo *data-driven* (guidate dai dati). Decisioni manageriali come l'introduzione di un nuovo prodotto sul mercato, la rimozione di un prodotto obsoleto, o l'analisi di una fetta di popolazione non ancora raggiunta dall'offerta, richiedono evidenze numeriche e oggettive. I **sistemi per il supporto alle decisioni** nascono esattamente per questo scopo: analizzare lo stato dell'azienda a partire dallo storico dei dati per prendere decisioni rapide, migliori e basate su fatti concreti (es. analizzare le vendite, l'indice di gradimento, il bilancio tra premi incassati e rimborsi erogati).

2.1.1 La Business Intelligence

La necessità di analizzare i dati aziendali sotto una "lente diversa" dà origine alla disciplina della **Business Intelligence (BI)**. Il termine *Intelligence* deriva dal latino *intus legere* (leggere dentro, comprendere in profondità). La Business Intelligence è la disciplina di supporto alla decisione strategica aziendale il cui obiettivo principale è la **trasformazione dei dati aziendali in informazioni fruibili**.

Queste informazioni devono essere analizzabili a **diversi livelli di dettaglio**. Ad esempio, un'azienda potrebbe voler analizzare le vendite di un prodotto per singola data, oppure aggregarle mensilmente per valutare l'evoluzione degli importi sottoscritti dall'inizio dell'anno. Questo permette di modulare le strategie di marketing o di rivedere il posizionamento del prodotto.

Affinché la Business Intelligence sia efficace per una tipologia di utenza eterogenea, è necessaria un'adeguata infrastruttura hardware e software dedicata. I campi di applicazione sono vastissimi e presentano tutti la stessa necessità di fondo:

- **Industrie manifatturiere**: gestione degli ordini, spedizioni e supporto clienti.

- **Distribuzione e Retail**: analisi del profilo utente e gestione del magazzino.
- **Servizi finanziari**: analisi degli acquisti tramite carte di credito, trasparenza finanziaria, antiriciclaggio.
- **Assicurazioni**: analisi delle richieste di indennizzo e riconoscimento delle frodi.
- **Telecomunicazioni**: analisi del traffico chiamate e individuazione di frodi.
- **Servizi pubblici e Sanità**: analisi dell'utilizzo delle risorse, degli accessi e dei risultati clinici.

2.2 Il Data Warehouse: Definizione e Motivazioni

Per supportare le attività decisionali, emerge la necessità di un nuovo sistema di memorizzazione e analisi, chiamato **Data Warehouse**. Secondo la definizione classica (W. H. Inmon, 1992), un Data Warehouse è una base di dati per il supporto alle decisioni, mantenuta separatamente dalle basi di dati operative dell'azienda, caratterizzata da dati:

- **Orientati ai soggetti di interesse**: strutturati per analizzare specifici temi aziendali (es. vendite, clienti) e non per supportare processi operativi.
- **Integrati e consistenti**: unificati a partire da fonti eterogenee.
- **Dipendenti dal tempo**: storicizzati per permettere analisi su ampi archi temporali (trend).
- **Non volatili**: una volta inseriti, i dati non vengono modificati dalle transazioni quotidiane, ma utilizzati in sola lettura per l'analisi.

2.2.1 Perché mantenere i dati separati?

Sebbene i dati analizzati nel Data Warehouse provengano (in gran parte) dai sistemi informativi aziendali, essi **devono essere mantenuti fisicamente separati** dai database operativi. Questa separazione è giustificata da due macro-motivazioni fondamentali:

1. Prestazioni (Separazione dei carichi di lavoro) Le query operative (sistemi OLTP - *Online Transaction Processing*) e le query analitiche (sistemi OLAP - *Online Analytical Processing*) hanno caratteristiche e impatti completamente diversi sul sistema.

- *Esempio operativo*: un utente preleva 100 euro al bancomat. Il sistema accede a pochissimi record, esegue una transazione rapida e aggiorna il saldo.
- *Esempio analitico*: un manager richiede un report aggiornato sull'andamento decennale di tutti i conti correnti di una filiale per valutare i costi del personale rispetto al volume d'affari.

Se queste due interrogazioni venissero eseguite sullo stesso database, si influenzerebbero negativamente a vicenda. La query analitica, dovendo leggere milioni di record ed effettuare calcoli complessi, bloccherebbe o rallenterebbe in modo inaccettabile l'operatività ordinaria (il prelievo al bancomat). Inoltre, i *metodi di accesso ai dati* sono ottimizzati in modo diverso a seconda che si debbano leggere pochi record specifici o scandire enormi volumi di dati per aggregazioni storiche.

2. Gestione e Struttura dei dati I dati operazionali non sono adatti per le analisi decisionali senza una preventiva rielaborazione. Nel Data Warehouse è necessario gestire:

- **Il consolidamento dei dati**: per un'analisi di trend, non interessa l'importo variabile del conto corrente durante le singole ore della giornata, ma la "fotografia" consolidata a fine giornata. I dettagli intra-giornalieri sono superflui.
- **Informazioni storiche e mancanti**: l'operazionale spesso sovrascrive i dati aggiornati, mentre il decisionale necessita della storicizzazione.
- **Qualità dei dati**: i dati estratti dai sistemi operativi devono essere puliti e resi coerenti prima dell'analisi.

2.3 Struttura e Analisi dei Dati: Il Modello Multidimensionale

Per modellare il Data Warehouse si utilizza il **modello multidimensionale**, un paradigma intrinsecamente orientato all'analisi. Quando si analizza un fenomeno aziendale, lo si osserva attraverso diverse *dimensioni di analisi*. I dati vengono quindi logicamente rappresentati come un **ipercubo** a tre o più dimensioni.

All'intersezione di queste dimensioni troviamo le **misure**, ovvero i valori numerici quantitativi su cui si effettuano i calcoli e le aggregazioni.

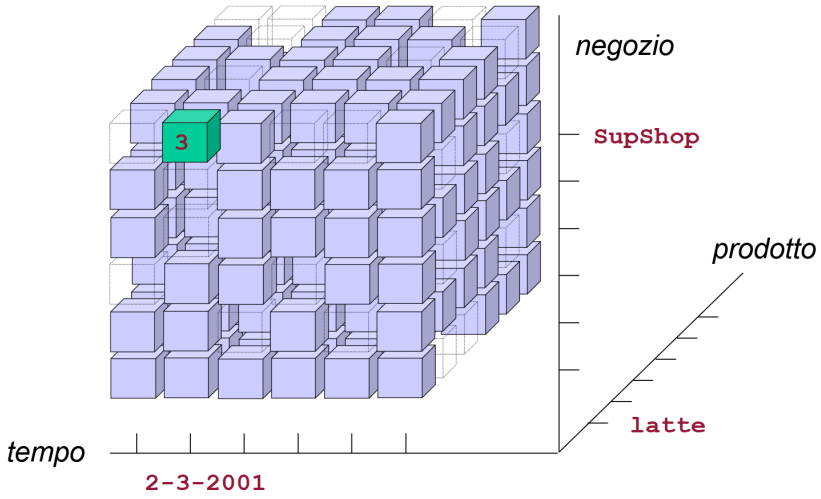


Figura 2.1: Rappresentazione logica a ipercubo. Ogni cubetto rappresenta una misura numerica all'intersezione di specifici valori per le dimensioni di analisi.

2.3.1 Esempio applicativo: Catena di Supermercati

Ipotizziamo di voler analizzare le vendite di una catena di supermercati. Le **Dimensioni di analisi** (gli assi dell'ipercubo) potrebbero essere:

- **Prodotto**: permette un'analisi per singolo articolo, per tipologia o per intera categoria merceologica.
- **Negozio (Spazio)**: permette l'analisi spaziale, confrontando il negozio X con la media dei negozi nella stessa città, provincia o regione.
- **Tempo**: permette di aggregare i dati su base giornaliera, mensile, semestrale o annuale.

Le **Misure** analizzate all'intersezione di queste dimensioni saranno valori numerici come la *quantità venduta* o l'*importo totale della vendita*. Ad esempio, il valore numerico contenuto nel cubetto corrispondente a "Succo di latte" (Prodotto), "Negozio A" (Negozio) e "10 Ottobre" (Tempo) potrebbe essere 3, indicando la quantità venduta in quel preciso contesto.

2.3.2 Rappresentazione Relazionale: Il Modello a Stella

Mentre l'ipercubo è il modello concettuale utilizzato per la visualizzazione e l'analisi intuitiva dei dati, per la *memorizzazione fisica* all'interno di un database relazionale si utilizza il cosiddetto **Modello a stella** (Star Schema).

In questo modello, le informazioni sono ripartite in due tipologie di tabelle:

1. **Tabella dei Fatti**: posizionata al centro dello schema, contiene le *misure numeriche* (es. quantità, importo) e le chiavi esterne per collegarsi alle dimensioni.
2. **Tabelle Dimensionali**: posizionate a raggiera attorno alla tabella dei fatti. Contengono gli *attributi descrittivi* (spesso numerosi e gerarchici) che descrivono il contesto di ogni misura.

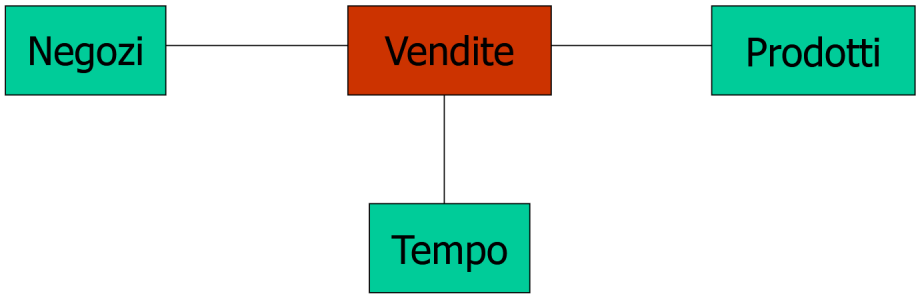


Figura 2.2: Rappresentazione relazionale tramite Modello a Stella. La tabella delle vendite (fatti) è collegata alle dimensioni (negozio, prodotto, tempo).

Osservazione sul livello di dettaglio: In un database operazionale troveremmo il dettaglio puntuale di ogni singolo scontrino fiscale. Nel Data Warehouse, essendo orientato all'analisi, il livello di granularità (ad esempio giornaliero) fa sì che il dettaglio del singolo scontrino venga aggregato per ottenere il totale venduto di quel prodotto in quel negozio in quella specifica giornata.

Osservazione sulla progettazione (Denormalizzazione): Durante la lezione il professore ha chiarito un punto fondamentale rispondendo a una domanda. Per memorizzare questo schema in un database, si creeranno quattro tabelle (Fatti-Vendite, Negozio, Tempo, Prodotti). Tuttavia, a differenza del modello relazionale classico studiato per l'operazionale (che richiede una rigorosa normalizzazione in 3NF per evitare anomalie di aggiornamento), nel Data Warehouse **le tabelle dimensionali vengono volutamente denormalizzate**. Questo viene fatto per semplificare le operazioni di lettura (riducendo il numero di `JOIN`) e ottimizzare le query, poiché i dati del DW sono in sola lettura e non subiscono frequenti aggiornamenti transazionali.

2.3.3 Dimensionamento del Data Warehouse: Calcolo della Tabella dei Fatti

Per comprendere i volumi di dati gestiti, stimiamo la dimensione della tabella dei fatti per la catena di supermercati ipotizzando un orizzonte temporale di 2 anni.

Parametri delle dimensioni:

- Dimensione **Tempo**: 2 anni $\times$ 365 giorni/anno $=$ 730 record.
- Dimensione **Negozio**: 300 record.

- Dimensione **Prodotto**: il catalogo totale contiene 30.000 prodotti.

Calcolo dell'Upper Bound (Limite superiore teorico): Se assumessimo che ogni singolo giorno, in ogni negozio, venga venduto almeno un pezzo di tutti i 30.000 prodotti a catalogo, il numero di righe generato nella tabella dei fatti sarebbe:

$$N_{\max} = 730 \times 300 \times 30.000 = 6.570.000.000 \text{ di righe}$$

Calcolo Realistico (Sparsità dei dati): Nella realtà, la matrice dei dati è *sparsa*. È irrealistico pensare che tutti i prodotti siano venduti ogni giorno ovunque. Si introduce quindi un'ipotesi statistica coerente con il business: ipotizziamo che, dei 30.000 a catalogo, in ogni negozio vengano venduti giornalmente solo 3.000 prodotti diversi. Il numero di righe stimato diviene quindi:

$$N_{\text{reale}} = 730 \text{ (giorni)} \times 300 \text{ (negozi)} \times 3.000 \text{ (prodotti venduti)} = 657.000.000 \text{ di righe}$$

Con uno spazio di archiviazione stimato per la sola tabella dei fatti di circa 21 GB (che tenderà a crescere qualora la stima della sparsità si rivelasse troppo prudente).

2.4 Strumenti di Analisi e Presentazione

Estrarre dati massivi è inutile se non vengono forniti gli strumenti per interpretarli. Gli strumenti di *Analytics* e visualizzazione operano su due fronti:

1. Analisi dei Dati:

- **Analisi OLAP (Online Analytical Processing)**: serve per il calcolo di funzioni aggregate complesse. Fornisce supporto per estrarre statistiche come la media mobile o le classifiche "top ten" in modo efficiente.
- **Data Mining**: applica tecniche algoritmiche pesanti e varie tipologie di analisi per scoprire pattern nascosti (es. regole di associazione, clustering).

2. Presentazione e Visualizzazione (Cruciale per le decisioni): La visualizzazione è fondamentale perché l'intuizione umana coglie trend ed eccezioni in un grafico in modo molto più immediato rispetto alla lettura di una tabella di numeri. Gli strumenti non devono limitarsi a restituire grafici statici, ma garantire una **visualizzazione esplorativa e navigabile** per favorire la ricerca di motivazioni. Ad esempio, attraverso l'operazione di **Drill-Down**, un manager può partire da un grafico aggregato del fatturato annuale, cliccare sull'anno e "scendere" al livello di dettaglio mensile o giornaliero, per comprendere quali fenomeni stiano generando un'anomalia in quel dato sintetico.

2.5 Architetture per Data Warehouse

Per implementare un sistema del genere, l'infrastruttura deve garantire la netta **separazione tra l'elaborazione transazionale (OLTP) e l'analisi dei dati (OLAP)**. Le architetture a un solo livello sono pertanto da scartare, preferendo architetture a due o più livelli che risultano maggiormente scalabili.

2.5.1 Data Warehouse e Data Mart

All'interno dei sistemi decisionali si distinguono due entità, basate sulla perimetrazione aziendale del dato:

- **Warehouse Aziendale**: contiene le informazioni sul funzionamento di *tutta* l'azienda. Il processo di modellazione è esteso e la sua realizzazione richiede molto tempo e risorse.
- **Data Mart**: è un sottoinsieme dipartimentale focalizzato su un settore specifico (es. solo l'ufficio marketing, o solo la gestione del personale). Può essere alimentato derivandolo dal Data Warehouse aziendale (approccio top-down) oppure direttamente dalle sorgenti (approccio bottom-up). Ha una realizzazione più rapida, ma richiede un'attenta progettazione preventiva per evitare problemi di integrazione futura tra Data Mart diversi.

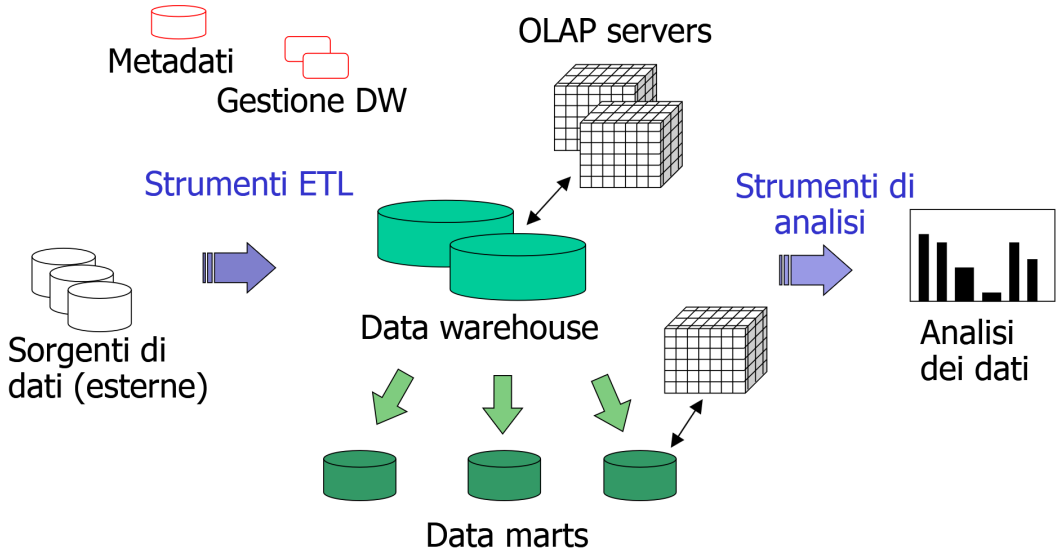


Figura 2.3: Visione d'insieme degli elementi costitutivi e del flusso dei dati in un'architettura Data Warehouse.

2.5.2 Server per Data Warehouse

I dati del Data Warehouse sono memorizzati su Server ottimizzati che possono adottare diverse tecnologie sottostanti, sebbene offrano la medesima interfaccia multidimensionale all'utente finale:

- **Server ROLAP (Relational OLAP)**: è un DBMS relazionale esteso. Utilizza tabelle (come il modello a stella) per rappresentare in modo compatto dati sparsi. Implementa estensioni SQL specifiche per gli aggregati e metodi di accesso fisici ottimizzati.
- **Server MOLAP (Multidimensional OLAP)**: i dati sono salvati nativamente in forma matriciale/multidimensionale con array proprietari. Poiché i dati sono sparsi (molte celle vuote nell'ipercubo), questo sistema richiede pesanti algoritmi di compressione. Offre primitive speciali OLAP molto veloci.
- **Server HOLAP (Hybrid OLAP)**: unisce le due tecnologie precedenti.

2.5.3 Il Processo ETL (Extraction, Transformation, Loading) e i Metadati

Il cuore dell'alimentazione del Data Warehouse è il processo ETL, eseguito sia durante il primo popolamento sia nei successivi aggiornamenti periodici. Consiste in tre fasi:

1. **Estrazione (Extraction)**: acquisizione e selezione dei dati grezzi dalle fonti originarie (sistemi interni o dati esterni es. social).
2. **Trasformazione e Pulitura (Transformation)**: è la fase più complessa di pre-elaborazione. I dati vengono rielaborati, resi coerenti (risolvendo inconsistenze e formati eterogenei tra fonti diverse) e integrati per passare dal formato operazionale a quello adatto all'analisi.
3. **Caricamento (Loading)**: memorizzazione fisica dei dati puliti all'interno del Data Warehouse e propagazione dei successivi aggiornamenti.

Per gestire l'ETL e l'intero ecosistema è indispensabile gestire i **Metadati** (dati sui dati). Essi descrivono:

- *La trasformazione e il caricamento*: descrivono lo schema logico-concettuale delle sorgenti, le trasformazioni effettuate, il formato, la provenienza. Uno standard proposto per l'interscambio è il **CWMI** (Common Warehouse Metadata Initiative).
- *La gestione dei dati*: descrivono la struttura dello schema logico del Data Warehouse stesso e delle viste materializzate create.
- *La gestione delle query*: mantengono il codice SQL, i piani di esecuzione e l'uso di memoria/CPU per monitorare le performance del sistema.

2.5.4 Tipologie di Architetture: 2 Livelli e 3 Livelli

Sulla base della complessità del processo ETL, si distinguono due architetture principali.

Architettura a 2 livelli (2-Tier) Il sistema è diviso concettualmente in due aree:

1. Livello delle sorgenti.
2. Livello del Data Warehouse (che riceve i dati ed è interrogato dai tool OLAP).

In questo scenario, il processo ETL agisce "al volo" (*on the fly*). Estrae, pulisce e carica i dati direttamente nel DW. È una soluzione adatta a processi di integrazione semplici. Garantisce il disaccoppiamento dalle sorgenti, alleggerisce il carico transazionale e gestisce bene le differenti granularità temporali, ma rischia di essere fragile in caso di errori durante le trasformazioni lunghe.

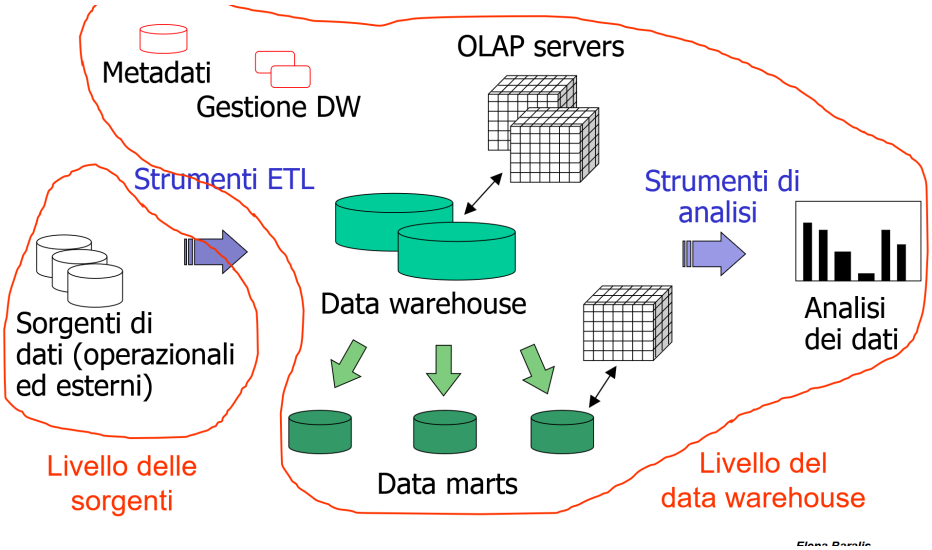


Figura 2.4: Architettura Data Warehouse a due livelli. L'elaborazione ETL avviene contestualmente al trasferimento verso il livello di analisi.

Architettura a 3 livelli (3-Tier) Introdotta per realtà organizzative grandi e complessi processi ETL. I livelli diventano tre:

1. Livello delle sorgenti.
2. **Livello di alimentazione (Staging Area / Operational Data Store)**: un'area di transito e di lavoro intermedia.
3. Livello del Data Warehouse.

Il vantaggio cruciale spiegato dal professore risiede nel **checkpoint**. La *Staging area* permette di separare logicamente e temporalmente l'estrazione e trasformazione dei dati dal caricamento finale. I dati pre-elaborati e puliti vengono prima salvati in quest'area (che ha un modello dati integrato ancora vicino all'operazionale). Successivamente avviene la fase di *Loading*. Se durante il caricamento (*loading*) nel Data Warehouse si verifica un errore, il sistema non è costretto a ripetere dall'inizio le onerose operazioni di estrazione e calcolo effettuate sui database sorgenti; è sufficiente ripartire dai dati salvati nella *staging area*. Il rovescio della medaglia è l'introduzione di un'ulteriore ridondanza fisica, che aumenta lo spazio su disco necessario.

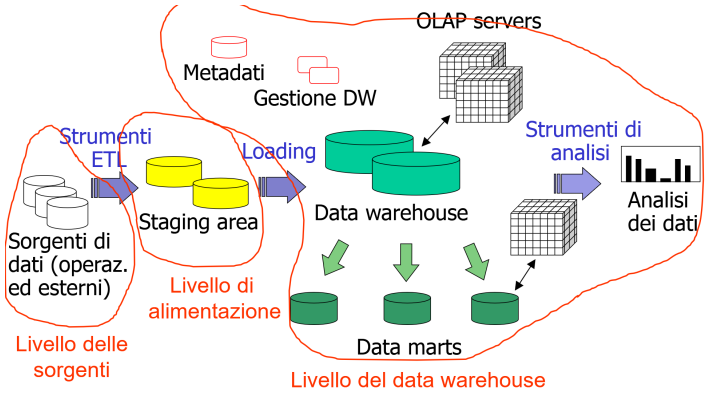


Figura 2.5: Architettura Data Warehouse a tre livelli. L'introduzione della Staging Area agisce da area di lavoro e checkpoint.

Capitolo 3

Progettazione di un Data Warehouse

La decisione di un'azienda di dotarsi di un sistema di data warehouse per la business intelligence (o per il supporto strategico e decisionale) è un passo fondamentale. Sebbene oggi sia una scelta diffusa non solo tra le grandi aziende ma anche in contesti di medie dimensioni, l'adozione di questi strumenti non risolve automaticamente tutti i problemi aziendali. L'obiettivo principale è ottenere evidenze di tipo *data-driven* sullo stato dell'azienda, ma il successo del sistema dipende da una corretta progettazione, dalla qualità dei dati e dalla collaborazione tra attori con visioni differenti.

Un'architettura tipica per il supporto decisionale è composta da almeno due livelli separati: il livello delle sorgenti (sistemi transazionali/operazionali) e il livello di organizzazione delle informazioni per le analisi (il data warehouse vero e proprio). Mantenere separati i sistemi operazionali dai sistemi decisionali è cruciale per ragioni di efficienza e prestazioni. Spesso si adotta un'architettura a tre livelli, introducendo un livello intermedio di alimentazione (con una *staging area*) in cui avvengono i processi di ETL (Estrazione, Trasformazione e Caricamento).

3.1 Fattori di Rischio e Approcci alla Progettazione

3.1.1 Fattori di rischio

La realizzazione di un data warehouse comporta dei rischi intrinseci dovuti alla necessità di far comunicare due "mondi" separati:

- **Gestione "politica" del progetto e collaborazione:** Da un lato vi sono gli amministratori dei DBMS tradizionali (sistemi informativi), abituati a logiche dove ogni dato deve essere memorizzato al massimo livello di dettaglio senza ridondanze. Dall'altro vi sono gli utenti di business (decision maker), che necessitano di dati aggregati per l'esplorazione e non conoscono a priori il livello di dettaglio di cui avranno bisogno. L'accettazione del sistema da parte degli utenti finali e la collaborazione dei "detentori" dell'informazione è vitale.
- **Aspettative elevate:** Esiste il rischio che gli utenti vedano il data warehouse come la panacea di tutti i problemi aziendali. Il sistema sarà invece efficace se e solo se nelle sorgenti esistono i dati corretti per alimentare le analisi richieste.
- **Qualità dei dati e processi di partenza:** I dati operazionali possono essere incompleti o inaffidabili, e i processi aziendali potrebbero non essere integrati o ottimizzati, compromettendo la base stessa su cui si fonda il supporto decisionale.

3.1.2 Approcci: Top-Down vs Bottom-Up

Per realizzare un data warehouse si possono seguire due approcci principali:

Approccio Top-Down

Consiste nel realizzare un data warehouse che fornisca fin da subito una visione globale e completa di tutti i dati aziendali. Si ha in mente l'intero spettro delle analisi necessarie per tutta l'azienda e si progetta il sistema complessivo.

- **Svantaggi:** Richiede un costo economico e temporale molto significativo, con analisi e progettazioni estremamente complesse. Ad esempio, pensare di creare con un approccio top-down un data warehouse per l'intero Politecnico di Torino (con dipartimenti, amministrazione, decine di migliaia di studenti e personale) è un progetto troppo ambizioso e di lungo termine.
- **Utilizzo:** In contesti medio-grandi è generalmente sconsigliato a causa della necessità di una visione troppo estesa.

Approccio Bottom-Up

Consiste in una realizzazione incrementale. Si parte da un settore aziendale specifico (un problema più circoscritto, seppur grande) e si realizza un **data mart**, ovvero un piccolo data warehouse specializzato per quel contesto (es. gestione del magazzino, gestione delle vendite).

- **Vantaggi:** Il costo e il tempo di consegna sono contenuti. I vari data mart, che inizialmente possono anche essere indipendenti, verranno poi aggregati e integrati in fasi successive per formare il data warehouse aziendale completo.

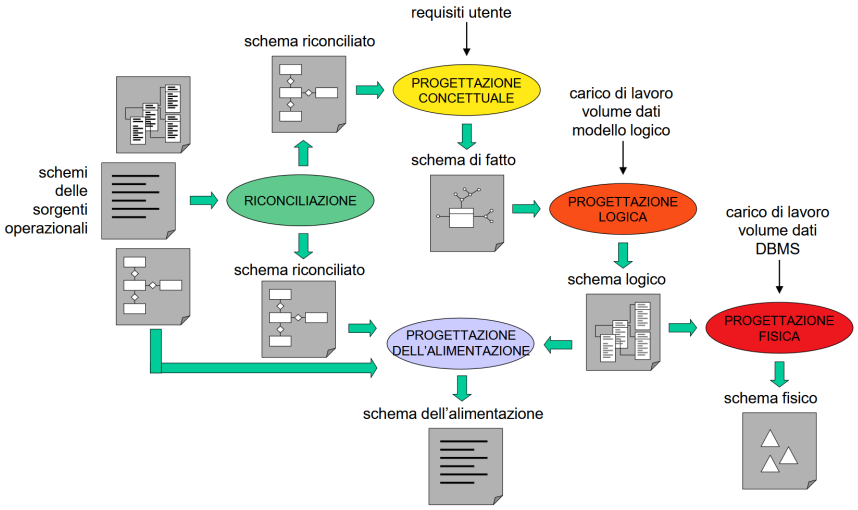


Figura 3.1: Approccio Bottom-Up: si parte da un data mart specifico e si procede all'integrazione con altri data mart per formare il data warehouse completo.

3.2 Analisi dei Requisiti

Il processo di realizzazione di un data mart (approccio bottom-up) inizia sempre dall'analisi dei requisiti. Si tratta di comprendere quali sono le analisi di interesse per gli attori di

business (che variano a seconda del ruolo, es. direttore vendite vs capo del personale) e identificare i dati necessari per supportarle.

La raccolta dei requisiti avviene tipicamente tramite **interviste** che coinvolgono sia i *business users* (utilizzatori finali) sia gli amministratori del sistema informativo. Di norma, il primo data mart sviluppato riguarda un settore strategico dell'azienda per dimostrare fin da subito l'efficacia del progetto.

La progettazione si basa su tre pilastri (pillar) fortemente interconnessi:

1. **Tecnologia:** Quale DBMS usare, spazio di archiviazione, livello di affidabilità.
2. **Dati:** Quali dati servono, se sono presenti nelle sorgenti interne o se vanno integrati con fonti esterne, e come effettuare l'integrazione.
3. **Applicazioni:** Che tipo di applicativo utilizzerà l'utente finale (dashboard interattiva, algoritmi predittivi, soluzioni agentiche basate su LLM/RAG). La tecnologia scelta deve supportare l'applicazione desiderata.

I requisiti si dividono in due grandi categorie: applicativi e strutturali.

3.2.1 Requisiti Applicativi

Definiscono il "che cosa" si vuole analizzare.

- **Descrizione degli eventi di interesse (Fatti):** Un fatto rappresenta una categoria di eventi aziendali (es. reclami, quantità venduta, giacenze di magazzino). È caratterizzato da coordinate di analisi (dimensioni) e da indicatori numerici (misure o KPI).
- **Carico di lavoro:** Quali e quanti attori accedono al sistema, la frequenza di generazione dei report, il numero di sessioni in contemporanea. Queste informazioni permettono di dimensionare opportunamente il sistema.

3.2.2 Requisiti Strutturali

Riguardano l'infrastruttura e i vincoli operativi.

- **Periodicità dell'alimentazione:** Il data warehouse necessita di un caricamento periodico (ETL). Solitamente avviene nelle ore notturne, quando il carico dei sistemi informativi e del data warehouse è minimo.
- **Spazio disponibile:** Sia per i dati grezzi che per strutture accessorie necessarie a velocizzare le interrogazioni (indici, viste materializzate).
- **Tipo di architettura:** A due o tre livelli, e definizione se i data mart saranno dipendenti (condividono dimensioni di analisi) o indipendenti.

3.3 Progettazione Concettuale: Dimensional Fact Model (DFM)

Dopo aver raccolto i requisiti (e aver definito uno schema riconciliato delle sorgenti nel caso di sistemi informativi eterogenei), si passa alla progettazione concettuale. In questo ambito, il **modello Entità-Relazione (ER) non è adatto**. Il modello ER nasce per i DBMS operazionali: il suo scopo è normalizzare i dati per evitare ridondanze e prevenire anomalie di aggiornamento. Tuttavia, uno schema normalizzato frammenta l'informazione; per ricostruirla in fase di interrogazione analitica (su milioni di record) sarebbero necessarie numerose operazioni di `JOIN`, rendendo il sistema inefficiente.

Nel data warehouse **rinunciamo alla normalizzazione e accettiamo la ridondanza** per ottenere query efficienti. Il modello concettuale di riferimento è il **Dimensional Fact Model (DFM)**, introdotto da Golfarelli e Rizzi, che offre una rappresentazione grafica basata su fatti, dimensioni e misure.

3.3.1 Elementi Base del DFM

- **Fatto:** Modella un insieme di eventi di interesse (es. vendite). Nel DFM è rappresentato dal nodo centrale. Solitamente, in un modello ER equivalente, il fatto rappresenta una relazione molti-a-molti (ennaria) tra tutte le dimensioni coinvolte (es. lo stesso prodotto può essere venduto in più negozi e in più date).
- **Dimensione:** Descrive le coordinate di analisi di un fatto (es. Negozio, Prodotto, Data). Le dimensioni sono tendenzialmente ortogonali tra loro.
- **Misura (o Metrica):** Una proprietà numerica associata al fatto, oggetto di operazioni di aggregazione (es. quantità venduta, incasso, numero di clienti).
- **Gerarchia:** Sulle dimensioni è possibile definire delle gerarchie (es. Data → Mese → Trimestre → Anno). Esse rappresentano generalizzazioni di concetti e modellano una **dipendenza funzionale** (relazione 1:N).

Le Gerarchie e le Dipendenze Funzionali

Nel DFM, un arco in una gerarchia rappresenta una dipendenza funzionale, ovvero, dato l'attributo di partenza, esiste *uno e un solo* valore per l'attributo di destinazione. L'arco va tracciato solo verso l'attributo meno generale immediatamente successivo. Ad esempio, esiste una dipendenza funzionale tra *Data* e *Anno*, ma l'arco viene disegnato da *Data* a *Mese*, e da *Mese* a *Trimestre*, e da quest'ultimo ad *Anno*. Inoltre, non si traccia un arco tra *Giorno della settimana* (lunedì, martedì...) e *Anno*, perché un "lunedì" non appartiene univocamente a un solo anno (non c'è dipendenza funzionale 1:N).

Un'attenzione particolare va posta al concetto temporale nel business: quando si parla di "analisi per mese", si intende quasi sempre il mese in un determinato anno (es. Settembre 2026), distinto dal mese di un altro anno. Se l'obiettivo fosse analizzare tutti i mesi di settembre a prescindere dall'anno, si parlerebbe di "mese dell'anno".

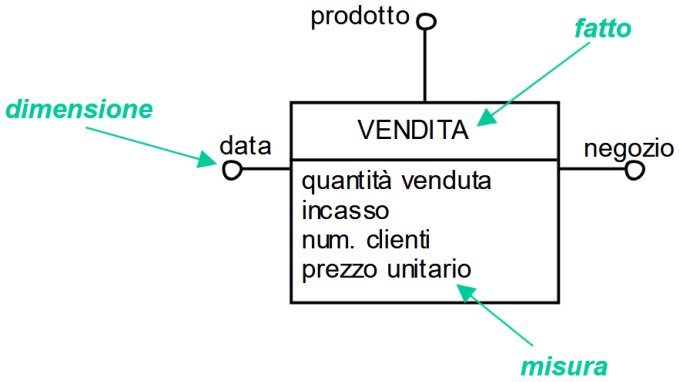


Figura 3.2: Esempio di Dimensional Fact Model per le vendite, con dimensioni (Negozio, Prodotto, Data), gerarchie (es. Negozio → Città) e misure (Qta, Incasso).

3.3.2 Confronto: DFM vs Modello ER

Se traducessimo un fatto in un modello ER, esso diventerebbe un'entità associativa centrale che collega con relazioni N:1 le varie entità dimensionali, le quali a loro volta si esploderebbero a cascata a causa delle dipendenze funzionali (gerarchie) in una miriade di tabelle (forma normale). Il DFM "collassa" visivamente e logicamente queste gerarchie.

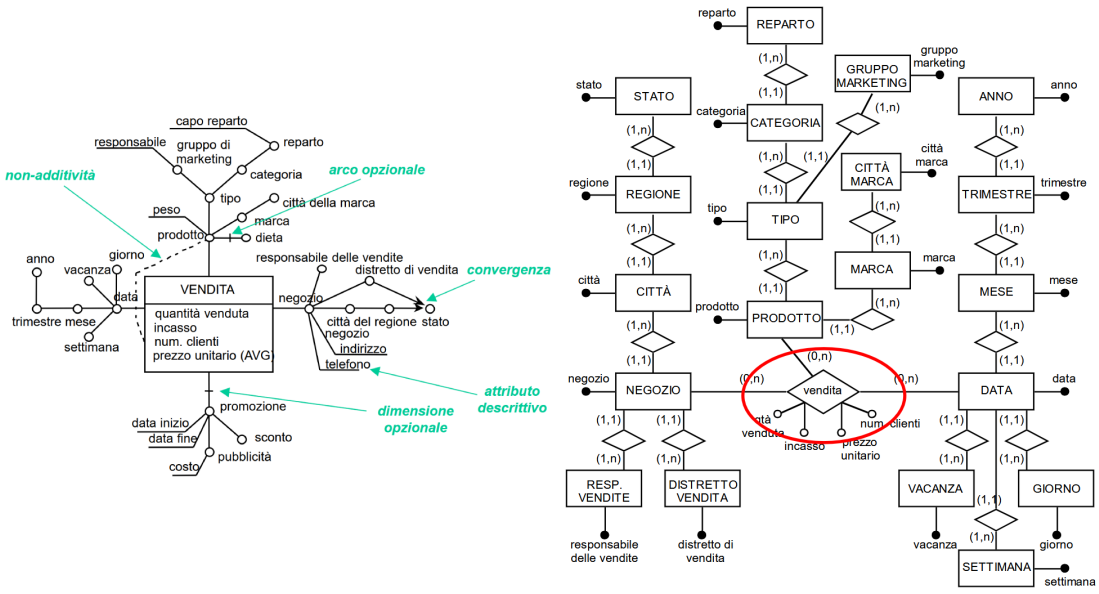


Figura 3.3: Confronto tra la notazione compatta del DFM (a sinistra) e la complessità di un modello ER equivalente (a destra) dove tutte le dipendenze funzionali sono esplicitate in tabelle separate.

3.4 Misure e Operatori di Aggregazione

Il processo tipico in un data warehouse è l'analisi a diversi livelli di dettaglio, che corrisponde in SQL all'applicazione della clausola `GROUP BY` navigando lungo le gerarchie. È fondamentale capire quali misure possono essere aggregate e come.

3.4.1 Classificazione delle Misure

Le metriche si dividono in tre categorie in base alla loro additività (la capacità di essere sommate lungo le dimensioni, specialmente il tempo):

1. **Misure di flusso:** Possono essere valutate cumulativamente alla fine di un periodo di tempo. Sono **sempre additive** lungo tutte le dimensioni.
    - *Esempi:* Quantità di prodotti venduti, importo incassato. Se vendo 10 oggetti oggi e 20 domani, in totale ho venduto 30 oggetti.
2. **Misure di livello:** Rappresentano un'istantanea (snapshot) di un fenomeno in un preciso istante di tempo. **Non sono additive lungo la dimensione temporale**.
    - *Esempi:* Giacenza di magazzino, saldo del conto corrente. Se oggi ho 100€ sul conto e domani ho 100€, la somma 200€ non ha alcun significato; il valore "fotografa" lo stato attuale. Tuttavia, *possono* essere additive lungo lo spazio (es. somma delle giacenze di tre magazzini diversi nello stesso istante).

3. **Misure unitarie:** Valutate in specifici istanti ed espresse in termini relativi. **Non sono additive** lungo nessuna dimensione.

    - *Esempio:* Prezzo unitario di un prodotto.

Un'ulteriore eccezione all'additività si ha quando l'aggregazione su una dimensione genera duplicati di conteggio. Ad esempio, la metrica *Numero Clienti* calcolata come numero di scontrini: è sommabile nel tempo, ma **non è additiva rispetto ai Prodotti**. Poiché uno stesso cliente può comprare più prodotti in uno stesso scontrino, aggregare i clienti per prodotto porterebbe a contare la stessa persona più volte (overlap). Sul DFM questa non-additività va esplicitata graficamente.

3.4.2 Operatori di Aggregazione

A seconda di come è possibile calcolare aggregazioni partendo da dati già pre-aggregati a un livello di dettaglio maggiore, gli operatori si classificano in:

- **Distributivi:** Il calcolo finale può sfruttare risultati aggregati intermedi. (es. `SUM`, `MIN`, `MAX`). *Esempio:* Per calcolare le vendite annuali, posso sommare i parziali delle vendite trimestrali già calcolati. Questo abbassa drasticamente i tempi di calcolo.
- **Algebrici:** Il risultato intermedio da solo non basta, ma è possibile calcolarlo se affiancato da misure di supporto. (es. `AVG`). *Motivazione:* La media delle medie è matematicamente scorretta. Per aggregare una media a un livello superiore, il data warehouse non dovrebbe memorizzare la "media" come metrica secca, ma memorizzare le metriche di base che la compongono (es. l'incasso e il numero di prodotti) per ricalcolarla in SQL come `SUM(Incasso) / SUM(Num_Prodotti)`.
- **Olistici:** Non è possibile sfruttare alcun aggregato parziale; è necessario ricalcolare la funzione partendo dai dati originali di massimo dettaglio. (es. Moda, Mediana).

3.5 Costrutti Avanzati nel DFM

La modellazione concettuale deve saper gestire casi della realtà aziendale più complessi.

3.5.1 Attributi Descrittivi e Gerarchie Condivise

- **Attributi descrittivi:** Sono caratteristiche associate a una dimensione su cui però *non si fanno aggregazioni* (es. Indirizzo, Telefono del negozio). Si rappresentano graficamente con una linea semplice (senza cerchio).
- **Gerarchia Condivisa:** Spesso una stessa dimensione assume "ruoli" multipli nello stesso fatto. L'esempio classico è il traffico telefonico: una chiamata ha un *Numero Chiamante* e un *Numero Chiamato*. Entrambi puntano all'entità Numero Telefonico (che possiede una sua gerarchia, es. Distretto, Uso). Nel DFM si tracciano due frecce separate dal Fatto alla dimensione, specificando il ruolo su ciascuna freccia. Altri esempi: aeroporto di partenza e destinazione in un volo; mittente e destinatario in una spedizione o in un bonifico.

3.5.2 L'Arco Multiplo

Come abbiamo visto, la gerarchia modella dipendenze funzionali 1:N. Ma cosa succede quando tra due attributi dimensionali esiste una **relazione Molti-a-Molti**? Si utilizza l'arco multiplo, rappresentato da due linee convergenti.

Casi d'uso:

1. **Libri e Autori:** Vogliamo analizzare le vendite di libri. Un libro può essere scritto da più autori, un autore scrive più libri. Date le vendite del "Libro X", ad esso è associato un numero indefinito di autori.

2. **Ricoveri e Diagnosi:** Un paziente in pronto soccorso può avere un "Gruppo di diagnosi" che racchiude N patologie diverse. Associando il costo del ricovero all'intero gruppo (più specifico) e connettendo il gruppo alla singola diagnosi (tramite arco multiplo), semplifichiamo il calcolo del costo evitando di duplicarlo o distribuirlo in modo arbitrario tra le singole patologie.

L'arco multiplo è obbligatorio quando non si conosce a priori il numero massimo di valori (es. numero di autori di un libro) oppure quando tale numero è grande.

3.5.3 L'Attributo Configurazione

Se abbiamo un attributo multivalore (come l'arco multiplo) ma **conosciamo a priori i valori ammissibili e questi sono pochi (generalmente ≤ 10)**, possiamo semplificare notevolmente il modello. Invece di creare un nodo separato e richiedere complessi `JOIN`, inseriamo nel modello un **Attributo Configurazione**.

Funzionamento: Si "esplode" l'informazione creando un attributo booleano (Sì/No) per ciascun valore possibile. *Esempi:*

- Un negozio di videogame che può vendere 4 specifiche tipologie di giochi. Avremo 4 campi booleani nel record del negozio.

- Le università e l'elenco dei servizi gratuiti (Wi-Fi, Sale studio, Mensa). Se i servizi mappati sono meno di 10, usiamo una configurazione.

Questo costrutto rende le query analitiche molto più semplici, potendo filtrare direttamente con predicati booleani nella clausola `WHERE`.

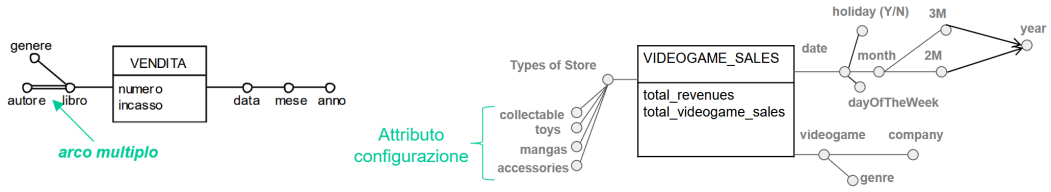


Figura 3.4: Rappresentazione di un arco multiplo (a sinistra) e di un attributo configurazione (a destra) nel Dimensional Fact Model.

3.5.4 Schemi di Fatto Vuoti

Non tutti i fatti registrano valori monetari o metriche di flusso. Se l'obiettivo è memorizzare **la pura presenza/verificarsi di un evento**, creiamo uno *schema di fatto vuoto*, ovvero privo di misure. *Esempio:* La frequenza degli studenti ai corsi. L'evento è caratterizzato dalle dimensioni Studente, Semestre, Corso. Se la tripla esiste nel Data Warehouse, l'evento si è verificato. In questo caso, l'unico operatore di aggregazione applicabile nelle query è il `COUNT(*)` per contare gli eventi accaduti.

3.6 La Rappresentazione del Tempo (Slowly Changing Dimensions)

Il data warehouse conserva lo storico aziendale. I fatti sono intrinsecamente legati al tempo (dimensione Data), ma anche gli attributi delle altre dimensioni possono subire **variazioni lente nel tempo** (Slowly Changing Dimensions - SCD), ad esempio un cliente che cambia stato civile, o un prodotto che cambia nome o packaging. Gestire questi cambiamenti è vitale per non falsare le analisi. Esistono tre modalità (o Tipi) principali:

3.6.1 Rappresentazione di Tipo 1 (Sovrascrittura)

È la fotografia dell'istante attuale. Il valore vecchio viene **sovrascritto** con quello nuovo.

- *Esempio:* Il cliente Mario Rossi si sposa. Lo stato civile viene sovrascritto. Tutti gli acquisti fatti da single figurano ora come fatti da "sposato". Anche un prodotto (es. cereali) che cambia nome, assocerebbe tutte le vendite passate al nuovo nome.
- *Problema:* Si perde totalmente la memoria storica e la correlazione tra l'evento temporale (il cambio) e i volumi. In contesti reali è fortemente sconsigliato.

3.6.2 Rappresentazione di Tipo 2 (Nuova Istanza)

Per ogni variazione si crea un **nuovo record (istanza)** della dimensione.

- *Esempio:* Avremo due record distinti: "Mario Rossi Single" e "Mario Rossi Sposato". Gli eventi passati sono legati al primo, i nuovi al secondo.
- *Problema:* I due record sono trattati dal sistema come entità indipendenti (come fossero due persone o due prodotti diversi). Non si può fare un'analisi aggregata che confronti facilmente "il prima e il dopo" per lo stesso soggetto senza complessi artifici.

3.6.3 Rappresentazione di Tipo 3 (Campionamento Temporale)

È l'approccio più completo e raccomandato, che richiede una modifica dello schema della dimensione. Oltre a creare una nuova istanza, si introducono attributi di validità e sequenzialità:

- Una coppia di *timestamp* (`inizio_validità`, `fine_validità`).
- Un identificativo di sequenza (o master ID) che lega tra loro tutte le incarnazioni della stessa entità logica.

In questo modo, quando Mario Rossi si sposa, la validità della prima istanza viene chiusa, se ne apre una nuova lo stesso giorno, ed entrambe puntano al medesimo soggetto reale, permettendo al sistema di ricostruire l'esatta storia dimensionale.

Tabella 3.1: Esempio di gestione di una variazione dimensionale con il Tipo 3

ID record	Master ID	Cliente	Stato civile	Inizio validità	Fine validità
101	25	Mario Rossi	Single	01/01/2022	14/06/2024
248	25	Mario Rossi	Sposato	15/06/2024	–

Il campo *Master ID* identifica la stessa entità logica nelle sue diverse istanze, mentre le date delimitano il periodo in cui ciascun record è valido.

3.7 Dimensionamento del Data Warehouse: Carico di Lavoro e Volume dei Dati

Durante la progettazione è necessario stimare lo spazio occupato su disco e le performance. Il calcolo del volume massimo teorico dei dati è dato dal **prodotto cartesiano delle cardinalità** di tutte le dimensioni. Tuttavia, le dimensioni nei modelli data warehouse sono considerate ortogonali, ma nella realtà fisica esiste la **sparsità**.

3.7.1 La Sparsità dei Dati

La sparsità indica la densità effettiva dei dati rispetto alle combinazioni teoriche. Se abbiamo le dimensioni Data, Fascia Oraria, Negozio e Prodotto: non è realistico ipotizzare che *tutti* i prodotti a catalogo vengano venduti in *tutti* i negozi, *tutti* i giorni, in *tutte* le fasce orarie. Se la combinazione massima è 10.000, e si stima che solo il 15% dei prodotti venga effettivamente venduto con questa frequenza, il fattore di sparsità interviene per ridurre la stima.

Legge dell'aggregazione: La sparsità si riduce al crescere del livello di aggregazione dei dati. Se passiamo dall'analisi per "Singola Data e Fascia oraria" all'analisi per "Mese", un prodotto che non era venduto tutti i giorni molto probabilmente verrà venduto almeno una volta nel corso dell'intero mese. Di conseguenza, salendo nella gerarchia, la matrice dei dati diventa molto più "densa" e la probabilità che l'evento si verifichi si avvicina al 100%.

Questo è il motivo principale per cui i data warehouse **non vengono implementati utilizzando fisicamente matrici n-dimensionali** pure. In una matrice, lo spazio per l'evento viene allocato in memoria che esso si verifichi o meno, sprecando enormi quantità di spazio. Si preferisce usare il **modello Relazionale (ROLAP)**, dove il record viene scritto (occupando spazio) solo se l'evento è realmente accaduto.

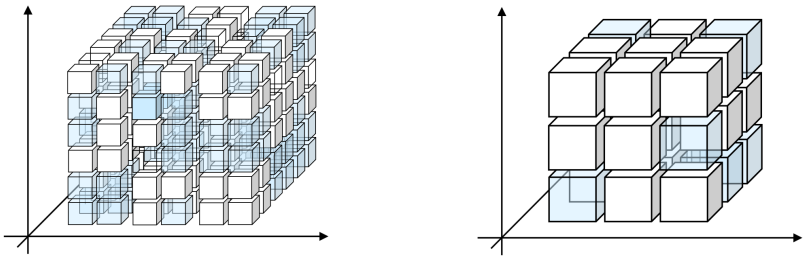


Figura 3.5: Esempio di sparsità dei dati: la matrice teorica (a sinistra) mostra tutte le combinazioni possibili, mentre la matrice reale (a destra) evidenzia solo le combinazioni effettivamente verificatesi.

3.8 Progettazione Logica: Il Modello Relazionale per il Data Warehouse

La progettazione logica produce uno **schema logico relazionale (ROLAP)** partendo dal modello concettuale DFM. Si basa su due princìpi opposti rispetto ai database tradizionali: 1. Denormalizzazione delle tabelle. 2. Tolleranza (e ricerca) della ridondanza dei dati per ottimizzare i tempi di lettura.

3.8.1 Lo Schema a Stella (Star Schema)

È l'architettura standard per i data warehouse relazionali.

- **Tabelle delle Dimensioni:** Si crea *una sola tabella per dimensione*. Le gerarchie vengono schiacciate e perse a livello strutturale: gli attributi di diverso livello (es. Data, Mese, Anno) diventano colonne della medesima tabella, generando forte ridondanza (es. il mese e l'anno saranno ripetuti identici per ogni giorno di quel mese). La **Chiave Primaria** della tabella dimensionale è sempre una chiave generata artificialmente (**chiave surrogata**, un contatore intero), mai una stringa o una logica di business, per velocizzare le operazioni di `JOIN`.
- **Tabella dei Fatti:** Si crea al centro dello schema. Contiene come chiavi esterne le chiavi surrogate delle dimensioni e, come attributi diretti, tutte le misure.

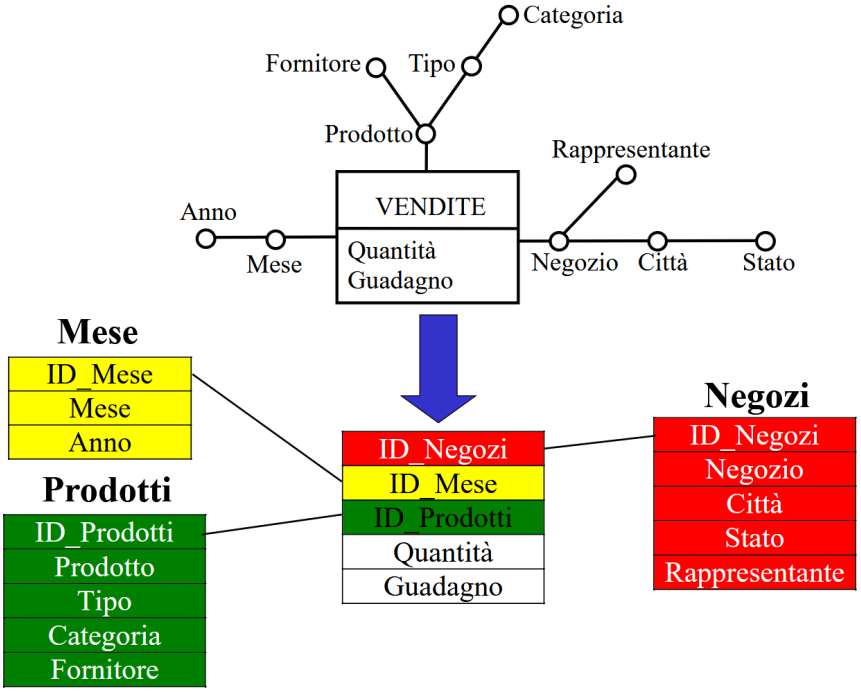


Figura 3.6: Struttura di uno Schema a Stella (Star Schema): il fatto centrale è circondato da tabelle dimensionali completamente denormalizzate.

3.8.2 Lo Schema a Fiocco di Neve (Snowflake Schema)

Lo *Snowflake Schema* tenta di reinserire la normalizzazione nello Star Schema. Sostanzialmente, si "taglia" una gerarchia dimensionale mettendola in una tabella separata (es. separando la gerarchia geografica dalla tabella Negozio).

- **Difetti:** Il risparmio di spazio su disco è marginale (perché le tabelle dimensionali sono trascurabili rispetto ai milioni di record della tabella dei fatti). In compenso, per ricostruire l'informazione il sistema dovrà eseguire `JOIN` aggiuntivi, degradando le prestazioni.
- **Uso reale:** Lo schema Snowflake è fortemente sconsigliato e raramente utilizzato nella pratica aziendale.

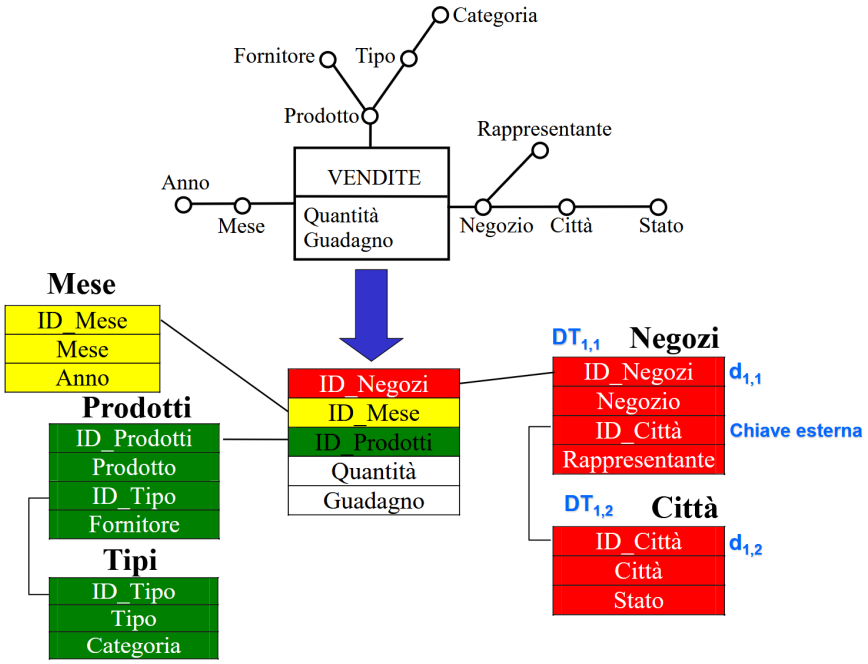


Figura 3.7: Struttura di uno Schema a Fiocco di Neve (Snowflake Schema): la gerarchia geografica è separata dalla tabella Negozio, richiedendo un join aggiuntivo per ricostruire l'informazione.

3.8.3 Traduzione Logica degli Archi Multipli

Come si mappa l'arco multiplo (relazione N:M tra dimensioni) in un database relazionale? Esistono due soluzioni principali (oltre a quella dell'attributo configurazione già discussa):

1. **Bridge Table (Tabella Ponte):** È la soluzione più ortodossa. Si crea una tabella intermedia tra il Fatto e la Dimensione multiautore (es. tra Vendita Libro e Autore). Questa tabella modella le coppie valide e introduce un **Peso** (es. 0.5 se gli autori sono due).

   - *Pro:* Permette interrogazioni pesate corrette (es. dividere l'incasso del libro equamente tra gli autori: `SUM(Incasso * Peso)`).
   - *Contro:* Rende complesse le query di semplice impatto (es. "Quante copie ha venduto l'autore in totale, a prescindere dal co-autoraggio?") e introduce un overhead di join.

2. **Push Down:** L'arco multiplo viene "spinto" direttamente nella Tabella dei Fatti. Il Fatto viene sdoppiato/moltiplicato: se un libro ha 3 autori, si creano 3 record nella tabella dei fatti, ognuno puntante a un autore diverso.

   - *Pro:* Costo di interrogazione bassissimo (elimina il join aggiuntivo).
   - *Contro:* Moltiplica enormemente lo spazio occupato dalla Tabella dei Fatti (che è già enorme). L'incasso andrebbe pre-frazionato (calcolato) durante la fase di ETL prima dell'inserimento, rendendo difficili eventuali variazioni. Le interrogazioni "di impatto" (senza frazionamento) diventano molto pericolose per il rischio di doppie somme.

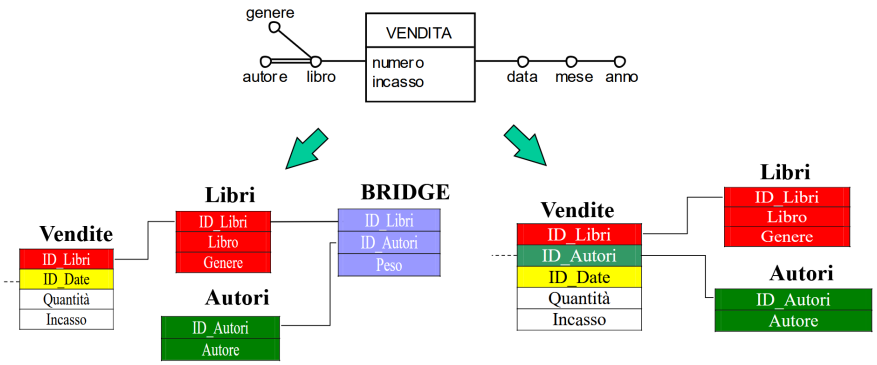


Figura 3.8: Traduzione logica di un arco multiplo: a sinistra la soluzione con Bridge Table, a destra la soluzione Push Down.

3.8.4 Dimensioni Degeneri e Junk Dimension

Una dimensione è *degenere* quando, dopo l'analisi, risulta composta da un **solo attributo** (privo di gerarchie). Per mappare queste dimensioni logicamente, esistono due vie:

1. **Integrazione nella Tabella dei Fatti:** (Push down). L'attributo viene inserito come normale colonna all'interno del Fatto, senza creare una tabella dimensionale esterna.

2. **Junk Dimension (Dimensione "pattumiera"):** Se ci sono molte dimensioni degeneri scorrelate (es. flag di stato, tipi di spedizione, codici semplici), le si compatta tutte in un'unica dimensione artificiale che contiene il *prodotto cartesiano* di tutte le combinazioni possibili di questi attributi. Nella tabella dei fatti risiederà solo un ID puntatore a questa riga. Questa soluzione è adottabile solo se la cardinalità risultante (le combinazioni totali) rimane piccola.

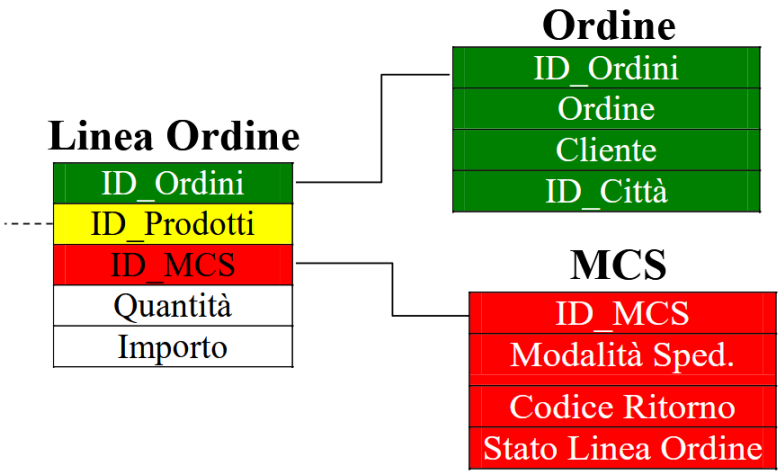


Figura 3.9: Esempio di Junk Dimension: le dimensioni degeneri "Tipo Spedizione" e "Flag Stato" vengono combinate in un'unica tabella dimensionale, riducendo il numero di join nella Tabella dei Fatti.

Capitolo 4

Analisi dei Dati in Ambienti OLAP ed Estensioni SQL

4.1 Operazioni di analisi dei dati nel Data Warehouse

Sul data warehouse creato attraverso la progettazione logica, si presenta la necessità di effettuare delle analisi. Tali analisi possono essere concepite come delle vere e proprie azioni di navigazione dei dati, che ci permettono di esplorare le gerarchie definite sulle diverse dimensioni. In un contesto decisionale, diventa inoltre fondamentale presentare il dato con un certo criterio di ordinamento per comprendere il suo posizionamento rispetto ad altri dati di riferimento (come ad esempio in una classifica).

Tra le operazioni principali troviamo:

- **Calcolo di funzioni aggregate**: effettuate lungo una o più dimensioni. Il supporto non si limita alle classiche somme o conteggi, ma si estende a funzioni avanzate necessarie per le analisi economiche (es. calcolo della media mobile, calcolo di mediane, individuazione di una top ten).
- **Operazioni di confronto**: essenziali per confrontare l'andamento degli affari, ad esempio calcolando il rapporto tra un totale parziale (l'incasso di un singolo negozio) e un totale complessivo (l'incasso di tutti i negozi in una città), oppure confrontando le vendite in mesi diversi. Queste operazioni sono difficili da eseguire con il solo SQL standard.
- **Analisi mediante Data Mining**: utilizzo di tecniche algoritmiche per estrarre pattern dai dati.

4.2 Strumenti di interfaccia per l'interrogazione

Per interrogare il data warehouse, l'utente ha a disposizione diverse tipologie di interfacce:

4.2.1 Ambiente controllato di query

In questo ambiente vengono predisposte ricerche complesse e procedure specifiche di analisi con una struttura prefissata e solitamente parametrica (es. l'utente può inserire un intervallo di date o una categoria di prodotti). Si utilizzano strumenti in cui è necessario lo sviluppo di codice ad hoc (es. *stored procedures*, applicazioni, viste materializzate con join e aggregazioni

predefinite). Sono disponibili strumenti flessibili per la gestione della reportistica che permettono di definire il layout grafico, la periodicità di pubblicazione e le liste di distribuzione del documento.

4.2.2 Ambiente di query ad hoc

Si tratta di un ambiente che permette la definizione di interrogazioni OLAP arbitrarie, progettate sul momento dall'utente, spesso tramite interfacce *point and click* (dove i click dell'utente generano automaticamente il codice SQL sottostante). Spesso l'interfaccia si basa sul paradigma del foglio elettronico (*spreadsheet*). Una sessione di lavoro OLAP in questo ambiente è caratterizzata da raffinamenti successivi: l'utente parte da una query generale e la modifica iterativamente per esplorare il dato in profondità. Questo ambiente è fondamentale quando i report predefiniti non sono sufficienti per rispondere a domande decisionali impreviste.

4.3 Analisi OLAP e Navigazione Multidimensionale

Le operazioni di ricerca disponibili per navigare i dati multidimensionali prendono il nome di operazioni OLAP. Possono essere combinate tra loro nella stessa interrogazione e utilizzate per raffinamenti successivi.

4.3.1 Roll up

Effettuare un'operazione di *roll up* significa ridurre il livello di dettaglio dei dati (passando da un dato più specifico a uno più generale). Viene eseguita principalmente interagendo con l'operatore `GROUP BY`. Il roll up si ottiene in due modi:

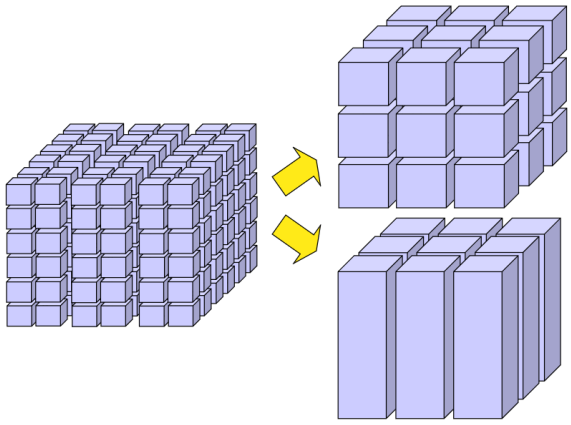


Figura 4.1: Rappresentazione visiva dell'operazione di Roll up: aggregazione dei dati da un livello più fine a uno più generale.

1. **Risalendo una gerarchia**: si cambia il livello di raggruppamento navigando la dimensione verso l'esterno. Ad esempio, passando da un aggregato calcolato per singolo *Negozio* a uno calcolato per *Città*. (Passaggio da `GROUP BY negozio, mese` a `GROUP BY città, mese`).

Month	Metrics / Customer Region	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germany	Canada
	Dollar Sales											
Jan 97		$ 620	$ 753	$ 30	$ 660	$ 2.405	$ 1.312	$ 440	$ 1.002	$ 1.002	$ 383	$ 210
Feb 97		$ 258	$ 252	$ 800	$ 975	$ 160	$ 582	$ 744	$ 310	$ 799	$ 118	$ 357
Mar 97		$ 648	$ 244	$ 148	$ 250	$ 1.085	$ 2.961	$ 650	$ 1.240	$ 119	$ 142	$ 96
Apr 97		$ 787	$ 588	$ 447	$ 486	$ 226	$ 506	$ 601	$ 119	$ 550	$ 85	
May 97		$ 1.350	$ 245	$ 936	$ 159	$ 664	$ 626	$ 107	$ 135	$ 200	$ 177	$ 230
Jun 97		$ 842	$ 582	$ 1.281	$ 937	$ 240	$ 774	$ 176	$ 1.139	$ 652	$ 254	$ 745
Jul 97		$ 652	$ 690	$ 486	$ 1.293	$ 605	$ 303	$ 818	$ 103	$ 124	$ 173	$ 66
Aug 97		$ 1.783	$ 304	$ 1.032	$ 170	$ 398	$ 356	$ 432	$ 190	$ 241	$ 407	$ 259
Sep 97		$ 581	$ 778	$ 3.558	$ 587	$ 440	$ 1.652	$ 1.071	$ 315	$ 210	$ 202	
Oct 97		$ 2.291	$ 1.840	$ 600	$ 656	$ 1.300	$ 718	$ 1.210	$ 427	$ 220	$ 520	$ 65
Nov 97		$ 39	$ 1.602	$ 1.082	$ 1.187	$ 842	$ 759	$ 745	$ 232	$ 101	$ 1.037	$ 37
Dec 97		$ 381	$ 1.588	$ 343	$ 118	$ 1.459	$ 635	$ 2.021	$ 259	$ 210	$ 119	$ 189
Jan 98		$ 311	$ 1.174	$ 2.634	$ 3.130	$ 954	$ 2.083	$ 1.351	$ 747	$ 426	$ 447	$ 1.141
Feb 98		$ 2.518	$ 702	$ 1.123	$ 1.336	$ 1.227	$ 3.887	$ 545	$ 268	$ 277	$ 282	
Mar 98		$ 2.459	$ 1.523	$ 1.178	$ 4.708	$ 1.420	$ 3.514	$ 1.948	$ 1.705	$ 276	$ 1.168	$ 63
Apr 98		$ 407	$ 841	$ 524	$ 712	$ 133	$ 2.486	$ 49	$ 390	$ 1.298	$ 221	$ 46
May 98		$ 667	$ 1.721	$ 440	$ 148	$ 80	$ 1.310	$ 303	$ 104	$ 657	$ 65	
Jun 98		$ 699	$ 1.096	$ 898	$ 353	$ 902	$ 839		$ 230	$ 155	$ 105	$ 75
Jul 98		$ 586	$ 1.897	$ 412	$ 226	$ 406	$ 361	$ 1.628	$ 267	$ 1.011	$ 41	$ 184
Aug 98		$ 894	$ 326	$ 792	$ 1.832	$ 1.199	$ 295	$ 1.816	$ 277	$ 102	$ 118	$ 115
Sep 98		$ 338	$ 3.179	$ 505	$ 427	$ 99	$ 2.976	$ 885	$ 135	$ 85	$ 1.110	$ 510
Oct 98		$ 544	$ 413	$ 1.467	$ 209	$ 679	$ 706	$ 556	$ 480	$ 485	$ 99	$ 160
Nov 98		$ 671	$ 459	$ 1.471	$ 2.066	$ 701	$ 716	$ 986	$ 1.127	$ 154	$ 440	$ 361
Dec 98		$ 836	$ 2.096	$ 1.726	$ 3.642	$ 395	$ 1.740	$ 1.943	$ 1.143	$ 366	$ 307	$ 118

Quarter	Metrics / Customer Region	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germany	Canada
	Dollar Sales											
Q1 1997		$ 1.526	$ 1.249	$ 978	$ 1.885	$ 3.650	$ 4.855	$ 1.834	$ 2.552	$ 1.920	$ 643	$ 663
Q2 1997		$ 2.979	$ 1.415	$ 2.664	$ 1.582	$ 1.130	$ 1.906	$ 884	$ 1.393	$ 1.402	$ 516	$ 975
Q3 1997		$ 3.016	$ 1.772	$ 5.076	$ 2.050	$ 1.443	$ 2.311	$ 2.321	$ 608	$ 575	$ 782	$ 325
Q4 1997		$ 2.711	$ 5.030	$ 2.025	$ 1.961	$ 3.601	$ 2.112	$ 3.976	$ 918	$ 531	$ 1.676	$ 291
Q1 1998		$ 5.288	$ 3.399	$ 4.935	$ 9.174	$ 3.601	$ 9.484	$ 3.844	$ 2.720	$ 979	$ 1.897	$ 1.204
Q2 1998		$ 1.773	$ 3.658	$ 1.862	$ 1.213	$ 1.115	$ 4.635	$ 352	$ 724	$ 2.110	$ 391	$ 121
Q3 1998		$ 1.818	$ 5.402	$ 1.709	$ 2.485	$ 1.704	$ 3.632	$ 4.329	$ 679	$ 1.198	$ 1.269	$ 809
Q4 1998		$ 2.051	$ 2.968	$ 4.664	$ 5.917	$ 1.775	$ 3.162	$ 3.485	$ 2.750	$ 1.005	$ 846	$ 639

Figura 4.2: Esempio di Roll up risalendo la gerarchia temporale: aggregazione delle vendite mensili nelle varie città per ottenere le vendite trimestrali.

2. **Eliminando una dimensione**: si rimuove un attributo dal raggruppamento. Ad esempio, passando da `GROUP BY prodotto, città` a `GROUP BY prodotto`. Eliminando la città, nel gruppo del prodotto convergeranno tutte le tuple associate a quel prodotto in qualsiasi città.

Category	Year	Metrics / Customer Region	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germa
		Dollar Sales										
Electronics	1997		$ 138	$ 1.774	$ 384	$ 138	$ 2.346	$ 2.554	$ 2.184	$ 566	$ 199	$
	1998		$ 1.184	$ 4.529	$ 1.892	$ 7.232	$ 651	$ 9.488	$ 476	$ 2.683	$ 462	$ 7
Food	1997		$ 759	$ 682	$ 729	$ 262	$ 588	$ 469	$ 807	$ 156	$ 615	$ 1
	1998		$ 538	$ 925	$ 959	$ 677	$ 213	$ 1.503	$ 261	$ 165	$ 175	$ 1
Gifts	1997		$ 2.532	$ 1.355	$ 1.854	$ 1.413	$ 2.535	$ 2.132	$ 1.904	$ 908	$ 375	$ 1.0
	1998		$ 1.955	$ 2.785	$ 2.800	$ 2.695	$ 1.813	$ 2.844	$ 1.778	$ 1.158	$ 717	$ 6
Health & Beauty	1997		$ 624	$ 640	$ 1.317	$ 647	$ 588	$ 754	$ 654	$ 143	$ 292	$ 3
	1998		$ 611	$ 887	$ 566	$ 382	$ 499	$ 1.162	$ 1.044	$ 273	$ 72	
Household	1997		$ 5.354	$ 4.112	$ 5.410	$ 4.446	$ 3.058	$ 3.974	$ 2.654	$ 3.545	$ 2.875	$ 1.9
	1998		$ 5.787	$ 5.320	$ 5.416	$ 6.812	$ 4.334	$ 5.008	$ 7.588	$ 2.139	$ 3.649	$ 2.7
Kid's Korner	1997		$ 201	$ 398	$ 485	$ 186	$ 409	$ 323	$ 396	$ 105	$ 34	$
	1998		$ 247	$ 422	$ 441	$ 380	$ 221	$ 592	$ 290	$ 198	$ 19	$
Travel	1997		$ 624	$ 505	$ 564	$ 386	$ 300	$ 978	$ 416	$ 48	$ 38	
	1998		$ 608	$ 559	$ 1.096	$ 611	$ 464	$ 316	$ 573	$ 257	$ 198	$

Category	Year	Metrics / Dollar Sales
Electronics	1997	$ 10.616
	1998	$ 29.299
Food	1997	$ 5.300
	1998	$ 5.638
Gifts	1997	$ 16.315
	1998	$ 20.047
Health & Beauty	1997	$ 6.042
	1998	$ 5.665
Household	1997	$ 38.383
	1998	$ 50.391
Kid's Korner	1997	$ 2.559
	1998	$ 2.943
Travel	1997	$ 4.497
	1998	$ 4.792

Figura 4.3: Esempio di Roll up eliminando una dimensione: aggregazione delle vendite per prodotto, sommando le vendite in tutte le città.

4.3.2 Drill down

È l'operazione inversa rispetto al roll up e consiste nell'aumentare il livello di dettaglio dei dati. Diventa utile, ad esempio, quando, analizzando un macrofenomeno (es. le vendite annuali), si nota uno scostamento significativo e si decide di scendere nel dettaglio per capirne la causa. Si ottiene mediante:

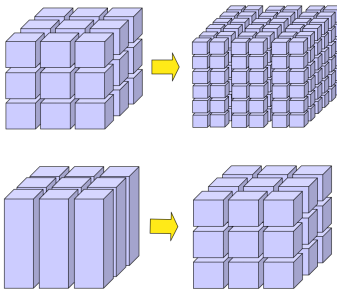


Figura 4.4: Rappresentazione visiva dell'operazione di Drill down: esplorazione dei dati da un livello più generale a uno più specifico.

1. **Discesa in una gerarchia**: ad esempio passando da un raggruppamento per *Città e Mese* a uno per *Negozio e Mese*. Il negozio è un dato più specifico.

Metrics	Dollar Sales										
Customer Region / Quarter	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germany	Canada
Q1 1997	$ 1.526	$ 1.249	$ 978	$ 1.885	$ 3.650	$ 4.855	$ 1.834	$ 2.552	$ 1.920	$ 643	$ 663
Q2 1997	$ 2.979	$ 1.415	$ 2.664	$ 1.582	$ 1.130	$ 1.906	$ 884	$ 1.393	$ 1.402	$ 516	$ 975
Q3 1997	$ 3.016	$ 1.772	$ 5.076	$ 2.050	$ 1.443	$ 2.311	$ 2.321	$ 608	$ 575	$ 782	$ 325
Q4 1997	$ 2.711	$ 5.030	$ 2.025	$ 1.961	$ 3.601	$ 2.112	$ 3.976	$ 918	$ 531	$ 1.676	$ 291
Q1 1998	$ 5.288	$ 3.399	$ 4.935	$ 9.174	$ 3.601	$ 9.484	$ 3.844	$ 2.720	$ 979	$ 1.897	$ 1.204
Q2 1998	$ 1.773	$ 3.658	$ 1.862	$ 1.213	$ 1.115	$ 4.635	$ 352	$ 724	$ 2.110	$ 391	$ 121
Q3 1998	$ 1.818	$ 5.402	$ 1.709	$ 2.485	$ 1.704	$ 3.632	$ 4.329	$ 679	$ 1.198	$ 1.269	$ 809
Q4 1998	$ 2.051	$ 2.968	$ 4.664	$ 5.917	$ 1.775	$ 3.162	$ 3.485	$ 2.750	$ 1.005	$ 846	$ 639



Metrics	Dollar Sales												
Customer City / Quarter	Arlin	San Pedro	Springfield	Chappel Hill	Scranburg	Pebble Beach	Martinsville	Maddon	Peoria	Pecos	Lake Barkley	Alcameda	Fingers Lake
Q1 1997	$ 675										$ 39		
Q2 1997				$ 203					$ 53				$ 135
Q3 1997				$ 276								$ 252	$ 63
Q4 1997	$ 215	$ 124			$ 113	$ 45	$ 192	$ 348				$ 79	$ 98
Q1 1998			$ 140	$ 174			$ 85				$ 237	$ 30	$ 119
Q2 1998								$ 12	$ 17				
Q3 1998	$ 734					$ 25	$ 1.535						
Q4 1998						$ 219	$ 119	$ 142		$ 85	$ 1.533		

Figura 4.5: Esempio di Drill down risalendo la gerarchia temporale: discesa dalle vendite trimestrali alle vendite mensili per le varie città.

2. **Aggiunta di una dimensione**: inserendo un nuovo attributo nel raggruppamento. Ad esempio da `GROUP BY città` a `GROUP BY città, prodotto`.

	Metrics	Dollar Sales	
	Year	1997	1998
Category			
Electronics		$ 10.616	$ 29.299
Food		$ 5.300	$ 5.638
Gifts		$ 16.315	$ 20.047
Health & Beauty		$ 6.042	$ 5.665
Household		$ 38.383	$ 50.391
Kid's Korner		$ 2.559	$ 2.943
Travel		$ 4.497	$ 4.792



	Metrics	Dollar Sales											
	Customer Region	North-East		Mid-Atlantic		South-East		Central		South		North-West	
	Year	1997	1998	1997	1998	1997	1998	1997	1998	1997	1998	1997	1998
Category													
Electronics		$ 138	$ 1.184	$ 1.774	$ 4.529	$ 384	$ 1.892	$ 138	$ 7.232	$ 2.346	$ 651	$ 2.554	$ 9.488
Food		$ 759	$ 538	$ 682	$ 925	$ 729	$ 959	$ 262	$ 677	$ 588	$ 213	$ 469	$ 1.503
Gifts		$ 2.532	$ 1.955	$ 1.355	$ 2.785	$ 1.854	$ 2.800	$ 1.413	$ 2.695	$ 2.535	$ 1.813	$ 2.132	$ 2.844
Health & Beauty		$ 624	$ 611	$ 640	$ 887	$ 1.317	$ 566	$ 647	$ 382	$ 588	$ 499	$ 754	$ 1.162
Household		$ 5.354	$ 5.787	$ 4.112	$ 5.320	$ 5.410	$ 5.416	$ 4.446	$ 6.812	$ 3.058	$ 4.334	$ 3.974	$ 5.008
Kid's Korner		$ 201	$ 247	$ 398	$ 422	$ 485	$ 441	$ 186	$ 380	$ 409	$ 221	$ 323	$ 592
Travel		$ 624	$ 608	$ 505	$ 559	$ 564	$ 1.096	$ 386	$ 611	$ 300	$ 464	$ 978	$ 316

Figura 4.6: Esempio di Drill down aggiungendo una dimensione: discesa dalle vendite aggregate per città alle vendite dettagliate per città e prodotto.

Osservazione (Sparsità dei dati). Come precisato dal professore, aumentando il livello di dettaglio (con il drill down) i dati tendono a diventare molto più **sparsi**. Nella visualizzazione tabellare inizieranno a comparire molte "celle vuote" (valori nulli), poiché scendendo nel dettaglio è meno probabile che uno specifico evento (es. la vendita di quel preciso prodotto, in quel preciso negozio, in quel giorno) si sia verificato.

4.3.3 Slice and dice

Quest'operazione consiste nel ridurre il volume dei dati per focalizzare l'attenzione su un sottoinsieme specifico. Tradotto in linguaggio SQL, corrisponde all'aggiunta di predicati di selezione nella clausola `WHERE`.

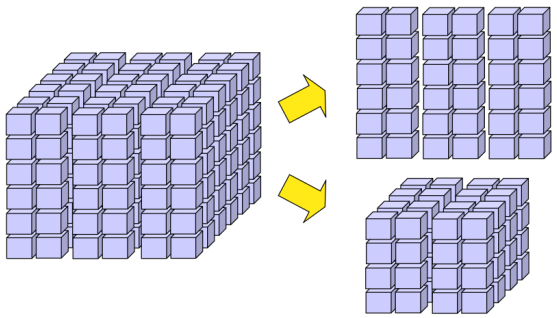
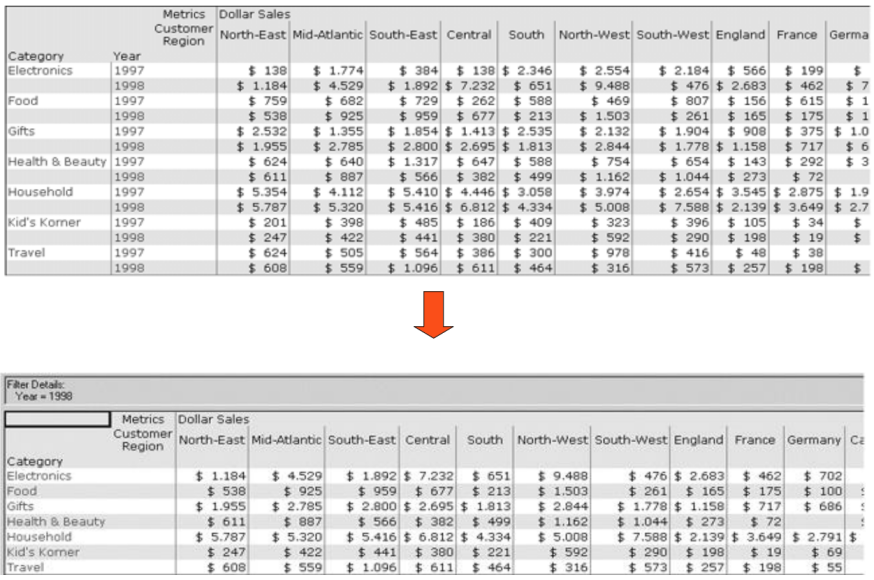


Figura 4.7: Rappresentazione visiva dell'operazione di Slice and Dice: selezione di un sottoinsieme specifico dei dati mediante predicati di filtraggio.

- **Slice (fetta)**: l'uso di un predicato di uguaglianza che seleziona una singola coordinata di una dimensione (es. `Anno = 2005`). Nella visualizzazione a ipercubo, si "affetta" il cubo prendendo solo uno strato.



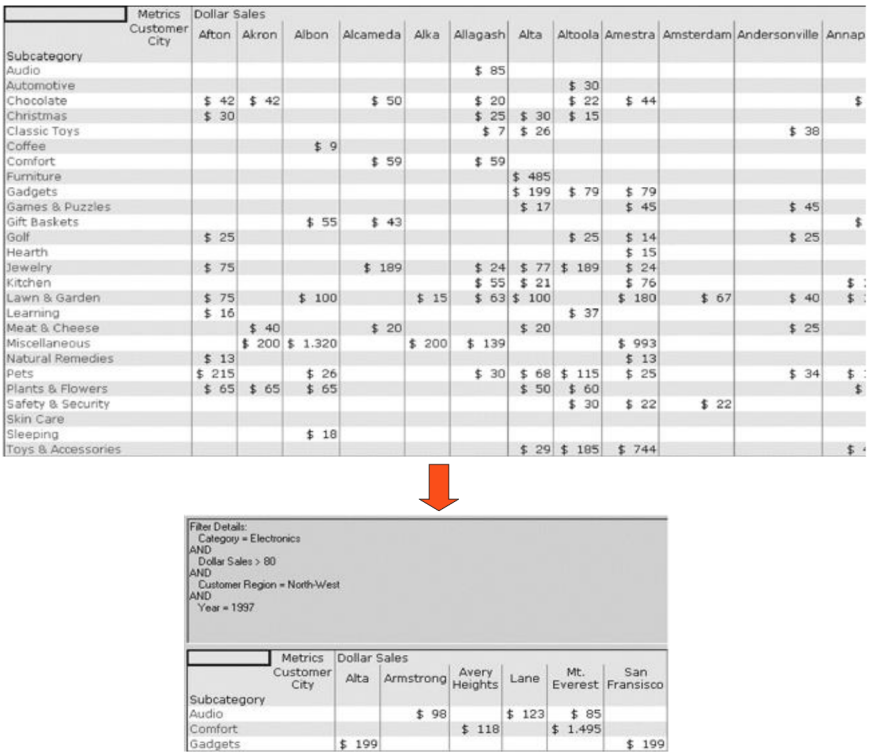
Category	Year	Metrics / Customer Region	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germa
		Dollar Sales										
Electronics	1997		$ 138	$ 1.774	$ 384	$ 138	$ 2.346	$ 2.554	$ 2.184	$ 566	$ 199	$
	1998		$ 1.184	$ 4.529	$ 1.892	$ 7.232	$ 651	$ 9.488	$ 476	$ 2.683	$ 462	$ 7
Food	1997		$ 759	$ 682	$ 729	$ 262	$ 588	$ 469	$ 807	$ 156	$ 615	$ 1
	1998		$ 538	$ 925	$ 959	$ 677	$ 213	$ 1.503	$ 261	$ 165	$ 175	$ 1
Gifts	1997		$ 2.532	$ 1.355	$ 1.854	$ 1.413	$ 2.535	$ 2.132	$ 1.904	$ 908	$ 375	$ 1.0
	1998		$ 1.955	$ 2.785	$ 2.800	$ 2.695	$ 1.813	$ 2.844	$ 1.778	$ 1.158	$ 717	$ 6
Health & Beauty	1997		$ 624	$ 640	$ 1.317	$ 647	$ 588	$ 754	$ 654	$ 143	$ 292	$ 3
	1998		$ 611	$ 887	$ 566	$ 382	$ 499	$ 1.162	$ 1.044	$ 273	$ 72	
Household	1997		$ 5.354	$ 4.112	$ 5.410	$ 4.446	$ 3.058	$ 3.974	$ 2.654	$ 3.545	$ 2.875	$ 1.9
	1998		$ 5.787	$ 5.320	$ 5.416	$ 6.812	$ 4.334	$ 5.008	$ 7.588	$ 2.139	$ 3.649	$ 2.7
Kid's Korner	1997		$ 201	$ 398	$ 485	$ 186	$ 409	$ 323	$ 396	$ 105	$ 34	$
	1998		$ 247	$ 422	$ 441	$ 380	$ 221	$ 592	$ 290	$ 198	$ 19	$
Travel	1997		$ 624	$ 505	$ 564	$ 386	$ 300	$ 978	$ 416	$ 48	$ 38	
	1998		$ 608	$ 559	$ 1.096	$ 611	$ 464	$ 316	$ 573	$ 257	$ 198	$

Filter Details:
Year = 1998

Category	Metrics / Customer Region	North-East	Mid-Atlantic	South-East	Central	South	North-West	South-West	England	France	Germany	Ca
	Dollar Sales											
Electronics		$ 1.184	$ 4.529	$ 1.892	$ 7.232	$ 651	$ 9.488	$ 476	$ 2.683	$ 462	$ 702	
Food		$ 538	$ 925	$ 959	$ 677	$ 213	$ 1.503	$ 261	$ 165	$ 175	$ 100	
Gifts		$ 1.955	$ 2.785	$ 2.800	$ 2.695	$ 1.813	$ 2.844	$ 1.778	$ 1.158	$ 717	$ 686	
Health & Beauty		$ 611	$ 887	$ 566	$ 382	$ 499	$ 1.162	$ 1.044	$ 273	$ 72		
Household		$ 5.787	$ 5.320	$ 5.416	$ 6.812	$ 4.334	$ 5.008	$ 7.588	$ 2.139	$ 3.649	$ 2.791	$
Kid's Korner		$ 247	$ 422	$ 441	$ 380	$ 221	$ 592	$ 290	$ 198	$ 19	$ 69	
Travel		$ 608	$ 559	$ 1.096	$ 611	$ 464	$ 316	$ 573	$ 257	$ 198	$ 55	

Figura 4.8: Esempio di Slice: selezione dei dati relativi all'anno 1998, evidenziando solo lo strato corrispondente nell'ipercubo.

- **Dice (dado/cubetto)**: una combinazione congiunta (in `AND`) di più predicati su dimensioni diverse (es. `Categoria = 'Alimentari' AND Città = 'Torino'`). Si restringe l'analisi a uno specifico "microcubo" di dati.



Subcategory	Metrics / Customer City	Afton	Akron	Albon	Alcameda	Alka	Allagash	Alta	Altoola	Amestra	Amsterdam	Andersonville	Annap
	Dollar Sales												
Audio							$ 85						
Automotive									$ 30				
Chocolate		$ 42	$ 42		$ 50		$ 20		$ 22	$ 44			$
Christmas		$ 30					$ 25	$ 30	$ 15				
Classic Toys							$ 7	$ 26				$ 38	
Coffee				$ 9									
Comfort					$ 59		$ 59						
Furniture								$ 485					
Gadgets								$ 199	$ 79	$ 79			
Games & Puzzles								$ 17		$ 45		$ 45	
Gift Baskets				$ 55	$ 43								$
Golf		$ 25							$ 25	$ 14		$ 25	
Hearth										$ 15			
Jewelry		$ 75			$ 189		$ 24	$ 77	$ 189	$ 24			
Kitchen							$ 55	$ 21		$ 76			$
Lawn & Garden		$ 75		$ 100		$ 15	$ 63	$ 100		$ 180	$ 67	$ 40	$
Learning		$ 16							$ 37				
Meat & Cheese			$ 40		$ 20			$ 20				$ 25	
Miscellaneous			$ 200	$ 1.320		$ 200	$ 139			$ 993			
Natural Remedies		$ 13								$ 13			
Pets		$ 215		$ 26			$ 30	$ 68	$ 115	$ 25		$ 34	$
Plants & Flowers		$ 65	$ 65	$ 65				$ 50	$ 60				$
Safety & Security									$ 30	$ 22	$ 22		
Skin Care													
Sleeping				$ 18									
Toys & Accessories								$ 29	$ 185	$ 744			$

Filter Details:
Category = Electronics
AND
Dollar Sales > 80
AND
Customer Region = North-West
AND
Year = 1997

Subcategory	Metrics / Customer City	Alta	Armstrong	Avery Heights	Lane	Mt. Everest	San Fransisco
	Dollar Sales						
Audio			$ 98		$ 123	$ 85	
Comfort				$ 118		$ 1.495	
Gadgets		$ 199					$ 199

Figura 4.9: Esempio di Dice: selezione dei dati relativi alla categoria 'Eletronics' vendite superiori a 60 dollari nella regione 'North-West', nell'anno 1997, evidenziando solo il microcubo corrispondente nell'ipercubo.

4.3.4 Pivot

Il *Pivot* permette di riorganizzare l'orientamento della struttura multidimensionale dei dati senza variarne il livello di dettaglio (non si fa aggregazione). È essenzialmente uno scambio o una riorganizzazione delle righe e delle colonne nella visualizzazione a griglia. L'utilità fondamentale del pivot, come specificato dal professore, risiede nel facilitare significativamente la lettura e il confronto visivo dei dati, mettendo colonne adiacenti per poter fare paragoni "a colpo d'occhio".

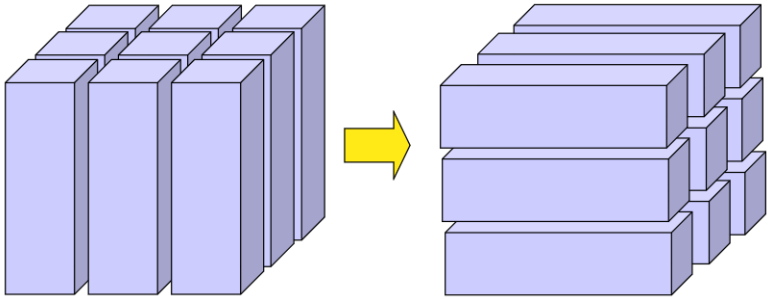


Figura 4.10: Ipercubo dei dati dopo l'operazione di Pivot.

		Metrics	Dollar Sales
Category	Year		
Electronics	1997		$ 10.616
	1998		$ 29.299
Food	1997		$ 5.300
	1998		$ 5.638
Gifts	1997		$ 16.315
	1998		$ 20.047
Health & Beauty	1997		$ 6.042
	1998		$ 5.665
Household	1997		$ 38.383
	1998		$ 50.391
Kid's Korner	1997		$ 2.559
	1998		$ 2.943
Travel	1997		$ 4.497
	1998		$ 4.792



	Metrics	Dollar Sales	
	Year	1997	1998
Category			
Electronics		$ 10.616	$ 29.299
Food		$ 5.300	$ 5.638
Gifts		$ 16.315	$ 20.047
Health & Beauty		$ 6.042	$ 5.665
Household		$ 38.383	$ 50.391
Kid's Korner		$ 2.559	$ 2.943
Travel		$ 4.497	$ 4.792

Figura 4.11: Esempio di riorganizzazione Pivot delle tabelle: spostamento degli anni dalle righe alle colonne per facilitare il confronto visivo dei dati.

4.4 Estensioni del linguaggio SQL

Le funzionalità descritte (come medie mobili, totali parziali verso totali cumulativi e classifiche) risultano difficili e molto inefficienti da programmare in SQL standard. SQL è un linguaggio **dichiarativo**: l'utente specifica *cosa* gli serve, ma non *come* il DBMS debba estrarlo. Questo lo differenzia dai linguaggi procedurali (come il C) in cui il programmatore detta i passaggi in sequenza.

Nel DBMS, un modulo chiamato ottimizzatore decide il piano di esecuzione basandosi sull'algebra relazionale. In una query standard:

```
SELECT X, Y FROM M, T WHERE <condizione> GROUP BY X
```

La sequenza logica di esecuzione è:

1. `FROM`: lettura delle tabelle.
2. `WHERE`: applicazione dei predicati e join.
3. `GROUP BY`: raggruppamento e collassamento dei dati.
4. `SELECT`: estrazione, proiezione degli attributi e visualizzazione.

Motivazione dell'estensione: Per poter estendere le funzionalità del linguaggio senza alterare e invalidare l'efficienza dei motori SQL esistenti, lo standard ANSI ha introdotto le nuove funzionalità analitiche (SQL-99/SQL3) all'interno della clausola `SELECT`. Essendo l'ultima istruzione ad essere eseguita, le estensioni ricevono in input i dati *già preparati* dalla query sottostante (già letti, filtrati ed eventualmente già aggregati).

4.5 Le Finestre di Calcolo (Window Functions)

La prima estensione fondamentale è la **finestra di calcolo**, introdotta dalla parola chiave `OVER`. A differenza del `GROUP BY` standard, che accorpa (*collassa*) N tuple in un'unica tupla generica di output, la finestra di calcolo **non collassa i dati**. Se riceve 10 tuple in input, produrrà 10 tuple in output, aggiungendo a ciascuna il valore dell'aggregato calcolato.

4.5.1 Elementi della clausola OVER

Una finestra di calcolo si inserisce nella `SELECT` e si compone di tre elementi opzionali, racchiusi tra parentesi tonde:

```
FUNZIONE_DI_AGGREGAZIONE(Attributo) OVER (
    [PARTITION BY ...]
    [ORDER BY ...]
    [FINESTRA_DI_AGGREGAZIONE]
)
```

1. **Partizionamento (`PARTITION BY`)**: divide le righe in sottogruppi logici. L'aggregato (ad es. la somma) si azzera ogni volta che cambia il valore dell'attributo di partizione (es. si calcola separatamente per ogni città). Se omesso, tutta l'estrazione costituisce un'unica grande partizione.
2. **Ordinamento (`ORDER BY`)**: applica un criterio di ordinamento (`ASC` o `DESC`) separatamente all'interno di ogni partizione. È fondamentale per dare senso a richieste sequenziali come la media mobile (se voglio i "mesi precedenti", i mesi devono essere in ordine).

3. **Finestra di aggregazione (Window clause)**: definisce il sottoinsieme di righe della partizione (rispetto alla riga attualmente processata) su cui calcolare l'aggregato.

4.5.2 Definizione dell'intervallo: Fisico vs Logico

La porzione di righe su cui calcolare l'aggregato può essere definita in due modi:

- **Intervallo Fisico (`ROWS`)**: si basa esclusivamente sul conteggio fisico delle righe (es. la riga corrente e le due righe precedenti posizionalmente).
  - *Sintassi es.*: `ROWS 2 PRECEDING` (la riga corrente e le 2 precedenti).
  - *Sintassi es.*: `ROWS UNBOUNDED PRECEDING` (dall'inizio della partizione alla riga corrente).

  È adatto quando i dati non presentano interruzioni nella sequenza, ed è possibile specificare più chiavi di ordinamento.

- **Intervallo Logico (`RANGE`)**: l'intervallo è definito in base al *valore* assunto dalla chiave di ordinamento (che deve essere di tipo numerico o temporale).
  - *Sintassi es.*: `RANGE 2 MONTH PRECEDING` (il mese corrente e i due mesi precedenti come concetto logico).

  Differenza sostanziale: Come precisato a lezione, se il dato è *sparso* e presenta un "buco" temporale (es. mancano i dati di aprile, passando direttamente da marzo a maggio), il raggruppamento logico per "2 mesi precedenti" a maggio includerà solo marzo (poiché aprile manca e febbraio è fuori range logico). Il raggruppamento fisico, invece, prenderà testardamente i due record precedenti disponibili nella tabella a prescindere dal loro effettivo valore temporale (prendendo marzo e febbraio). Il professore consiglia, ove i dati possano presentare lacune non note a priori, di prediligere l'intervallo logico per evitare errori semantici.

4.5.3 Applicazioni Pratiche ed Esempi

Data una base dati `Vendite(Città, Mese, Importo)` dove la coppia (Città, Mese) funge da chiave primaria (un solo record per combinazione):

Esempio 1: Calcolo della media mobile

Richiesta: Visualizzare per ogni città e mese l'importo e la media mobile dell'importo rispetto al mese corrente e ai due mesi precedenti, separatamente per città.

```
SELECT Città, Mese, Importo,
       AVG(Importo) OVER (
           PARTITION BY Città
           ORDER BY Mese
           ROWS 2 PRECEDING
       ) AS MediaMobile
FROM Vendite;
```

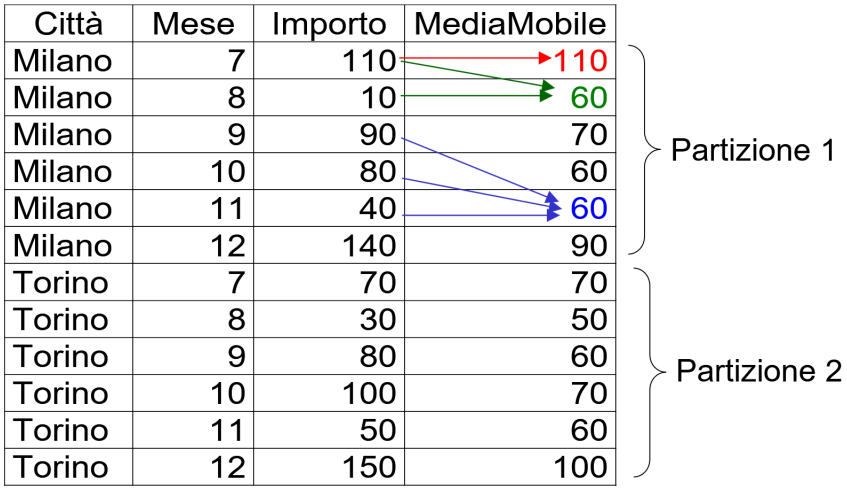


Figura 4.12: Esempio di calcolo della media mobile per ogni città e mese, considerando il mese corrente e i due mesi precedenti.

Spiegazione: Non serve `GROUP BY` esterna perché i dati di base hanno già la granularità voluta. La finestra partiziona per città (calcolo isolato per ogni città), ordina per mese (necessario per scandire il tempo) e prende le 2 righe precedenti + la corrente. Nei mesi in cui la finestra è "incompleta" (es. gennaio non ha mesi precedenti), la media viene calcolata solo sulle righe disponibili.

Esempio 2: Calcolo dei totali cumulativi

Richiesta: Somma cumulativa delle vendite al trascorrere dei mesi, azzerata per ogni città.

```
SELECT Città, Mese, Importo,
       SUM(Importo) OVER (
           PARTITION BY Città
           ORDER BY Mese
           ROWS UNBOUNDED PRECEDING
       ) AS SommaCumul
FROM Vendite;
```

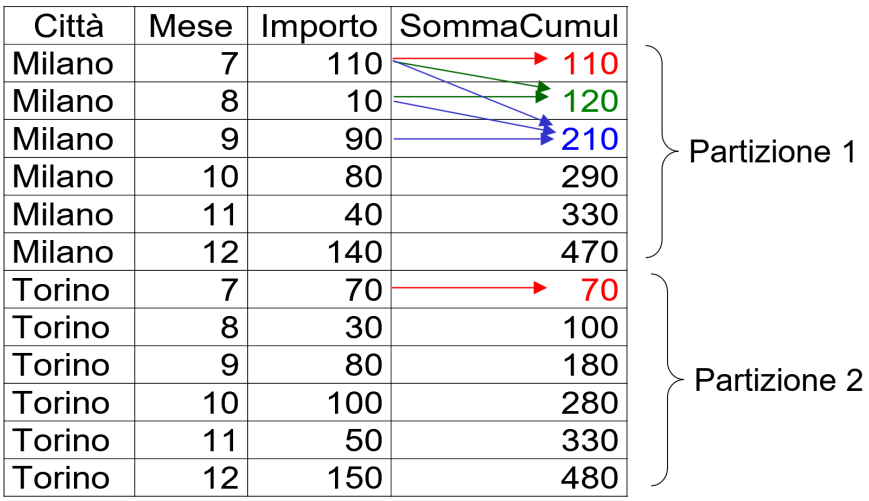


Figura 4.13: Esempio di calcolo dei totali cumulativi delle vendite per ogni città al trascorrere dei mesi.

Spiegazione: L'istruzione `ROWS UNBOUNDED PRECEDING` raccoglie per ogni riga processata tutte le righe dall'inizio della sua partizione (città) fino a essa. Il totale cresce progressivamente.

Esempio 3: Confronto tra dato dettagliato e macro-aggregato
Le finestre di calcolo permettono di evitare la riscrittura del codice e l'uso di query nidificate o *table functions*, permettendo di leggere la tabella una sola volta. Richiesta: Visualizzare importo corrente, importo totale per città, e importo totale assoluto di tutte le città.

```
SELECT Città, Mese, Importo,
       SUM(Importo) OVER (PARTITION BY Città) AS ImpTotaleCitta,
       SUM(Importo) OVER () AS ImpComplessivoAssoluto
FROM Vendite;
```

Città	Mese	Importo	ImpTotale	
Milano	7	110	470	Partizione 1
Milano	8	10	470	
Milano	9	90	470	
Milano	10	80	470	
Milano	11	40	470	
Milano	12	140	470	
Torino	7	70	480	Partizione 2
Torino	8	30	480	
Torino	9	80	480	
Torino	10	100	480	
Torino	11	50	480	
Torino	12	150	480	

Figura 4.14: Esempio di confronto tra dato dettagliato e macro-aggregato: visualizzazione dell'importo corrente, dell'importo totale per città e dell'importo totale assoluto di tutte le città.

Spiegazione: Se omettiamo `ORDER BY` e la finestra di aggregazione, il default applica la funzione di aggregazione sull'intera partizione. Se si utilizza la parentesi `OVER ()` completamente vuota, la funzione opera su *tutti i dati* estratti dalla `FROM` e `WHERE`. Questo permette di calcolare indici in un colpo solo, es. la percentuale di incasso rispetto al totale della città: `(Importo / SUM(Importo) OVER (PARTITION BY Città)) * 100`.

4.6 Integrazione delle Finestre di Calcolo con la clausola GROUP BY

La vera potenza analitica si ottiene combinando il raggruppamento standard `GROUP BY` con le funzioni `OVER`. Bisogna ricordare due regole fondamentali esposte dal professore:

1. **Visibilità degli attributi:** Se c'è una `GROUP BY`, gli unici attributi accessibili nella `SELECT` (e conseguentemente dentro la clausola `OVER`) sono gli attributi di raggruppamento. Se nella finestra serve partizionare per *Regione*, ma la query aggrega per *Città e Mese*, bisogna forzatamente aggiungere *Regione* alla clausola `GROUP BY`.

2. **Argomenti delle funzioni finestra:** L'input della finestra **non è più la riga cruda della tabella**, ma la riga pre-aggregata. Pertanto, come argomento della media mobile o del cumulativo non si scriverà più l'attributo semplice `Importo`, ma l'aggregato calcolato dalla `GROUP BY`, ovvero `SUM(Importo)`.

Osservazione (Uso degli Alias). L'alias assegnato nella SELECT (es. `SUM(Importo) AS TotMese`) non può mai essere riutilizzato all'interno delle clausole `OVER` o dei calcoli paralleli nella stessa SELECT. Questo perché l'etichetta (*label*) diventa disponibile per il database solo all'ultimo istante, ed è utilizzabile esclusivamente nella clausola finale di `ORDER BY` generale per riordinare la visualizzazione a schermo. All'interno della query, va sempre ridigitato `SUM(Importo)`.

Esempio Avanzato: Sia dato uno schema: `Vendite(SK, TK, ImportoGiornaliero)`, `Supermercato(SK, Città, Regione, Provincia)`, `Tempo(TK, Data, Mese, Anno, Semestre)`.

Richiesta: Per ogni città e semestre estrarre l'importo delle vendite. A fianco, calcolare l'incasso cumulativo al trascorrere dei semestri calcolato separatamente per ogni Regione.

```
SELECT S.Città, T.Semestre,
       SUM(V.ImportoGiornaliero) AS ImportoMese,
       SUM( SUM(V.ImportoGiornaliero) ) OVER (
           PARTITION BY S.Regione
           ORDER BY T.Semestre
           ROWS UNBOUNDED PRECEDING
       ) AS IncassoCumulRegione
FROM Vendite V, Supermercato S, Tempo T
WHERE V.SK = S.SK AND V.TK = T.TK
GROUP BY S.Città, T.Semestre, S.Regione;
```

Passaggi Logici:

- Effettuiamo i JOIN tra la tabella dei fatti e le dimensioni.
- Effettuiamo un raggruppamento per `Città` e `Semestre` per calcolare le vendite al livello di dettaglio richiesto. Aggiungiamo `Regione` alla `GROUP BY` per poterla utilizzare nella `PARTITION BY`. (Il professore sottolinea che aggregare per `Città, Semestre, Regione` non altera i gruppi rispetto ad aggregare per `Città, Semestre` dato che la città implica la regione).
- La prima `SUM(V.ImportoGiornaliero)` opera a livello di `GROUP BY` e collassa i record giornalieri restituendo il totale semestrale per la città.
- La `SUM` più esterna (con la clausola `OVER`) prende in input quei totali semestrali e li somma in modo cumulato, tenendo traccia dell'ordinamento temporale.

4.7 Funzioni di Ranking (Classifiche)

Le funzioni per ricavare la posizione in un ordinamento (*ranking*) si utilizzano sempre all'interno della clausola `OVER`. Sono disponibili due funzioni distinte, che differiscono per la gestione dei casi di "pari merito":

- `RANK()`: in caso di valori identici, assegna la stessa posizione a entrambi i record, ma lascia "buchi" nella sequenza successiva. Se due record sono a pari merito in posizione 1, il record immediatamente successivo sarà in posizione 3.
- `DENSE_RANK()`: assegna la stessa posizione in caso di pari merito, ma **non lascia buchi**. Dopo due posizioni 1, il successivo assume la posizione 2. (Da utilizzare in contesti in cui non è ammissibile saltare scaglioni, es. graduatorie di concorso senza limiti fisici di posto).

Esempio d'uso: Assegnare una graduatoria alle coppie (Città, Semestre) in base agli incassi massimi (in ordine decrescente), separatamente per ogni regione.

```
SELECT S.Città, T.Semestre, SUM(V.ImportoGiornaliero),
       RANK() OVER (
           PARTITION BY S.Regione
           ORDER BY SUM(V.ImportoGiornaliero) DESC
       ) AS GraduatoriaRegionale
FROM Vendite V, Supermercato S, Tempo T
WHERE V.SK = S.SK AND V.TK = T.TK
GROUP BY S.Città, T.Semestre, S.Regione
ORDER BY S.Regione, GraduatoriaRegionale;
```

L'`ORDER BY` finale, che sfrutta la label `GraduatoriaRegionale`, è necessario per ordinare *l'output visivo*. Senza di esso, a livello logico la graduatoria è stata calcolata correttamente ed associata ai record, ma il risultato verrebbe stampato a schermo in ordine sparso.

4.8 Estensioni della clausola GROUP BY

Per necessità di reportistica complessa, spesso è utile generare all'interno della stessa *query* totali generali e totali parziali a vari livelli della gerarchia. Lo standard SQL-99 ha introdotto tre operatori per la clausola `GROUP BY`, permettendo di ottenere risultati di più raggruppamenti con un solo passaggio (evitando letture multiple del database, risparmiando su JOIN e ordinamenti). Il raggruppamento genererà nella tabella finale righe in cui alcuni campi sono valorizzati a `NULL`. Quel `NULL` va interpretato come l'indicatore che, per quella specifica riga, la dimensione è stata aggregata (è un totale).

4.8.1 L'operatore ROLLUP

Calcola le aggregazioni eliminando un attributo alla volta procedendo **da destra verso sinistra**. Pertanto, l'ordine con cui si indicano gli attributi in `ROLLUP` è fondamentale e dipende dalla logica di aggregazione richiesta.

```
GROUP BY ROLLUP (Città, Mese, Prodotto)
```

Questa clausola calcolerà l'unione di quattro raggruppamenti distinti:

1. Raggruppamento per `(Città, Mese, Prodotto)`
2. Raggruppamento per `(Città, Mese)` (tolto Prodotto)
3. Raggruppamento per `(Città)` (tolto Mese)
4. Totale complessivo aggregato `()` (tolta Città)

Città	Mese	Pkey	TotVendite
Milano	7	145	110
Milano	7	150	10
Milano	...	...	...
Milano	7	**NULL**	8500
Milano	8	...	...
Milano	**NULL**	**NULL**	150000
Torino	...	...	150
Torino	...	**NULL**	2500
Torino	**NULL**	**NULL**	135000
...	...	...	...
NULL	**NULL**	**NULL**	25005000

Figura 4.15: Esempio di aggregazioni calcolate con l'operatore R OLLUP: da un raggruppamento dettagliato a uno più generale, fino al totale complessivo.

4.8.2 L'operatore CUBE

Se il `ROLLUP` analizza solo il percorso lineare destra-sinistra, il `CUBE` calcola **tutte le possibili combinazioni** (sottoinsiemi) degli attributi indicati. Nel caso di `CUBE(A, B, C)`, genererà: `(A,B,C)`, `(A,B)`, `(B,C)`, `(A,C)`, `(A)`, `(B)`, `(C)`, `()`. In questo caso l'ordine degli attributi tra le parentesi è ininfluente. Il calcolo sfrutta efficacemente l'ordinamento e la proprietà distributiva degli aggregati pre-calcolati (usando i subtotali di `(A,B)` per calcolare `(A)` ecc.).

4.8.3 L'operatore GROUPING SETS

Consente di definire in modo totalmente esplicito quali specifici livelli di aggregazione si desiderano ottenere, separandoli con una virgola (le parentesi tonde si usano se il singolo raggruppamento è composto da più attributi).

```
GROUP BY GROUPING SETS ((Città, Mese), (Mese), (Prodotto))
```

In questo caso, verranno calcolati solo ed esclusivamente questi 3 gruppi (anche disgiunti). Questa istruzione restituisce di fatto l'unione dei risultati di tre distinte query `GROUP BY` in un'operazione sola molto più efficiente.

Capitolo 5

Progettazione Fisica e Alimentazione del Data Warehouse

5.1 Progettazione Fisica

La progettazione logica di una base di dati definisce uno schema indipendente da come le informazioni vengono effettivamente memorizzate sul disco, garantendo così l'indipendenza logica. La progettazione fisica, al contrario, ha l'obiettivo di definire le strutture fisiche di memorizzazione e di accesso ai dati al fine di ottimizzare i tempi di esecuzione delle interrogazioni. Poiché SQL è un linguaggio dichiarativo (si specifica *cosa* cercare e non *come*), spetta a un modulo interno del DBMS, chiamato **ottimizzatore**, decidere il piano di esecuzione più efficiente. Il compito del progettista fisico è quindi fornire all'ottimizzatore le strutture accessorie adeguate.

5.1.1 Caratteristiche del carico di lavoro nel Data Warehouse

I sistemi di Data Warehouse sono progettati per soddisfare specifiche esigenze di analisi. Il carico di lavoro tipico presenta le seguenti caratteristiche:

- **Interrogazioni complesse con aggregazioni:** Le query richiedono l'accesso a una frazione significativa di ogni tabella e comportano operazioni onerose come join tra tabelle di grandi dimensioni (la tabella dei fatti e le dimensioni) e raggruppamenti (`GROUP BY`).
- **Accesso in sola lettura:** Durante le normali operazioni di analisi, i dati non vengono modificati dagli utenti.
- **Aggiornamento periodico:** I dati vengono aggiornati in finestre temporali ben definite (es. ogni notte), momento in cui è necessario ricostruire o aggiornare anche le strutture fisiche di accesso, come gli indici e le viste.

5.1.2 Viste Materializzate

Definizione e Differenza con le Viste Logiche

Per abbattere i tempi di esecuzione delle operazioni più onerose (in particolare Join e `GROUP BY`), si utilizzano le **viste materializzate**. A differenza delle *viste logiche* studiate nelle basi di dati relazionali (che contengono solo la definizione della query, utile spesso per motivi di riservatezza, e vengono ricalcolate a ogni esecuzione), le viste materializzate sono **sommari precalcolati della tabella dei fatti, memorizzati esplicitamente sul disco**.

L'utente finale o l'analista non ha bisogno di conoscere l'esistenza di queste viste. Le interrogazioni vengono scritte facendo riferimento allo schema multidimensionale standard (tabella dei fatti e dimensioni). Sarà l'ottimizzatore del DBMS che, qualora riconosca che una vista materializzata precalcolata può rispondere in modo più efficiente alla query o a parte di essa, effettuerà in modo trasparente la riscrittura dell'interrogazione (meccanismo di *aggregate navigation*).

Creazione di una Vista Materializzata

Una vista materializzata si definisce tramite un'istruzione SQL. Al suo interno contiene tipicamente:

- Un'operazione di Join tra la tabella dei fatti e le dimensioni di interesse.
- Una clausola `GROUP BY` per precalcolare l'aggregazione, riducendo il dettaglio delle dimensioni originali.
- Funzioni di aggregazione (come `SUM`, `COUNT`, ecc.) sulle misure.

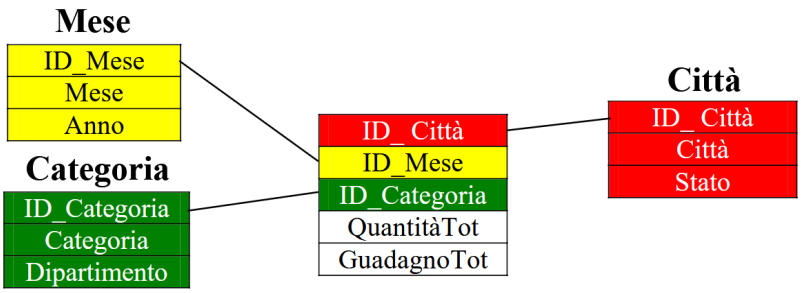


Figura 5.1: Definizione SQL di una vista materializzata.

Avvertenza sulle metriche: Una singola vista materializzata può essere utilizzata per rispondere a più interrogazioni diverse, aggregando ulteriormente i dati precalcolati. Tuttavia, bisogna prestare estrema attenzione all'operatore di aggregazione. Ad esempio, non è possibile calcolare la media delle medie. Le metriche memorizzate nella vista devono permettere il calcolo corretto ai livelli di dettaglio superiori (ad esempio, memorizzando somma e conteggio anziché direttamente la media).

Il Reticolo Multidimensionale

Per visualizzare le possibili aggregazioni e le dipendenze tra di esse, si utilizza il **reticolo multidimensionale** (o reticolo delle viste). Ogni nodo del reticolo rappresenta una possibile granularità (una combinazione di attributi dimensionali).

Il nodo di partenza (spesso in basso) rappresenta il dettaglio massimo presente nella tabella dei fatti (es. Prodotto, Data, Negozio). Risalendo le gerarchie delle dimensioni, si ottengono viste via via più aggregate (es. Categoria, Mese, Città). Se esiste un collegamento diretto (un arco) tra un nodo A e un nodo B posizionato più in alto, significa che il risultato della vista A può essere utilizzato per calcolare il risultato della vista B.

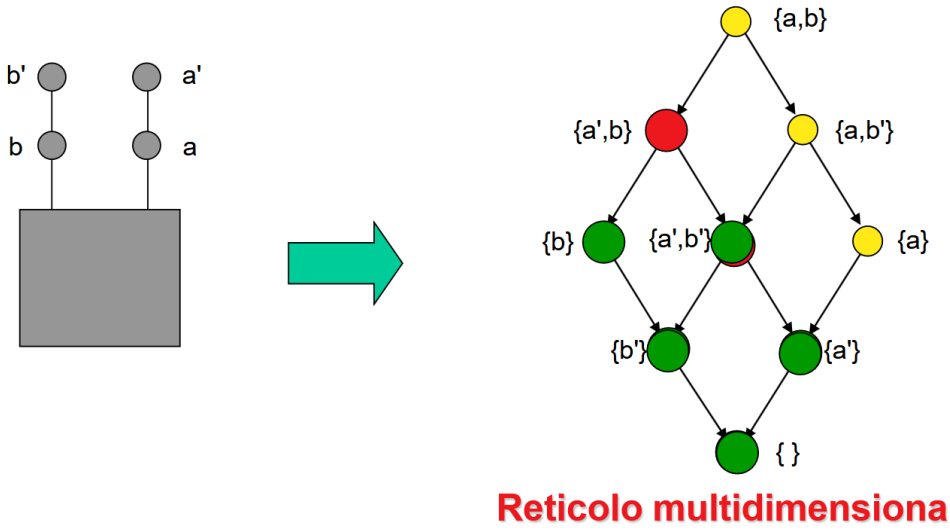


Figura 5.2: Esempio di reticolo multidimensionale che illustra le dipendenze tra le viste materializzabili.

L'importanza delle Gerarchie: Durante la lezione, il professore ha sottolineato un errore concettuale comune nella lettura del reticolo. Affinché si possa aggregare da un livello a un altro (es. da Tipo Prodotto a Categoria, o da Città a Regione), deve esistere un vincolo gerarchico rigoroso. Ad esempio, dati come Data, Mese e Trimestre formano una gerarchia rigida. Al contrario, se Tipo e Categoria fossero dimensioni ortogonali (dove a un tipo non è associata una sola categoria univoca, come giorno della settimana e mese), non ci potrebbe essere un arco diretto nel reticolo. Il reticolo assume quindi validità solo se supportato dalla corretta definizione delle gerarchie dimensionali.

Il Problema della Scelta delle Viste

Il numero di possibili viste materializzabili in un Data Warehouse reale è enorme. Non è pensabile materializzarle tutte. La scelta costituisce un problema di ottimizzazione che mira a minimizzare specifiche funzioni di costo:

- Il costo di esecuzione delle interrogazioni (che si abbassa aumentando le viste).
- Il costo di aggiornamento delle viste stesse (che si alza aumentando le viste, poiché a ogni refresh del data warehouse le viste vanno ricalcolate).

A ciò si aggiungono dei vincoli pratici:

- Spazio su disco disponibile.
- Finestra temporale disponibile per l'aggiornamento notturno (se l'aggiornamento richiede 8 ore ma la finestra utile è di sole 4 ore, il sistema non è sostenibile).

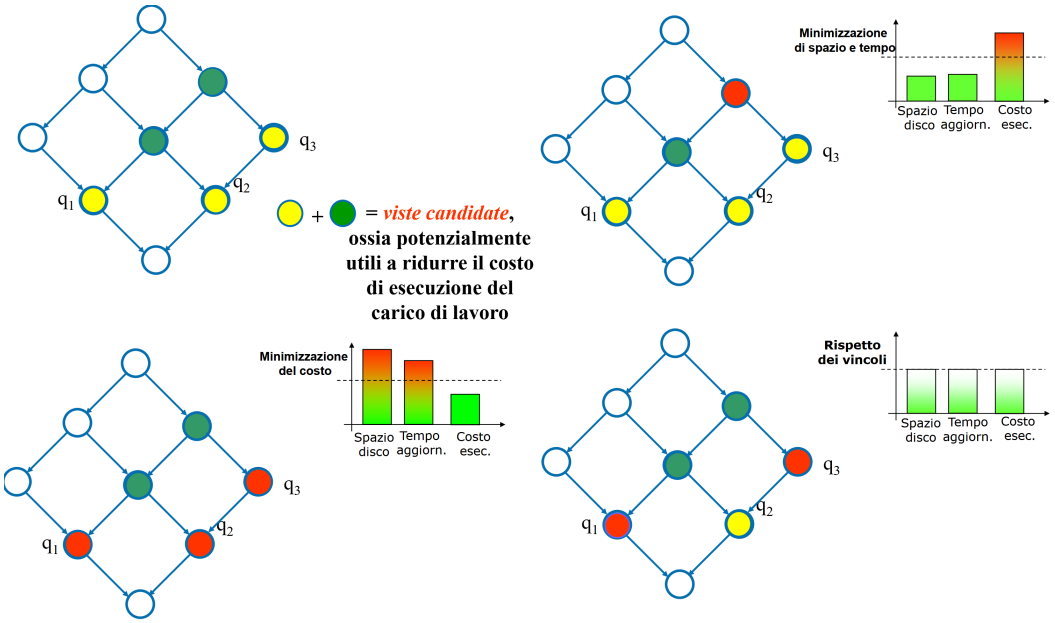


Figura 5.3: Analisi dei costi per la scelta delle viste ottimali: bilanciamento tra spazio, tempo di aggiornamento e tempo di query.

In sintesi, si può optare per tre strategie principali (illustrate nell'esempio discusso in aula per risolvere tre specifiche query):

1. **Una sola vista aggregata molto specifica (basso livello):** Risponde a tutte le interrogazioni. Occupa più spazio e ha un tempo di esecuzione medio-alto per query molto aggregate, ma è l'unica vista da mantenere aggiornata.

2. **Viste separate e mirate per ogni query:** Tempi di esecuzione minimi, ma massima occupazione di spazio e massimo costo di manutenzione/aggiornamento.

3. **Soluzione intermedia:** Creare alcune viste intermedie nel reticolo che riescano a servire più query contemporaneamente offrendo il miglior compromesso tra tempo di risposta e costi di manutenzione.

5.1.3 Altre Strutture Fisiche: Indici e Parallelismo

Oltre alle viste materializzate, la progettazione fisica si avvale di altre strutture:

- **Indici B+-tree:** Poco adatti per gli attributi dimensionali del data warehouse che spesso hanno una cardinalità bassa (es. sesso, stato civile, regione).

- **Indici Bitmap e Indici di Join:** Strutture più efficienti per gli attributi dimensionali, frequentemente coinvolti nei predicati di selezione e nelle operazioni di join.

Il **tuning** del database è la fase operativa in cui, tramite il monitoraggio, si verifica se l'ottimizzatore sta effettivamente utilizzando gli indici e le viste creati dal progettista. In caso contrario, queste strutture fisiche consumano risorse (per l'aggiornamento) senza portare alcun beneficio e vanno rimosse o modificate. Inoltre, le interrogazioni complesse (specialmente Join e Group By) beneficiano enormemente del **parallelismo** (frammentazione dei dati, e parallelizzazione inter-query e intra-query).

5.2 Alimentazione del Data Warehouse (Processo ETL)

L'architettura del Data Warehouse prevede che i dati di analisi siano fisicamente separati da quelli dei sistemi operazionali (sorgenti). Per mantenere aggiornato il Data Warehouse, è necessario un processo denominato **ETL** (Extraction, Transformation and Loading). Spesso, tra le sorgenti e il data warehouse, si inserisce un'area di appoggio intermedia detta **Staging Area**, che porta l'architettura a tre livelli e facilita l'esecuzione di trasformazioni complesse.

Il processo ETL deve gestire due scenari ben distinti:

1. **Il primo popolamento (tempo zero):** Il Data Warehouse e le dimensioni sono completamente vuoti e occorre acquisire una fotografia storica completa di tutto il periodo di interesse aziendale.
2. **L'aggiornamento periodico (es. giornaliero):** Il Data Warehouse è già popolato, bisogna estrarre e caricare unicamente le variazioni e le nuove transazioni avvenute dall'ultimo caricamento.

5.2.1 Estrazione (Extraction)

L'estrazione è la fase di acquisizione dei dati dalle sorgenti. Può essere:

- **Statica:** Una vera e propria fotografia dell'intero database operazionale, eseguita per il primo popolamento.
- **Incrementale:** Si estraggono solo i dati inseriti, modificati o cancellati dall'ultimo aggiornamento.

Le sfide dell'Estrazione Incrementale

La complessità dell'estrazione incrementale dipende fortemente dalla natura dei dati operazionali:

- **Dati storicizzati:** Il sistema OLTP salva storicamente tutte le variazioni (es. transazioni bancarie). L'estrazione è semplice perché ogni evento è tracciato.
- **Dati semi-storicizzati o Transitori:** Il DBMS mantiene solo l'immagine corrente del dato (es. scorte di magazzino in un dato istante) sovrascrivendo i valori precedenti. L'estrazione incrementale è molto complessa perché si perde traccia di cosa sia cambiato e di eventuali cancellazioni.

Per comprendere questa complessità, consideriamo il seguente esempio di estrazione incrementale tra il 4 aprile e il 6 aprile.

Tabella 5.1: Fotografia DBMS al 4/4/2010

CodProdotto	**Cliente**	**Qtà**	
1	Greco di tufo	Malavasi	50
2	Barolo	Maio	150
3	Barbera	Lumini	75
4	Sangiovese	Cappelli	45

Tabella 5.2: Fotografia DBMS al 6/4/2010

Cod	Prodotto	Cliente	Qtà
1	Greco di tufo	Malavasi	50
2	Barolo	Maio	150
4	Sangiovese	Cappelli	145
5	Vermentino	Maltoni	25
6	Trebbiano	Maltoni	150

Un sistema ETL efficiente deve dedurre dalla differenza tra questi stati un file di aggiornamento che tracci esattamente le operazioni logiche avvenute, non potendosi limitare alle sole letture:

Tabella 5.3: Estrazione Incrementale risultante (D=Delete, U=Update, I=Insert)

Cod	Prodotto	Cliente	Qtà	Azione
3	Barbera	Lumini	75	D (cancellato)
4	Sangiovese	Cappelli	145	U (modifica qtà da 45 a 145)
5	Vermentino	Maltoni	25	I (nuovo inserimento)
6	Trebbiano	Maltoni	150	I (nuovo inserimento)

Tecniche per l'estrazione incrementale

Per realizzare concretamente questa estrazione, si possono utilizzare diverse tecniche, ognuna con precisi trade-off:

	Statica	Marche temp.	Assistita appl.	Trigger	Log
Gestione dati transitori	NO	Incompleta	Completa	Completa	Completa
Supporto sistemi a file	SI	SI	SI	SI	NO
Complessità procedure	Bassa	Bassa	Alta	Media	Bassa
Modifiche ad applicazioni	Nessuna	Probabile	Probabile	Nessuna	Nessuna
Impatto prestazioni operaz.	Nessuna	Nessuna	Medio	Medio	Nessuna

Tabella 5.4: Confronto semplificato tra le tecniche di estrazione (rielaborato dalle slide).

- **Assistita dall'applicazione:** L'applicativo stesso viene modificato per scrivere da qualche parte i dati che sta cambiando. Richiede modifiche ai programmi (applicazioni on top al DBMS) e aumenta il carico applicativo, ma è spesso l'unica via per i vecchi sistemi *legacy*.
- **Uso del Log:** Si legge il file di log interno del DBMS (che per sua natura traccia ogni operazione per garantire la recovery). È molto efficiente e non impatta sul carico applicativo, ma il log è in formato proprietario del vendor del database.
- **Tramite Trigger:** Si programmano dei trigger interni al DBMS che scattano autonomamente su `INSERT`, `UPDATE` e `DELETE`. Non richiede la modifica del programma applicativo esterno, ma incrementa notevolmente il carico di lavoro del DBMS in fase transazionale.
- **Tramite Timestamp (Marche temporali):** Si aggiunge una colonna data/ora ai record per tracciare l'ultima modifica. È un metodo parziale perché fallisce nel tracciare i dati *cancellati* (se il record scompare, scompare anche il suo timestamp).

5.2.2 Pulitura (Data Cleansing)

I dati estratti da sorgenti eterogenee presentano quasi sempre problemi di *Data Quality*: duplicati, dati mancanti, formati non uniformi (es. "C.so" vs "Corso"), valori errati. Le procedure di pulitura includono:

- **Uso di dizionari:** Standardizzazione dei domini (es. unificare le sigle delle province, CAP validi).
- **Tecniche di fusione approssimata (Approximate Join e Purge/Merge):** Spesso uno stesso cliente è memorizzato in due filiali con grafie leggermente diverse (es. "Rossi M." e "Mario Rossi"). Non avendo una chiave univoca comune, bisogna applicare algoritmi di similarità testuale per identificare se si tratta della stessa entità e procedere all'unione dei record (purge/merge).

Nota sull'Intelligenza Artificiale nell'ETL: Come sottolineato dal professore durante la lezione, i sistemi ETL moderni si stanno evolvendo rapidamente. Per affrontare problemi di pulitura complessa (come join approssimati senza chiavi certe o riconoscimento di entità), si sta diffondendo l'uso di tecniche di Data Science, inclusi algoritmi di clustering e l'uso di modelli linguistici di grandi dimensioni (LLM). L'ETL non è più solo una semplice procedura SQL, ma assomiglia sempre di più alla fase di *data preparation* del Machine Learning.

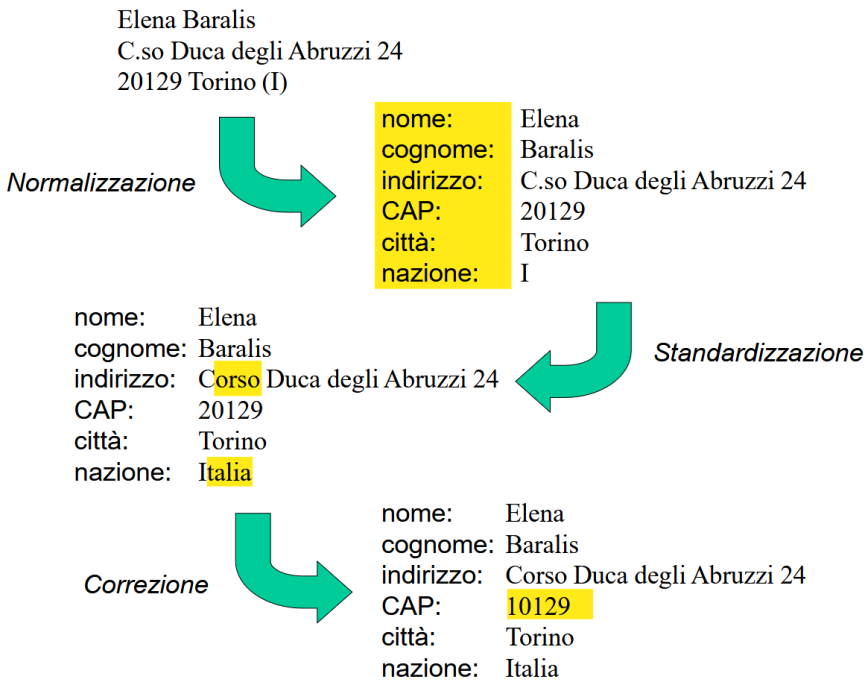


Figura 5.4: Esempi operativi di pulitura e risoluzione del problema del Purge/Merge.

5.2.3 Trasformazione

Questa fase converte i dati dal formato in cui sono memorizzati operazionalmente a quello integrato del data warehouse. Questo passaggio, che di norma si completa nella Staging Area, prevede:

- Passaggio allo **schema riconciliato** (notazione uniforme).
- Generazione dei **valori aggregati** se richiesti dallo schema.

- **Generazione delle chiavi surrogate:** Fondamentale per svincolare l'identificativo interno del data warehouse dalle chiavi applicative (che potrebbero cambiare nel tempo nei sistemi sorgente o presentare conflitti).

5.2.4 Caricamento (Loading)

La fase finale è la propagazione dei dati trasformati fisicamente nel Data Warehouse. Esiste una **gerarchia rigorosa e obbligatoria** per l'ordine di caricamento, al fine di rispettare l'integrità referenziale. I passi sono i seguenti:

1. **Alimentazione delle Dimensioni:** Prima di poter caricare un evento, le entità che lo descrivono devono esistere. Il sistema legge gli aggiornamenti dalla Staging Area, verifica se il record dimensionale esiste già (es. un cliente). Se esiste, ne estrae la *chiave surrogata*; se non esiste, inserisce il nuovo record generando una nuova chiave.

2. **Alimentazione della Fact Table:** I nuovi eventi vengono inseriti nella tabella dei fatti. Durante questo inserimento, il sistema non usa i codici operazionali originali, ma recupera (tramite query) le *chiavi surrogate* che sono state appena aggiornate/generate nel passo 1.

3. **Aggiornamento delle Viste Materializzate e degli Indici:** Poiché i dati di base sono cambiati, tutte le strutture fisiche di supporto (che contengono aggregazioni precalcolate) devono essere riallineate con i nuovi valori.

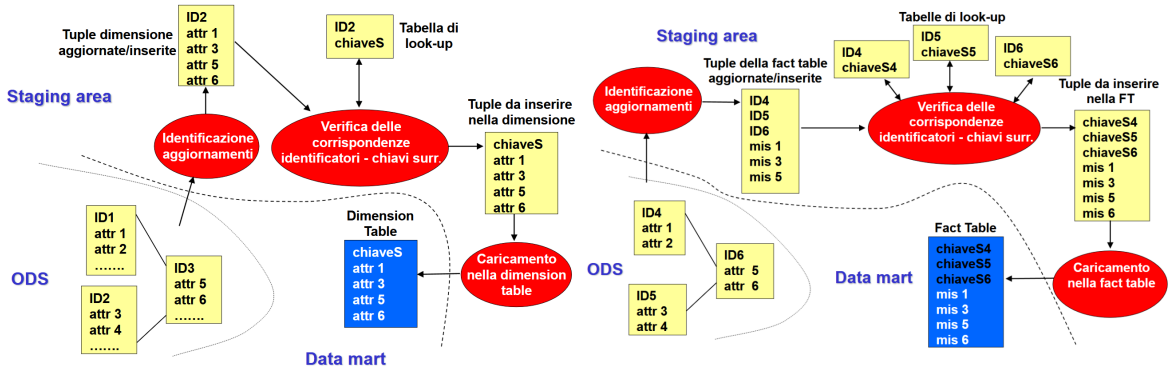


Figura 5.5: Flusso logico per l'alimentazione delle dimensioni e della fact table.

Capitolo 6

Estensioni al Linguaggio SQL in Oracle per l'Analisi dei Dati

Questo capitolo introduce le estensioni al linguaggio SQL introdotte nel DBMS Oracle per supportare le operazioni di Business Intelligence e l'analisi dei dati in ambito Data Warehouse. Gran parte di queste estensioni si rifà allo standard SQL, pur presentando alcune specificità sintattiche, in particolare per quanto riguarda la definizione delle finestre di aggregazione.

Le funzionalità principali introdotte (funzioni OLAP) includono:

- **Finestre di calcolo** (tramite la clausola `OVER` / `WINDOW`);
- **Funzioni di ranking** per la generazione di classifiche (`RANK`, `DENSE_RANK`, ecc.);
- **Estensione della clausola `GROUP BY`** (`ROLLUP`, `CUBE`, `GROUPING SETS`), che permette di generare raggruppamenti multipli con un singolo accesso ai dati.

6.1 Le Finestre di Calcolo

La finestra di calcolo permette di eseguire aggregazioni e analisi su un sottoinsieme di righe correlate alla riga corrente, senza collassare il risultato in un'unica riga per gruppo (come invece accade con la `GROUP BY` tradizionale). All'interno della finestra è possibile definire il partizionamento (`PARTITION BY`), l'ordinamento (`ORDER BY`) e l'ampiezza della finestra di aggregazione stessa.

Per illustrare il funzionamento della finestra di aggregazione, consideriamo una tabella relativa alle vendite con il seguente schema: `VENDITE(Città, Data, Importo)`.

Nella definizione della porzione di finestra su cui effettuare il calcolo, è possibile adottare due approcci distinti: il raggruppamento a livello fisico e quello a livello logico.

6.1.1 Raggruppamento a livello fisico (`ROWS`)

Il raggruppamento fisico conta esplicitamente il numero di tuple (righe) da includere nella finestra, a prescindere dal valore che assumono gli attributi. Questo approccio è utile quando si ha la garanzia che i dati siano continui, ma non tiene conto di eventuali "buchi" nella sequenza temporale o logica.

Esempio di raggruppamento fisico *Selezionare, separatamente per ogni città, per ogni data l'importo e la media dell'importo considerando la riga corrente e le due righe che la precedono.*

In questo caso, calcoliamo una **media mobile**. Prima di scrivere la query, analizziamo i passaggi logici:

1. **Partizionamento**: I dati vanno calcolati "separatamente per ogni città", quindi occorre partizionare rispetto all'attributo `Città`.
2. **Ordinamento**: È fondamentale ordinare i record per `Data`. Come sottolinea il professore, se non si effettuasse l'ordinamento, si rischierebbe di sommare una data di gennaio con una di dicembre, ottenendo un calcolo privo di senso logico.
3. **Finestra**: Si richiede la riga corrente e le due *precedenti*. Usando un approccio fisico parliamo esplicitamente di "righe", da cui la sintassi `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`.

```
SELECT Città, Data, Importo,
       AVG(Importo) OVER (
           PARTITION BY Città
           ORDER BY Data
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS MediaMobile
FROM Vendite
ORDER BY Città, Data;
```

Nota: Nella query manca l'operatore `GROUP BY` tradizionale. Sulla finestra applichiamo direttamente `AVG(Importo)`. L'`ORDER BY` finale, esterno alla funzione finestra, serve unicamente per una migliore visualizzazione dei risultati a schermo.

6.1.2 Raggruppamento a livello logico (`RANGE`)

Il raggruppamento logico, a differenza di quello fisico, valuta il *valore* assunto dal campo di ordinamento (tipicamente una data o un numero) per definire l'intervallo. Se, ad esempio, un negozio rimane chiuso in una specifica data, e quindi manca il record, l'approccio logico non andrà a ripescare un giorno ancora precedente per arrivare a "due righe", ma si limiterà a considerare i record effettivamente presenti nell'intervallo temporale richiesto.

Esempio di raggruppamento logico *Selezionare, separatamente per ogni città, per ogni data l'importo e la media dell'importo considerando la riga corrente e le vendite avvenute nei due giorni precedenti.*

```
SELECT Città, Data, Importo,
       AVG(Importo) OVER (
           PARTITION BY Città
           ORDER BY Data
           RANGE BETWEEN INTERVAL '2' DAY PRECEDING AND CURRENT ROW
       ) AS MediaUltimi3Giorni
FROM Vendite
ORDER BY Città, Data;
```

La sintassi `RANGE BETWEEN INTERVAL '2' DAY PRECEDING AND CURRENT ROW` definisce un intervallo logico di tempo. Il professore precisa che, in Oracle, oltre a `DAY`, è possibile utilizzare anche `MONTH` e `YEAR`. L'inclusione di `CURRENT ROW` specifica che il calcolo della media comprende anche la riga corrente, valutando così gli ultimi 3 giorni complessivi (i 2 precedenti + quello attuale).

6.2 Funzioni di Ranking

Le funzioni di ranking permettono di associare ad ogni record una posizione all'interno di una classifica. Per i prossimi esempi, consideriamo il seguente schema relazionale che modella forniture aziendali:

- `FRN(COD_F, Nome, Sede_F)`: Fornitori;
- `ART(COD_A, Tipo, Colore, Peso)`: Articoli;
- `PRG(COD_P, Nome, Sede_P)`: Progetti;
- `FAP(COD_F, COD_P, COD_A, Q)`: Quantità `Q` di articolo fornita per un progetto da un fornitore.

6.2.1 `RANK` e `DENSE_RANK`

Esempio: Classifica di vendita degli articoli *Selezionare per ogni articolo la quantità totale fornita e il ranking in funzione delle quantità totali fornite per ogni articolo.*

Poiché desideriamo una classifica generale di tutti gli articoli (i "top venduti"), non abbiamo bisogno di frammentare i dati: la finestra di calcolo **non richiederà un partizionamento**, ma soltanto un criterio di ordinamento.

```
SELECT COD_A, SUM(Q) AS QTot,
       RANK() OVER (
           ORDER BY SUM(Q)
       ) AS RankVendite
FROM FAP
GROUP BY COD_A;
```

Osservazione sull'ordinamento: Nel codice mostrato sulle slide, non è specificato `DESC` nell'`ORDER BY`. Il professore fa notare che un ordinamento naturale per una classifica di vendita sarebbe decrescente (il prodotto più venduto al primo posto). Tuttavia, non specificando nulla, il sistema ipotizza un ordinamento **ascendente** (`ASC`). Pertanto, il prodotto con la quantità minore otterrà la prima posizione.

La differenza fondamentale tra `RANK` e `DENSE_RANK` risiede nella gestione dei **pari merito**:

- **`RANK`**: a parità di valore assegna la stessa posizione, ma salta le posizioni successive (es. due record al 3° posto portano il record successivo al 5° posto).
- **`DENSE_RANK`**: assegna la stessa posizione per i pari merito, ma *non salta* le posizioni successive (es. due record al 3° posto, il successivo sarà al 4° posto).

Tabella 6.1: Confronto dei risultati tra RANK() e DENSE_RANK()

COD_A	QTot	RankVendite (RANK)	DenseRankVendite (DENSE_RANK)
A2	300	1	1
A5	1100	2	2
A4	1300	3	3
A6	1300	3	3
A1	1900	5	4
A3	4500	6	5

6.2.2 Doppio Ranking

È possibile definire all'interno della medesima `SELECT` molteplici colonne di rank, calcolate rispetto a criteri di ordinamento differenti. Questo avviene perché la funzione genera semplicemente una nuova colonna aggiuntiva rispetto ai dati esistenti.

Esempio di doppio ranking *Selezionare per ogni articolo il codice, il peso, la quantità totale fornita, il ranking in funzione del peso e il ranking in funzione della quantità totale fornita.*

In questo caso, siccome vogliamo visualizzare anche il peso (attributo di `ART`) e calcolare la quantità (attributo di `FAP`), dobbiamo eseguire una `JOIN` tra le due tabelle.

```
SELECT ART.COD_A, ART.Peso,
       SUM(Q) AS QTot,
       RANK() OVER (ORDER BY ART.Peso) AS R_Peso,
       RANK() OVER (ORDER BY SUM(Q)) AS R_Vend
FROM FAP, ART
WHERE FAP.COD_A = ART.COD_A
GROUP BY ART.COD_A, ART.Peso
ORDER BY R_Peso;
```

Osservazione sulla clausola `GROUP BY`: L'attributo `Peso` è stato aggiunto alla `GROUP BY` poiché compare nella `SELECT` e non all'interno di una funzione di aggregazione. Tuttavia, come sottolineato a lezione, dato un articolo esiste uno e un solo peso associato; di conseguenza, l'aggiunta di questo attributo alla `GROUP BY` non altera in alcun modo la cardinalità o la semantica dei gruppi generati (si ha sempre un gruppo per ogni singolo prodotto). Inoltre, è importante distinguere l'ordinamento per il ranking dal partizionamento: *ranking in funzione del peso* significa che il peso fa da criterio di ordinamento (`ORDER BY ART.Peso` all'interno della clausola `OVER`), non di partizionamento.

Tabella 6.2: Risultato del doppio ranking

COD_A	Peso	Qtot	R_Peso	R_Vend
A1	12	1900	1	5
A5	12	1100	1	2
A4	14	1300	3	3
A2	17	300	4	1
A3	17	4500	4	6
A6	19	1300	6	3

6.2.3 Selezione Top-N nel Ranking (Utilizzo di Table Function)

Spesso l'obiettivo aziendale non è analizzare un intero listino di 30.000 prodotti, ma restringere l'attenzione ai "Top N" (es. i 2 articoli più venduti) per evitare sprechi di risorse e migliorare la leggibilità.

Tuttavia, non è possibile filtrare direttamente il risultato di una funzione di rank inserendo una clausola del tipo `WHERE RankVendite <= 2` all'interno della stessa query. Il motivo, chiarito dal professore, risiede nell'**ordine di esecuzione delle istruzioni SQL**: la clausola `WHERE` valuta i record estratti prima della generazione delle colonne della `SELECT`, ma le funzioni finestra (come il rank) vengono processate proprio all'interno della `SELECT`.

La soluzione consiste nell'utilizzare le **Table Function** (o *inline views*): una tabella temporanea generata localmente e a runtime, fornendo un'istruzione SQL come sorgente dati all'interno della clausola `FROM` della query principale.

```sql
SELECT * FROM
  (
   SELECT COD_A, SUM(Q),
          RANK() OVER (ORDER BY SUM(Q)) AS RankVendite
   FROM FAP
   GROUP BY COD_A
  )
WHERE RankVendite <= 2;
```

Quando la query esterna elabora la clausola `FROM`, avvia l'esecuzione della sottointerrogazione. Questa genera una tabella virtuale provvista di tre colonne (tra cui `RankVendite`). A quel punto, la query esterna può regolarmente filtrare i dati applicando la `WHERE` sull'attributo precalcolato.

6.3 Ulteriori Funzioni per Finestre di Calcolo

Oltre alle tradizionali metriche di aggregazione e ai rank, Oracle rende disponibili ulteriori funzioni per caratterizzare e isolare i dati all'interno delle partizioni.

6.3.1 `ROW_NUMBER`

La funzione `ROW_NUMBER()` assegna un numero progressivo sequenziale ad ogni riga all'interno della partizione, basandosi sul criterio di ordinamento specificato.

Esempio *Partizionare gli articoli in base alla tipologia ed enumerare in modo progressivo i dati all'interno di ogni partizione. All'interno di ogni partizione i dati sono ordinati in base al peso.*

```sql
SELECT Tipo, Peso,
       ROW_NUMBER() OVER (
           PARTITION BY Tipo
           ORDER BY Peso
       ) AS RowNumberPeso
FROM ART;
```

Differenza tra `ROW_NUMBER` e `RANK`: L'aspetto più rilevante discusso in aula è il comportamento in caso di **pari merito**. A differenza di `RANK` e `DENSE_RANK` (che assegnerebbero alle tuple con stesso peso il medesimo punteggio), `ROW_NUMBER` non gestisce concettualmente i pari merito, ma forza un numero sequenziale. Se avessimo l'obiettivo di selezionare un numero esatto e fisso di elementi (es. i primi 5 prodotti), un filtro `<= 5` su un `RANK` in presenza di più pari merito potrebbe restituirci più di 5 righe in totale (es. tre prodotti a pari merito al 3° posto). Utilizzando `ROW_NUMBER`, si garantisce l'estrazione di esattamente 5 record, operando una discriminazione arbitraria tra gli ex aequo all'interno del gruppo.

6.3.2 `CUME_DIST`

La funzione `CUME_DIST` calcola la **funzione cumulativa di distribuzione** dei valori all'interno di un gruppo (partizione). Matematicamente, per ogni riga x, il valore di `CUME_DIST` si ottiene come:

$$\text{CUME_DIST}(x) = \frac{\text{Numero di righe che precedono o hanno lo stesso valore di } x}{\text{Numero totale di righe nella partizione } (N)}$$

Esempio esplicativo Prendendo come riferimento una singola partizione generata per gli articoli di tipo "Vite" (contenente $N = 10$ tuple totali), ordinate per peso:

- Se esiste 1 solo record con peso 12, il suo `CUME_DIST` sarà $1/10 = 0.1$ (ovvero, il 10% dei record ha un peso minore o uguale a 12).
- Vi è 1 record con peso 14. Cumulando il precedente si arriva alla seconda posizione: $2/10 = 0.2$.
- Vi sono **quattro** record a pari merito con peso 16. La funzione valuta tutti i valori minori **o uguali**, includendo di fatto tutto il blocco dei pari merito. La posizione massima di questo blocco è la sesta su dieci. Pertanto, ad *ognuno* di questi 4 record verrà assegnato il valore $6/10 = 0.6$.
- I due record con peso 17 arrivano alla posizione $8 \rightarrow 8/10 = 0.8$.
- L'ultimo record di peso 20 copre il 100% della partizione $\rightarrow 10/10 = 1$.

Questa funzione risulta estremamente utile per studiare la distribuzione statistica di una determinata metrica all'interno di sottogruppi aziendali.

6.3.3 `NTILE(n)`

La funzione `NTILE(n)` viene utilizzata per suddividere i record di ogni partizione in n sottogruppi (detti anche bucket). La funzione associa ad ogni record l'identificificativo del sottogruppo di appartenenza.

Oracle cerca di distribuire il numero di tuple in maniera omogenea. Se il numero totale di righe non è perfettamente divisibile per n, le dimensioni dei bucket avranno cardinalità il più possibile uniformi (con tolleranza ± 1).

Esempio *Partizionare in base alla tipologia, ordinare in base al peso, e suddividere in 3 sottogruppi.*

```
SELECT Tipo, Peso,
      NTILE(3) OVER (
          PARTITION BY Tipo
          ORDER BY Peso
      ) AS Ntile3Peso
FROM ART;
```

Riprendendo la partizione "Vite" di $N = 10$ tuple, per suddividerla in 3 gruppi occorrono dimensioni che, sommate, diano 10. Il motore SQL allocherà i primi 4 record al gruppo 1, i successivi 3 al gruppo 2 e gli ultimi 3 al gruppo 3. Ad ogni record viene assegnato l'identificativo 1, 2 o 3, permettendo facilmente selezioni basate su fasce (ad esempio, isolare il "terzo medio" dei dati). Se una partizione possiede meno tuple del valore n (es. partizione "Barra" ha un solo record), verrà generato solamente il sottogruppo 1.

Capitolo 7

Data Lakes

Capitolo 8

Data Mining Process

Capitolo 9

Data Preparation

Capitolo 10

Data Mining: Association Rules

Capitolo 11

Data Mining: Classificazione

Capitolo 12

Data Mining: Clustering

Capitolo 13

ESERCIZI

Capitolo 14

Interrogazioni Analitiche in SQL Esteso: Esercitazione Pratica

Questo capitolo è dedicato allo svolgimento di una serie di interrogazioni analitiche utilizzando le estensioni di SQL (come quelle supportate da Oracle), con particolare attenzione all'utilizzo delle funzioni di finestra (*window functions*) per il calcolo di ranking e incassi cumulativi.

14.1 Osservazione preliminare: Ordinamento e selezione degli ultimi elementi

Prima di affrontare l'esercitazione sul Data Warehouse, è utile fare una considerazione tecnica emersa durante la lezione riguardo alla selezione degli ultimi elementi di una classifica (ad esempio, gli ultimi 10 elementi).

Un approccio per ottenere questo risultato consiste nell'ordinare i dati in modo inverso rispetto al criterio logico della classifica (ad esempio, se la classifica normalmente è decrescente, si ordinano i dati in modo crescente, o viceversa) e successivamente estrarre i primi 10 record. Dal punto di vista del motore di ricerca e del recupero del dato, questo metodo è molto efficiente e corretto. Tuttavia, bisogna prestare attenzione alla **visualizzazione**: estraendo i record con un ordinamento inverso, l'elenco risulterà capovolto (es. l'elemento in decima posizione apparirà per primo, seguito dal nono, dall'ottavo, e così via).

La logica di recupero dei dati è quindi separata dalla logica di visualizzazione. Se c'è la necessità di conoscere la posizione assoluta (il *rank* esatto) di quegli elementi all'interno della totalità dei dati, sarà necessario effettuare un calcolo aggiuntivo sul numero totale dei record (o sui valori distinti assunti dall'attributo di ranking).

14.2 Analisi dello Schema del Data Warehouse

Le interrogazioni che andremo a risolvere si basano su uno schema di Data Warehouse strutturato secondo il Dimensional Fact Model. Lo schema è composto da una tabella dei fatti centrale e da quattro dimensioni.

Le tabelle a disposizione sono le seguenti:

- **Cliente** (`CodCliente`, `Cliente`, `Provincia`, `Regione`)
- **Categoria** (`CodCat`, `Categoria`)
- **Agente** (`CodAgente`, `Agente`, `Agenzia`)
- **Tempo** (`CodTempo`, `Mese`, `Trimestre`, `Semestre`, `Anno`)

- **Fatturato** (`CodTempo`, `CodCliente`, `CodCatArticolo`, `CodAgente`, `TotFatturato`, `NumArticoli`, `TotSconto`)

14.2.1 Gerarchie e particolarità delle Dimensioni

È fondamentale analizzare le gerarchie presenti all'interno delle dimensioni prima di procedere con le interrogazioni:

- **Dimensione Cliente:** Presenta la gerarchia canonica in cui a ogni cliente è associata una sola provincia, e ogni provincia appartiene a una sola regione.
- **Dimensione Tempo:** Segue una gerarchia rigida in cui dato un mese esiste un solo trimestre, che a sua volta è collegato univocamente a un semestre e, infine, all'anno.
- **Dimensione Agente:** Ogni agente è associato a una e una sola agenzia, mentre la stessa agenzia gestisce più agenti.
- **Dimensione Categoria:** Questa tabella possiede un solo attributo descrittivo (`Categoria`) oltre alla chiave. Nella teoria del Data Warehousing, una dimensione con un solo attributo viene definita **dimensione degenere** (*Junk Dimension*). Tuttavia, in questo schema è stata modellata come una tabella dimensionale tradizionale: questa scelta progettuale è giustificata dall'ipotesi che, in futuro, si possano aggiungere nuovi attributi o creare nuove gerarchie associate alla categoria.

La tabella dei fatti **Fatturato** ha una chiave primaria composta dai codici delle quattro dimensioni (`CodTempo`, `CodCliente`, `CodCatArticolo`, `CodAgente`). Le **metriche** su cui possiamo eseguire i calcoli sono tre: il totale del fatturato (`TotFatturato`), il numero degli articoli venduti (`NumArticoli`) e il totale dello sconto (`TotSconto`).

14.3 Esercizio 1: Calcolo di aggregati e Ranking Globale

14.3.1 Testo della richiesta

Visualizzare per ogni categoria di articoli:

1. la categoria;
2. la quantità totale fatturata per la categoria in esame;
3. il fatturato totale associato alla categoria in esame;
4. il rank della categoria in funzione della quantità totale fatturata;
5. il rank della categoria in funzione del fatturato totale.

14.3.2 Ragionamento e Passaggi Logici

Per rispondere alla query, dobbiamo incrociare i dati tra la tabella dei fatti `Fatturato` e la dimensione `Categoria`. La condizione di join sarà `Fatturato.CodCatArticolo = Categoria.CodCat`.

Poiché la richiesta richiede dati aggregati "per ogni categoria", è necessario applicare un'operazione di raggruppamento (`GROUP BY Categoria`). Le metriche base sono facilmente ricavabili applicando la funzione `SUM()` agli attributi `NumArticoli` (per la quantità totale) e `TotFatturato`.

Il calcolo del *rank* richiede l'utilizzo delle funzioni di finestra (`RANK() OVER`). È importante chiederci se sia necessario partizionare i dati. In questo specifico caso, **non serve il partizionamento** (`PARTITION BY`). Il motivo risiede nel fatto che la query base genera un singolo record per ogni categoria. Pertanto, noi vogliamo assegnare una posizione in classifica a ciascuna categoria calcolata rispetto a *tutte* le altre. Non essendoci un livello gerarchico superiore a "categoria" nel nostro schema rispetto al quale calcolare classifiche separate, la finestra di calcolo si applica all'intero set di dati raggruppato.

Viene richiesto di calcolare il rank due volte, con due criteri di ordinamento diversi:

- `ORDER BY SUM(NumArticoli) DESC`: per assegnare la prima posizione alla categoria con la maggiore quantità di articoli venduti. L'uso di `DESC` assicura che il valore più alto abbia rank 1.
- `ORDER BY SUM(TotFatturato) DESC`: per calcolare la classifica in base al ricavo.

14.3.3 Codice SQL

```sql
SELECT
    C.Categoria,
    SUM(F.NumArticoli) AS QuantitaTotale,
    SUM(F.TotFatturato) AS FatturatoTotale,
    RANK() OVER (
        ORDER BY SUM(F.NumArticoli) DESC
    ) AS RankPerQuantita,
    RANK() OVER (
        ORDER BY SUM(F.TotFatturato) DESC
    ) AS RankPerFatturato
FROM
    Fatturato F,
    Categoria C
WHERE
    F.CodCatArticolo = C.CodCat
GROUP BY
    C.Categoria;
```

14.4 Esercizio 2: Ranking con Partizionamento

14.4.1 Testo della richiesta

Visualizzare per ogni provincia:

1. la provincia;
2. la regione della provincia;
3. il fatturato totale associato alla provincia;
4. il rank della provincia in funzione del fatturato totale, separato per regione.

14.4.2 Ragionamento e Passaggi Logici

Le informazioni geografiche (Provincia e Regione) si trovano nella dimensione `Cliente`. Bisognerà quindi effettuare un join tra `Fatturato` e `Cliente` tramite l'attributo `CodCliente`.

La query richiede di considerare l'aggregazione a livello di provincia. È necessario usare la clausola `GROUP BY Provincia`. Sorge spontanea una domanda: possiamo inserire anche la `Regione` nella clausola `SELECT` e nel `GROUP BY`? La risposta è **sì**. Data una provincia, esiste infatti una e una sola regione di appartenenza (relazione funzionale $N : 1$). Inserire `Regione` nel `GROUP BY` non altera la granularità del risultato (che rimane a livello di provincia), ma permette di proiettare la regione nella selezione.

Il concetto semantico del Rank rispetto al raggruppamento: Durante la lezione il professore ha precisato un dettaglio semantico. Quando eseguiamo un `GROUP BY` su un singolo attributo (es. solo per provincia), possiamo dire di "assegnare il rank all'attributo Provincia". Quando però raggruppiamo per più attributi congiuntamente, l'espressione più formalmente corretta è "assegnare un rank ad ogni record generato dalla query", poiché la riga rappresenta la combinazione degli attributi raggruppati.

Il calcolo del rank in questo esercizio introduce la clausola `PARTITION BY`. Viene esplicitamente richiesto che il rank sia "separato per regione". Ciò significa che per ogni regione si aprirà una nuova finestra di calcolo, e all'interno di quella regione la provincia con il fatturato più alto assumerà rank 1.

14.4.3 Codice SQL

```sql
SELECT
    C.Provincia,
    C.Regione,
    SUM(F.TotFatturato) AS FatturatoTotale,
    RANK() OVER (
        PARTITION BY C.Regione
        ORDER BY SUM(F.TotFatturato) DESC
    ) AS RankProvinciaPerRegione
FROM
    Fatturato F,
    Cliente C
WHERE
    C.CodCliente = F.CodCliente
GROUP BY
    C.Provincia,
    C.Regione;
```

14.5 Esercizio 3: Ranking Multi-Dimensionale

14.5.1 Testo della richiesta

Visualizzare per ogni provincia e mese:

1. la provincia;
2. la regione della provincia;
3. il mese;
4. il fatturato totale associato alla provincia nel mese in esame;
5. il rank della provincia in funzione del fatturato totale, separato per mese.

14.5.2 Ragionamento e Passaggi Logici

Questa query estende la precedente aggiungendo la dimensione temporale. L'interrogazione richiederà di unire tre tabelle: `Fatturato`, `Cliente` e `Tempo`.

Le entità fondamentali su cui basare il raggruppamento dettate dalla query sono **Provincia** e **Mese**. Come best-practice suggerita, quando si costruisce il raggruppamento, è utile inserire prima questi due attributi principali e aggiungere solo successivamente le eventuali dipendenze funzionali, come la `Regione`. Nonostante in SQL l'ordine degli attributi nel `GROUP BY` sia ininfluente sull'esecuzione, scriverli in questo modo aiuta lo sviluppatore ad avere ben chiara la granularità concettuale della query.

Il rank richiesto deve essere ricalcolato separatamente in ogni diverso periodo temporale. Di conseguenza, la finestra di calcolo deve essere definita indicando `PARTITION BY Mese`.

14.5.3 Codice SQL

```sql
SELECT
    C.Provincia,
    T.Mese,
    C.Regione,
    SUM(F.TotFatturato) AS FatturatoTotale,
    RANK() OVER (
        PARTITION BY T.Mese
        ORDER BY SUM(F.TotFatturato) DESC
    ) AS RankProvinciaPerMese
FROM
    Fatturato F,
    Cliente C,
    Tempo T
WHERE
    C.CodCliente = F.CodCliente
    AND F.CodTempo = T.CodTempo
GROUP BY
    C.Provincia,
    T.Mese,
    C.Regione;
```

14.6 Esercizio 4: Calcolo dell'Incasso Cumulativo (Running Total)

14.6.1 Testo della richiesta

Visualizzare per ogni regione e mese:

1. la regione;
2. il mese;
3. il fatturato totale associato alla regione nel mese in esame;
4. l'incasso cumulativo al trascorrere dei mesi, separato per ogni regione;
5. l'incasso cumulativo al trascorrere dei mesi, separato per ogni anno e regione.

14.6.2 Ragionamento e Passaggi Logici

Questa interrogazione sposta il focus dalle classifiche alle **somme cumulative** nel tempo, che si realizzano combinando la funzione di aggregazione `SUM()` con una finestra basata sull'istruzione `ROWS UNBOUNDED PRECEDING`.

Per prima cosa, eseguiamo i join necessari (`Fatturato`, `Cliente`, `Tempo`) e definiamo la clausola `GROUP BY`. Gli attributi di raggruppamento principali sono `Regione` e `Mese`. Tuttavia, prestando attenzione all'ultima richiesta, notiamo che ci serve differenziare il calcolo cumulativo anche per **Anno**. Sappiamo che ogni `Mese` definisce univocamente un `Anno`; pertanto, dobbiamo obbligatoriamente aggiungere l'attributo `Anno` nel `GROUP BY` affinché esso diventi disponibile nella `SELECT` (e conseguentemente nella `PARTITION BY`).

Vediamo nel dettaglio i due incassi cumulativi richiesti:

- **Incasso cumulativo separato per regione:** La formula richiede di sommare progressivamente i valori aggregati mensili. Si scrive `SUM(SUM(TotFatturato))`. Il partizionamento avviene solo rispetto a `Regione`. L'ordinamento, affinché l'accumulazione abbia senso cronologico, si effettua con `ORDER BY Mese`. Per sommare tutto dalla prima registrazione disponibile in quella partizione fino al mese corrente, si aggiunge la direttiva `ROWS UNBOUNDED PRECEDING`.

- **Incasso cumulativo separato per regione e per anno:** La logica è identica, ma in questo caso si richiede che all'inizio di ogni nuovo anno accertato il contatore "si azzeri". Per implementare questo comportamento, è sufficiente aggiungere l'attributo temporale alla partizione: `PARTITION BY Regione, Anno`. In questo modo, gennaio non sommerà i mesi dell'anno precedente, perché inizierà una nuova partizione.

Interpretazione fisica e temporale (Approfondimento in Lezione): Rispondendo alla perplessità di uno studente su come visualizzare mentalmente le due colonne generate in tempo reale, il professore ha fornito la seguente delucidazione. Supponiamo di avere un Data Warehouse popolato con i dati degli anni compresi tra il 2024 e il 2026:

- La colonna del *primo cumulativo* (solo per regione) presenterà un unico blocco che cresce costantemente per ogni regione. A Gennaio 2024 avremo l'incasso di un mese. A Febbraio 2024 la somma di due. Arrivando a Gennaio 2025, in questa colonna troveremo la somma di **tutti i 12 mesi del 2024 più Gennaio 2025**.

- La colonna del *secondo cumulativo* (regione e anno), invece, una volta raggiunto l'inizio del nuovo anno (Gennaio 2025), perderà memoria del 2024. Il suo valore coinciderà esclusivamente con l'incasso di Gennaio 2025 per poi riniziare ad accumulare fino a Dicembre 2025.

Infine, sui mesi futuri non conclusi o senza registrazioni (es. un presunto "Ottobre 2026" se non sono state effettuate vendite), i dati semplicemente non saranno generati, poiché il sistema lavora interrogando i record con granularità mensile fisicamente memorizzati nel Data Warehouse fino a quel momento.

14.6.3 Codice SQL

```sql
SELECT
    C.Regione,
    T.Anno,
    T.Mese,
    SUM(F.TotFatturato) AS FatturatoMese,
```

```
    SUM(SUM(F.TotFatturato)) OVER (
        PARTITION BY C.Regione
        ORDER BY T.Mese
        ROWS UNBOUNDED PRECEDING
    ) AS CumulativoStoricoPerRegione,
    SUM(SUM(F.TotFatturato)) OVER (
        PARTITION BY C.Regione, T.Anno
        ORDER BY T.Mese
        ROWS UNBOUNDED PRECEDING
    ) AS CumulativoAnnuoPerRegione
FROM
    Fatturato F,
    Cliente C,
    Tempo T
WHERE
    F.CodCliente = C.CodCliente
    AND F.CodTempo = T.CodTempo
GROUP BY
    C.Regione,
    T.Anno,
    T.Mese;
```

Capitolo 15

Progettazione di un Data Warehouse: Caso Studio Magazzini

Il presente capitolo illustra il processo completo di progettazione di uno schema concettuale e logico per un Data Warehouse, partendo da un caso di studio aziendale. L'obiettivo didattico è applicare i concetti teorici per trasformare le specifiche di un committente in un modello dati ottimizzato per l'analisi (OLAP), partendo dall'ipotesi di avere a disposizione un database transazionale (OLTP) di origine da cui estrarre e aggregare le informazioni.

15.1 Metodologia di Progettazione

Prima di affrontare il caso di studio specifico, è fondamentale fissare i passi metodologici necessari per passare dalla richiesta del committente allo schema logico finale. La progettazione si articola nei seguenti punti:

1. **Identificazione dei fatti di interesse e delle misure:** Un "fatto" rappresenta il nucleo dell'analisi aziendale. Le misure (o metriche) sono le grandezze numeriche che descrivono il fatto. Nel Data Warehouse, queste metriche sono solitamente espresse tramite operatori aggregabili (come somme o conteggi). Si evita, di norma, di memorizzare valori come le medie dirette, preferendo memorizzare numeratore e denominatore (es. totale e conteggio) poiché la media non è facilmente riaggregabile al variare della cardinalità.

2. **Identificazione delle dimensioni e dei livelli minimi di aggregazione:** Le dimensioni rappresentano le prospettive (o assi) lungo le quali si vogliono "tagliare" o aggregare le metriche. Il livello minimo di aggregazione determina la granularità della tabella dei fatti (la *fact table*): definisce il livello di dettaglio massimo a cui i dati saranno analizzabili.

3. **Definizione delle gerarchie:** All'interno di ogni dimensione, gli attributi sono organizzati in gerarchie che rappresentano relazioni di tipo $1 : N$ (ad esempio, una città appartiene a una sola provincia, ma una provincia contiene più città). Comprendere queste gerarchie è vitale per costruire correttamente i rami del nostro modello concettuale.

4. **Costruzione dello schema concettuale:** Trasformazione dei requisiti in un modello grafico (es. *Star Schema* o schema a stella), che evidenzia la fact table centrale e i rami dimensionali, gestendo casistiche particolari come relazioni molti-a-molti (gestite

tramite doppi archi e tabelle ponte) o "dimensioni degeneri" (che non necessitano di una tabella dimensionale a sé stante).

5. **Definizione dello schema logico:** Traduzione dello schema concettuale in tabelle relazionali, ponendo particolare attenzione alla formattazione degli attributi (specialmente quelli temporali) per far sì che le gerarchie, che si perdono nel passaggio allo schema logico, vengano implicitamente preservate all'interno dei dati.

6. **Stima del carico di lavoro e verifica:** Controllo della validità del modello rispetto alle interrogazioni (*query*) tipiche fornite dal committente, assicurandosi che il livello di aggregazione sia sufficiente per rispondere alle richieste senza essere inutilmente eccessivo.

15.2 Analisi delle Specifiche e Identificazione dei Fatti

Il problema in esame riguarda una grande ditta italiana di elettrodomestici con magazzini distribuiti sul territorio nazionale. La dirigenza ha due necessità analitiche principali:

- **Analisi dell'uso dei magazzini:** Capire la percentuale di utilizzo in funzione della dislocazione geografica per decidere su ampliamenti, ridimensionamenti o affitti a terzi.
- **Stima dei costi delle polizze assicurative:** Analizzare la copertura contro furti e danni, i cui premi si basano sulla quantità media giornaliera di prodotti presenti e sul loro valore complessivo, per le diverse categorie e modelli.

Leggendo le specifiche, emerge subito un'entità geografica di base: *l'Italia*. Tuttavia, poiché tutti i magazzini si trovano in Italia, la Nazione rappresenta una **dimensione universo**. Mappare questa dimensione sarebbe ridondante: non faremmo mai aggregazioni per nazione in questo contesto, poiché il valore sarebbe sempre e solo "Italia". Pertanto, la scartiamo fin dall'inizio.

Sembra che ci troviamo di fronte a due fatti semanticamente distinti: l'occupazione fisica dello spazio e la valorizzazione dei prodotti per le assicurazioni. L'approccio più sicuro è modellarli inizialmente come due *fact table* separate. Successivamente, se scoprissimo che condividono esattamente le stesse dimensioni allo stesso livello di granularità, potremmo valutarne l'unione per questioni di ottimizzazione; tuttavia, mantenerle separate all'inizio è la scelta metodologica corretta.

15.3 Progettazione Concettuale: Schema a Stella

15.3.1 Fatto 1: Uso dei Magazzini

Il primo fatto di interesse riguarda le superfici. Il testo specifica che l'azienda vuole analizzare "la percentuale giornaliera di superficie libera rispetto alla superficie totale disponibile".

Identificazione delle Metriche

Le misure da tracciare nella fact table sono due:

- **Superficie libera:** È il dato che varia dinamicamente e descrive l'effettivo spazio vuoto.

- **Superficie totale disponibile:** Questo è un punto di attenzione cruciale. In molti casi, la capienza totale di un magazzino è un dato statico (o che varia molto raramente); se così fosse, sarebbe molto più efficiente modellarla come attributo della dimensione *Magazzino*, risparmiando spazio (non la si tracerebbe ogni giorno per ogni magazzino). Tuttavia, le specifiche affermano esplicitamente: *"La superficie totale disponibile per ogni magazzino può variare di giorno in giorno, in quanto alcune aree possono essere affittate a terzi o essere inagibili"*. Poiché varia giornalmente, **deve obbligatoriamente essere una metrica** tracciata giorno per giorno nella fact table.

Identificazione delle Dimensioni e Gerarchie

Le aggregazioni richieste per il Fatto 1 si basano su due dimensioni principali:

1. Dimensione Spaziale (Luogo)

L'analisi deve avvenire "in funzione del magazzino e del luogo in cui esso si trova (città, provincia, regione)". Il livello minimo di aggregazione è chiaramente il **Magazzino**. Un magazzino non si divide su più città. La gerarchia è lineare (relazione $1 : N$):

$$\text{Magazzino} \rightarrow \text{Città} \rightarrow \text{Provincia} \rightarrow \text{Regione}$$

2. Dimensione Temporale

L'analisi deve essere possibile a livello di singola **Data** (livello minimo), ma anche aggregabile per Mese, Trimestre, Quadrimestre, Semestre e Anno. Costruire la gerarchia temporale richiede attenzione affinché ogni attributo mantenga una relazione univoca verso il livello superiore. Se indicassimo il "Mese" semplicemente con un numero da 1 a 12, non sapremmo a quale anno appartiene, rompendo la gerarchia logica in fase di traduzione tabellare.

Per preservare la gerarchia, i dati devono essere formattati includendo l'eredità del padre. Ad esempio:

- Data: 12-10-2026
- Mese: 10-2026
- Trimestre: 3-2026 (indica il terzo trimestre dell'anno)
- Anno: 2026

La struttura gerarchica del tempo presenta alcune diramazioni dal Mese in poi:

- **Dal Mese al Trimestre:** Esiste una relazione $1 : N$ diretta (un mese appartiene a un solo trimestre).
- **Dal Mese al Quadrimestre:** Esiste una relazione $1 : N$ diretta (un mese appartiene a un solo quadrimestre).
- **Tra Trimestre e Quadrimestre: Non esiste** alcuna gerarchia. Un quadrimestre non è formato da un numero intero di trimestri. Quindi Mese diverge in due rami separati.
- **Dal Trimestre al Semestre:** Due trimestri compongono esattamente un semestre, quindi esiste una freccia gerarchica tra Trimestre e Semestre.
- **Verso l'Anno:** Quadrimestre e Semestre convergono entrambi nell'Anno.

Osservazione aggiuntiva: Come spiegato a lezione, si crea in questo modo un'abbondanza di dati ripetuti. In un database relazionale normale si ricaverebbe il mese dalla data per via programmatica. In un Data Warehouse questa ridondanza è voluta per accelerare le

aggregazioni in lettura tramite indici. Inoltre, se fosse richiesto fare analisi comparative mensili prescindendo dall'anno (es. "Tutti gli incassi di ottobre di ogni anno"), sarebbe necessario aggiungere una dimensione "Mese dell'anno" (valori 1-12) collegata direttamente alla Data e non dipendente dall'Anno.

Figura 15.1: Schema concettuale a stella per il fatto "Uso Magazzini", con evidenza della gerarchia temporale divergente.

15.3.2 Fatto 2: Stima dei Costi delle Polizze

Il secondo obiettivo riguarda la valutazione del contenuto dei magazzini per scopi assicurativi.

Identificazione delle Metriche

I premi assicurativi sono calcolati considerando il numero di prodotti presenti e il loro valore complessivo. Poiché l'analisi mira a descrivere queste quantità su base aggregata per tipologia e data, le metriche della *fact table* saranno:

- **Numero di prodotti:** Conteggio aggregato dei prodotti presenti nel magazzino in un determinato giorno, per un determinato modello.
- **Valore prodotti:** Il valore economico aggregato associato a tale quantità.

Il database OLTP di origine conterrà verosimilmente le tracce del singolo prodotto identificato dal suo numero di serie, ma in fase di estrazione verso il Data Warehouse aggregheremo questi dati in un unico record (tupla) per Magazzino, Data e Modello. Le richieste parlano di valutazioni medie, ma nel modello di base salviamo somme e conti da cui deriveremo le medie a livello logico/applicativo.

Identificazione delle Dimensioni

Leggendo i requisiti, notiamo che l'analisi dei prodotti deve avvenire in funzione di:

- **Tempo (singola data):** La dimensione temporale è la medesima del Fatto 1.
- **Luogo (magazzino):** La dimensione spaziale è la medesima del Fatto 1.
- **Prodotto (modello e categoria):** Viene introdotta una nuova dimensione legata al bene fisico. Il livello minimo di aggregazione richiesto non è il singolo oggetto specifico, ma il **Modello** (es. Computer Acer modello Y). I modelli sono poi raggruppati in **Categorie** (es. scope elettriche, impastatrici).

Modellazione della relazione Modello - Categoria: Durante la lezione è stato posto un quesito fondamentale su come modellare la relazione tra Modello e Categoria. Sono state prospettate due opzioni grafiche e logiche:

- *Opzione A (Rappresentazione orizzontale o con doppio arco - Molti a Molti):* Implica che un modello possa possedere simultaneamente più categorie o caratteristiche non mutualmente esclusive (come un *one-hot encoding* se le opzioni sono poche e note, o una tabella ponte se sono illimitate).
- *Opzione B (Gerarchia Lineare* $1 : N$*):* Implica che a un modello sia assegnata una e una sola categoria.

Nel nostro dominio di riferimento, un modello di elettrodomestico appartiene tipicamente a una singola categoria funzionale. Esiste quindi una corrispondenza $1 : N$ (una categoria racchiude molti modelli, ma un modello ha una sola categoria). Pertanto, la modellazione corretta è quella lineare gerarchica:

$$\text{Modello} \rightarrow \text{Categoria}$$

Figura 15.2: Schema concettuale a stella per il fatto "Prodotti". Le dimensioni Tempo e Luogo sono in comune con la fact table precedente.

15.4 Verifica del Carico di Lavoro (Query Tipiche)

Un passaggio cruciale post-progettazione concettuale è la validazione del modello tramite il confronto con le interrogazioni frequenti fornite dal committente (slide "Interrogazioni frequenti"). Questa operazione serve per accertarsi che: 1. I livelli minimi di aggregazione (la granularità) scelti permettano di rispondere alle domande. 2. Non vi siano dimensioni o livelli inseriti inutilmente (sovra-raffinamento).

Analizzando brevemente alcune richieste:

- *Query a:* Parla di "coppia (magazzino, data)". Il nostro livello minimo per spazio e tempo lo supporta. Cita inoltre la "settimana precedente". La "settimana" non era esplicitamente definita nella gerarchia temporale prima di questa riga; pertanto è un attributo che deve essere gestito e inserito nella gerarchia della Data (divergente rispetto al mese, in quanto le settimane si sovrappongono ai mesi).
- *Query b, c, d, e, f:* Chiedono aggregazioni per città, provincia, regione a livello di mese o semestre. Poiché tutte queste dimensioni risiedono a livelli gerarchici superiori rispetto al nostro livello minimo, il sistema sarà perfettamente in grado di effettuare il raggruppamento (GROUP BY) partendo dai livelli minimi (magazzino e data).

Tutte le informazioni, comprese le suddivisioni per modello, risultano coperte dalle fact table ipotizzate.

15.5 Progettazione Logica: Dallo Schema Concettuale alle Tabelle

L'ultimo passo è il passaggio al modello relazionale. Nello schema logico avremo un insieme di tabelle collegate tra loro dalle chiavi. Una *fact table* si costruisce componendo una chiave primaria (Primary Key) multi-attributo formata dalle chiavi esterne (Foreign Key) che puntano ai livelli minimi delle dimensioni, seguite dai campi delle metriche.

Un principio cardine affrontato a lezione riguarda gli identificativi (ID). Definiremo una chiave surrogata (un ID numerico incrementale) per ogni dimensione al suo livello minimo. Questo ID avrà una corrispondenza $1 : 1$ con il valore del livello di aggregazione minimo, ma garantirà un'efficienza drastica nelle operazioni di *join* rispetto all'uso di stringhe di testo come chiavi primarie.

15.5.1 Dimensioni Condivise (Conformed Dimensions)

Osservando i due schemi a stella generati, emerge un dato fondamentale: le gerarchie "Luogo" e "Tempo" sono identiche per entrambi i fatti. Condividono sia il significato semantico sia

i valori sottostanti (i magazzini aziendali e i giorni di calendario sono gli stessi). Invece di duplicare queste dimensioni, le creeremo una volta sola, realizzando delle **dimensioni conformate**. Il Data Warehouse finale sarà composto da 5 tabelle totali (2 Fact Table e 3 Tabelle Dimensionali), collegate come descritto di seguito.

15.5.2 Definizione delle Tabelle Relazionali

1. Tabella Dimensionale: Tempo
Raccoglie l'intera gerarchia temporale. La gerarchia, come precedentemente discusso, viene incorporata nel valore delle colonne (es. il campo trimestre conterrà "1-2026", non solo "1").

```
Tempo (
    ID_tempo (PK),
    data,
    settimana_anno,
    mese,
    trimestre,
    quadrimestre,
    semestre,
    anno
)
```

2. Tabella Dimensionale: Magazzino
Raccoglie la dislocazione geografica.

```
Magazzino (
    ID_magazzino (PK),
    nome_magazzino,
    città,
    provincia,
    regione
)
```

3. Tabella Dimensionale: Prodotto
Raccoglie le caratteristiche del bene.

```
Prodotto (
    ID_modello (PK),
    nome_modello,
    categoria
)
```

4. Fact Table 1: Uso Magazzini
La chiave primaria è composta dalle chiavi esterne delle dimensioni coinvolte nel Fatto 1.

```
F_Uso_Magazzini (
    ID_magazzino (PK, FK),
    ID_tempo (PK, FK),
    superficie_libera,
    superficie_totale
)
```

5. Fact Table 2: Prodotti (Polizze)
La chiave primaria include anche la dimensione Prodotto, poiché in uno stesso giorno, in uno stesso magazzino, convivono più modelli di elettrodomestici.

```
F_Prodotti_Polizze (
    ID_magazzino (PK, FK),
    ID_tempo (PK, FK),
    ID_modello (PK, FK),
    numero_prodotti,
    valore_prodotti
)
```

In base ai dati volumetrici forniti nelle slide (es. 10 anni di dati storici, ∼500 magazzini, ∼100 modelli), sarà poi possibile, in una fase successiva di amministrazione del database, stimare la cardinalità (numero di righe) di ciascuna di queste tabelle per definire requisiti di storage e prestazioni.